

含光伏与储能微网的分层购电与动态调整策略

摘 要

针对含光伏与储能微网中提前购电承诺、预测偏差与合同调整相互耦合的问题，本文以状态增量描述储能调度，在**统一能量与现金账本**下，构建从确定性规划、场景反馈、合同追索到变价结算的分层决策框架，分别处理可实施调度、不确定性响应与费用来源解释。

针对问题一，利用充放电损耗的凸分段线性结构，将含物理互斥的调度问题转化为**最优值等价的状态增量线性规划**，并证明可从库存序列恢复互斥动作。题给预测日最优费用为 35126.95 元，较无储能降低 26.90%；最优面分析进一步区分指定购电量的确定性与整日同费用调度的非唯一性。

针对问题二，构造**历史配对残差场景与日内反馈**，以两日规划的影子价格连接跨日库存价值，在零点额度封存后滚动调整储能。2025 年 2 月至 12 月的 334 日连续回放中，F-fast 现金费用为 1377.60 万元；相对场景固定调度，普通计划费增加 0.57 万元，紧急购电费减少 51.81 万元，费用改善主要来自缺口费用下降。

针对问题三，按当天剩余窗口校准融合预报，在零点设置一次 6 点信号追索，并于 6、12、18 点分段改约。由不对称改约费与紧急购电费，推得固定储能合同子问题的**70%/90% 分位条件**，解释合同保持、增购与退订的边界。已比较配置中，完整更新方案现金费用最低，为 1350.32 万元，较问题二减少 27.29 万元；同时保留 18 点官方预报误差高于历史预测而完整策略费用较低的结果。

针对问题四，在当日价格日前已知条件下，沿用问题二、问题三策略的现金费用分别为 1443.33、1414.83 万元；以前日价格预测时，分别为 1460.35、1432.01 万元。通过将费用变化分为**均价项、协方差项与重新决策项**，识别出价格与计费权重的正向跨日协变是换价增费的主要来源。

通过**显式互斥对照、逐格账本核验与参数扫描**检查模型与实现，并以未来信息扰动核对可用信息边界。年度结论限于已说明量测条件下的回顾性策略比较，现金费用与末库存共同评价。

关键词：微网购电；状态增量线性规划；残差场景反馈；合同分位条件；电价协方差

一、 问题重述

王成山、李鹏讨论了分布式发电、微网与智能配电网的发展关系 [1]。光伏出力与居民负载在日内并不同步，储能可以转移电量，却同时受到容量、功率和转换效率的限制。微网运营者还需提前承诺购电量：承诺不足将产生高价紧急购电，承诺过量则形成已经付费但未调用的额度。因此，购电安排既取决于能量供需，也取决于信息到达时刻和合同调整规则。

题目在同一微网系统上逐步增加不确定性和可调整的决策。问题一给定单日负载、光伏预测与分时价格，要求在首尾储电量相等的条件下安排购电和充放电，使全天费用最小。问题二将运行延伸到全年：每天零点根据此前信息确定当天购电额度，运行中由储能和高价紧急电共同处理实际偏差。问题三允许在光伏预报更新后修订尚未交付的合同，需同时考虑退订、增购与储能调度。问题四进一步引入日际电价变化，分别延伸问题二、三的策略，比较不同价格信息条件下的费用。

四问均需给出题定时段的购电与储能安排，并填写五份原始结果模板。下文以完整运行费用、库存及其构成为主要比较对象；指定日期和时段的结果分别列于对应问题的结果小节，与相应策略的连续轨迹保持一致。完整结果文件的使用方式见附录A。

二、 问题分析

四问可以围绕两个相互连接的对象展开：储能路径决定电量何时使用，购电合同决定这些电量如何付费。能量平衡和设备约束贯穿始终，后续问题改变的是信息到达方式及决策的可调整范围。因此，模型从可核对的确定性基线出发，逐步引入日前承诺、日内反馈、合同追索和变动电价。

图1给出四类设备及母线能量关系。利用光伏 v 、实际购电 $a + e$ 和电池放电 b 共同供给负载 L 及充电 c ；弃光 $w = V - v$ 留在母线之外。合同额度决定费用，未调用额度不形成实际能流。

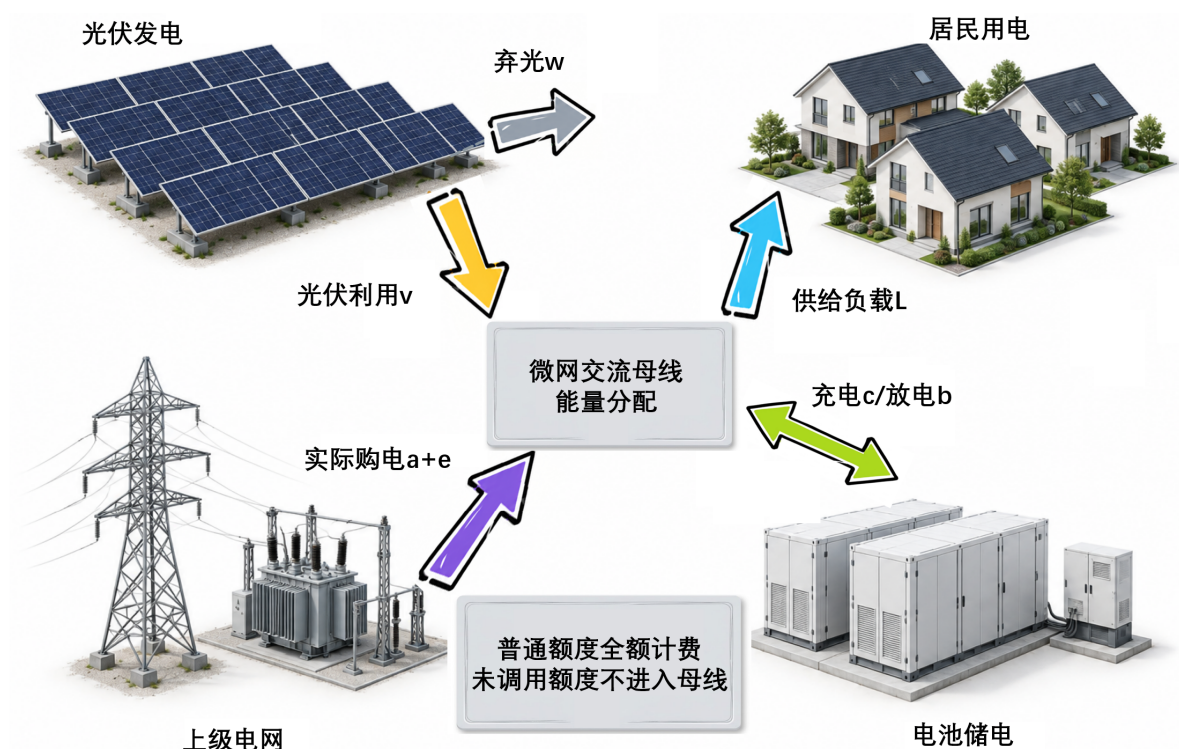


图1 微网结构与能量流示意。设备为 AI 辅助生成的通用示意，非真实场站照片；箭头表示方向，不表示数量，充放电在同一十分钟格互斥

在确定性问题中，充、放电两个非负变量的乘积互斥容易引入离散选择。以电池内部储电量的增减为变量，则动作方向由增量符号确定，母线耗电成为凸分段线性函数。问题一由此得到连续线性规划，并通过动作恢复证明其物理可实施性；这一表示随后用于场景规划和滚动控制。

进入问题二，零点看到的是预测，交付时承担的却是真实缺口。普通额度全额收费且不可退回，计划不足与过量的代价不同。历史配对残差将这种风险带入日前计划；日末库存的影子价格把次日电量需求带回当日目标；在额度封存后，日内反馈仅改变储能动作。点预测固定调度、场景固定调度和场景反馈调度的比较，依次呈现这两层调整带来的完整运行差异。

问题三开放了另一层调整：新预报到达后，未来合同额度也可以重新决定。模型须在更新后的剩余需求预测、增退购费用和后续库存之间权衡，并区分本次正式生效的合同与留待后次发布的远期候选。零点规划还可预留未来信号分支，使日前承诺为随后调整留下余地。问题四则保持这两类决策结构，将决策价格与交付结算价格分开；通过固定动作换价，辨认费用变化来自时段均价、价格与计费权重的协变，还是策略重新调整。

上述递进关系与日内时序见图2。全年实测用于构造实际账本，各次决策仅使用已揭示量测及已经发布的未来预报，普通合同额度与实际调用电量始终分账。



图 2 四问递进关系与日内信息时间轴

图中的当期量测用于 F-fast 的区间反馈近似：假设能量管理系统具有区间内快速测量和闭环控制能力，以当前十分钟平均量近似运行条件。滞后参考将预测参考后移一格，执行保护仍使用当前量测。两种控制方式的量测范围据此区分；未来实测始终位于决策可用信息之外。

三、模型假设与符号说明

3.1 数据与统一计量

附件 1 提供代表日负载、光伏预测和固定分时电价，附件 2 提供 2025 年全年十分钟实测，附件 3 提供带发布时间的光伏预报，附件 4 提供日际变化的电价，附件 5 为结果模板。原附件保持不变，模型计算、历史残差和回放结果均由上述资料派生。

每天划分为 $T = 144$ 个时段，时长为 $\Delta t = 1/6$ h，输入功率乘以 Δt 转换为区间电量。本文将附件时标解释为区间末端，例如 00:10 对应 00:00–00:10，索引 $t = 60$ 对应 10:00–10:10。结果按原模板位置填写，模板文字与附件时标的对应关系见表 22。

母线电量与电池内部储电量分别计量，二者通过充放电效率联系。主要符号见表 1；无特别说明时省略日期下标 d 。

表 1 主要符号及单位

符号	单位	含义
d, t, s	—	日期、十分钟时段及场景索引。
L, V	kWh	母线负载与可用光伏电量。
g, q, m	kWh	确定性购电量、零点原计划及最终生效合同额度。
a, u, e	kWh	实际调用额度、已付费未调用额度及紧急购电。
c, b, v, w	kWh	母线侧充电、放电、利用光伏及弃光。
S, x	kWh	电池内部储电量及相邻时段的储电增量。
$p, \tilde{p}$	元/kWh	实际结算价格与决策价格。
$\lambda, \bar{S}$	元/kWh; kWh	日末库存估值斜率与饱和阈值。

3.2 物理假设与评价设置

物理假设与费用边界归纳如下，设备数值见表2。

- (1) 充、放电效率均为单程 0.9，往返效率为 0.81。
- (2) 系统不向电网售电；允许放弃未利用光伏，不允许主动弃置电池放出的能量。
- (3) 同一十分钟时段充放电互斥，紧急购电可以用于满足负载及充电需求。
- (4) 优化目标为购电现金费用，不计设备投资和电池寿命成本。

表 2 储能基础参数与边界

参数	数值	使用方式
额定容量	12000 kWh	基础设备容量，SOC 定义为 $S/12000$ 。
内部储电范围	1200–10800 kWh	对应 10%–90% 的 SOC 范围。
充、放电功率上限	5000 kW	每格母线电量上限 $B = 5000/6$ kWh。
充、放电效率	$\eta_c = \eta_d = 0.9$	连接母线电量与内部储电变化。
初始内部储电量	6000 kWh	问题一采用循环终端；连续运行用于一月预热。

问题一使用题给预测日，其余问题在一月预热后，对二月至十二月共 334 日连续回放。历史残差用于有限场景近似，日末库存价值用于跨日决策衔接；各自定义见第五章。价格信息在第七章分别设定，策略比较与参数形成过程的解释范围集中于第八章。

四、 问题一：确定性条件下的购电与储能调度

给定同一天的负载、光伏预测和分时电价，购电与储能需要共同满足供需平衡及首尾库存相等的要求。光伏富余、电价差与转换损耗决定电量是否值得跨时段转移，库存和功率边界则限制这一转移。为同时表达这些关系，本文以内部储能电量的变化连接物理动作与购电费用。

4.1 物理模型与状态增量

记 g_t 为确定性预测日的电网购电量, $B = P_{\max}\Delta t$ 为每格充、放电上限。沿用第三章的计量与参数, 母线平衡和库存递推为

$$\begin{aligned} g_t + V_t + b_t &= L_t + c_t + w_t, \\ S_{t+1} &= S_t + \eta_c c_t - b_t / \eta_d. \end{aligned} \quad (1)$$

其中 $g_t \geq 0$ 、 $0 \leq w_t \leq V_t$ 、 $0 \leq c_t, b_t \leq B$, 且 $c_t b_t = 0$ 。放电时 $c_t = 0$, 由母线平衡和 $w_t \leq V_t$ 可得 $b_t \leq L_t$, 因此放出的电池电量必须由负载承接。

令 $x_t = S_{t+1} - S_t$ 。互斥充放电使 x_t 的符号唯一确定动作方向:

$$c_t = \frac{[x_t]_+}{\eta_c}, \quad b_t = \eta_d [-x_t]_+, \quad \phi(x_t) = c_t - b_t = \max\{x_t / \eta_c, \eta_d x_t\}. \quad (2)$$

这里 $[z]_+ = \max(z, 0)$ 。由于 $\eta_d \leq 1/\eta_c$, ϕ 是凸分段线性函数。给定状态增量时, 最少购电量为 $[L_t - V_t + \phi(x_t)]_+$; 用两条仿射下界表示其上图 [2], 得到

$$\begin{aligned} \min \quad & \sum_{t=0}^{T-1} p_t g_t, \\ \text{s.t.} \quad & g_t \geq 0, \\ & g_t \geq L_t - V_t + x_t / \eta_c, \quad g_t \geq L_t - V_t + \eta_d x_t, \\ & -\min(B, L_t) / \eta_d \leq x_t = S_{t+1} - S_t \leq \eta_c B, \\ & S_{\min} \leq S_t \leq S_{\max}, \quad S_0 = S_T = 6000. \end{aligned} \quad (3)$$

模型仅含 T 个购电量和 $T + 1$ 个库存量, 共 289 个连续变量; 充放电与弃光量在求解后恢复。

本文采用 Python/SciPy 调用 HiGHS 求解调度与合同子问题。问题一使用对偶单纯形法 (highs-ds), 问题二至四的连续线性规划采用 highs 接口自动选择求解方法。日内控制根据当前信息与库存重解剩余时域, 仅执行首步; 发布时改约仅提交本次生效区段。显式充放电互斥对照通过 SciPy 的 milp 接口求解。

4.2 最优值等价性与动作恢复

命题。在正电价、固定单程效率、允许弃光及表2的物理边界下, 式(3)与物理互斥模型具有相同最优费用, 每个 LP 最优解均可恢复为同费用的物理调度。

证明。任一物理可行解均满足 $c_t - b_t = \phi(x_t)$ 。由母线平衡及 $w_t \geq 0$, 有 $g_t \geq L_t - V_t + \phi(x_t)$, 故两条购电下界成立; 功率限制和 $b_t \leq L_t$ 给出增量界, 库存边界与循环条件保持不变。因此每个物理解对应一个同费用的 LP 解。

反向取一个 LP 最优解。固定状态序列后，各格购电只受本格下界约束， $p_t > 0$ 使 $g_t = [L_t - V_t + \phi(x_t)]_+$ 。按式(2)恢复 c_t, b_t ，并令 $r_t = L_t + c_t - b_t$ 。充电时 $r_t \geq 0$ ；放电时增量下界保证 $b_t \leq L_t$ ，仍有 $r_t \geq 0$ 。取

$$g_t = [r_t - V_t]_+, \quad w_t = [V_t - r_t]_+, \quad (4)$$

便有 $0 \leq w_t \leq V_t$ 和 $g_t + V_t = r_t + w_t$ 。恢复动作满足互斥、功率和原库存边界，且费用不变。两方向映射给出相反的最优值不等式，故最优费用相等。□

状态增量恢复的作用不能由简单删除互斥约束代替。例如构造单格 $L = 1$ 、 $V = 2$ kWh，首尾库存相同， $(c, b, w, g) = (1, 0.81, 0.81, 0)$ 与 $(0, 0, 1, 0)$ 均达到零购电费，前者却同时充放电。正电价本身并不排除这种松弛最优解 [3]；式(2)则直接给出互斥动作。

4.3 代表日结果与调度解释

代表日最优购电费为 35126.95 元，购电量为 59482.70 kWh，首尾库存均为 6000.00 kWh。表3和表4给出题目指定时段的一组最优计划，图3展示对应的整日轨迹。

表3 指定时段购电量与全天结果

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.00	12:00–12:10	480.41	14:00–14:10	0.00
16:00–16:10	445.43	18:00–18:10	531.89	20:00–20:10	0.00
全天购电量/kWh		59482.70	全天购电费/元		35126.95

表4 四小时充放电量与日初、日末库存 (kWh)

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	4500.00	0.00	04:00–08:00	833.33	6365.84
08:00–12:00	4787.96	1703.00	12:00–16:00	5286.04	91.10
16:00–20:00	0.00	5780.13	20:00–24:00	5333.33	2859.87
0:00 库存		6000.00	24:00 库存		6000.00

无储能时按 $[L_t - V_t]_+$ 逐格购电，费用为 48052.05 元；储能调度节省 12925.10 元，降幅为 26.90%。从能量账看，电池额外利用光伏 6247.96 kWh，同时产生转换损耗 3940.73 kWh，两者相抵使外网购电减少 2307.24 kWh。因此，外购电量的下降来自光伏消纳超过储能损耗，而费用的下降还包括购电时段的转移。

从价格看，低价购入 1 kWh 经电池仅能替代 0.81 kWh 高价购电，局部转移须满足 $p_{\text{low}} < 0.81p_{\text{high}}$ 。这一条件解释了价差为何可以抵消损耗，整日最优安排则仍需联合满足功率、库存与首尾循环条件。图中的夜间充电、白天光伏利用及负载高峰放电，正是这些因素共同作用的一组解。表中同一四小时内充、放电量均为正，对应不同十分钟时段的先后动作。

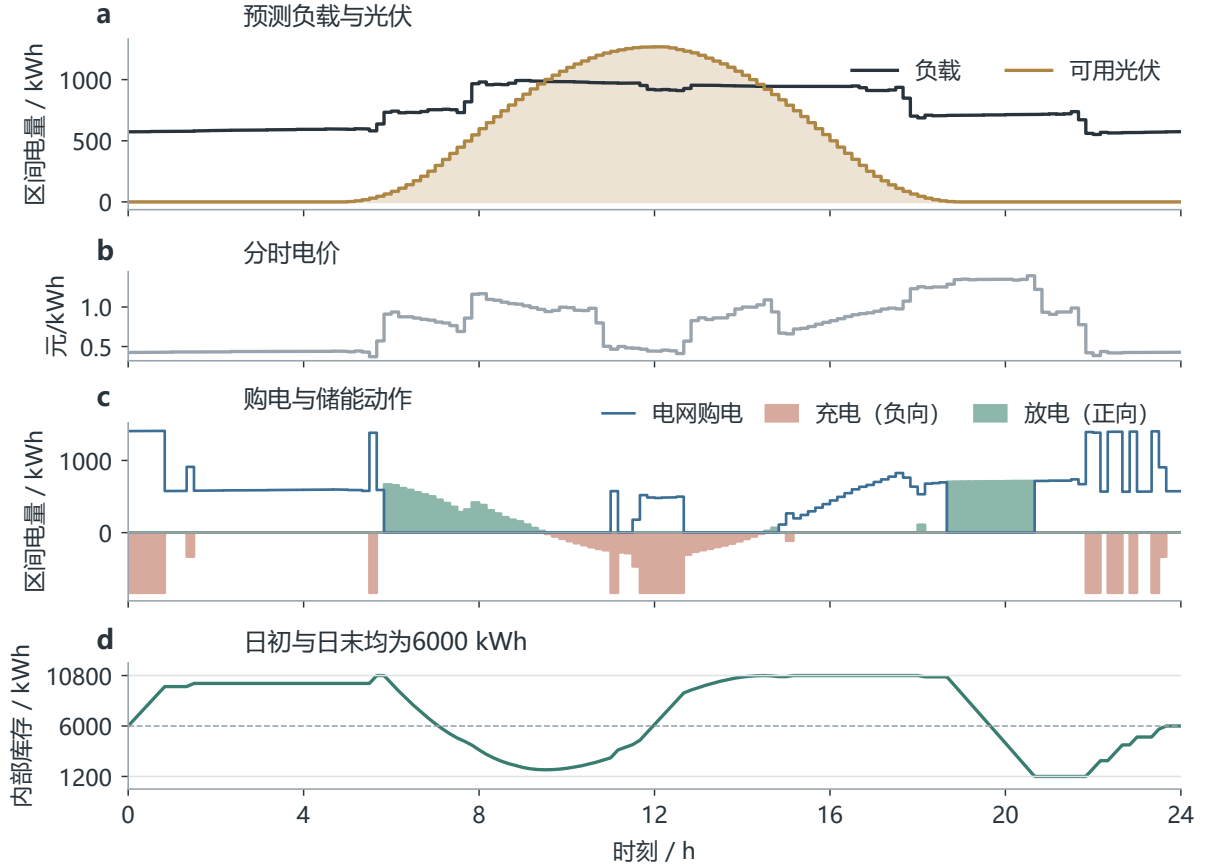


图3 代表日的一组最优调度。负载、光伏、购电与充放电按十分钟电量表示，库存为区间边界状态

最优费用确定后，整日动作仍有一定自由度。逐坐标考察最优面，两个购电量和两个中间库存可以变化，而**题目指定六格的购电量固定**。同价夜间充电可在 23:10–23:20 与 23:30–23:40 之间作等量转移，保持费用和充放电总量；因此即使以总充放电量为次目标，也仍存在多组代表解。本文报告其中一组，完整最优面及时间映射对照见附录B.2与表22，参数响应和独立核验见第八章。上述模型解决了预测给定时电量如何转移的问题。实际运行中，预测与实现值存在偏差，购电又须提前承诺；下一问保留状态增量表示，将购电额度与真实调用分开，并让储能在偏差出现后承担反馈调节。

五、 问题二：不确定负载下的计划购电与日内反馈

负载与光伏偏离零点预测后，既有购电承诺不能撤回，短缺又需要支付高价紧急购电费。储能不仅承担峰谷转移，还需在保留后续库存与填补当前缺口之间取舍。本文比较点预测固定调度 D、场景固定调度 S 和场景反馈调度 F-fast，依次考察预测误差进入日前计划、日内信息进入储能控制后的完整运行结果。

5.1 承诺额度与实际现金结算

每日零点封存全天普通购电额度 q_t 。令 $A_t = L_t + c_t - b_t \geq 0$ ，按光伏、普通额度、紧急购电的顺序分配实际供给：

$$\begin{aligned} v_t &= \min(V_t, A_t), & a_t &= \min(q_t, A_t - v_t), \\ e_t &= A_t - v_t - a_t, & u_t &= q_t - a_t, & w_t &= V_t - v_t. \end{aligned} \quad (5)$$

普通额度全部计费，实际调用 a_t 进入母线，未调用额度 u_t 不退款也不供电； w_t 则是未利用的光伏。相应物理平衡与现金费用为

$$a_t + v_t + e_t + b_t = L_t + c_t, \quad S_{t+1} = S_t + \eta_c c_t - b_t / \eta_d, \quad (6)$$

$$C_d = \sum_{t=0}^{143} p_t q_t + \sum_{t=0}^{143} 5p_t e_t. \quad (7)$$

日末库存估值用于制定计划，但不从式(7)的现金中扣除。紧急电允许与充电同时出现，实际电量由母线和库存约束共同决定。

购电余量的作用可由无储能单格说明。若净负载为随机量 N ，则

$$\min_{q \geq 0} pq + 5p \mathbb{E}[(N - q)_+]. \quad (8)$$

连续分布且最优解 $q > 0$ 时，边际平衡给出 $\Pr(N > q) = 1/5$ ，即非负化的 80% 分位数。引入储能后，各格剩余需求通过库存相连，需联合确定额度与储能路径。

5.2 历史残差场景与日前线性规划

预测仅使用零点前已有数据。将周五、周六归为低负载型，其余日期归为高负载型，负载点预测取最近至多 3 个同类型日的均值；光伏取最近至多 14 日的加权均值，距起点 j 天的权重正比于 0.7^{j-1} 。无同类负载历史时，附件 1 模板按低、高类型分别乘 78400/111025、124000/111025；无光伏历史时使用附件 1 预测。这些规则在评价期保持固定，参数的描述性核对与选定背景见第八章。

对历史日 $j < d$ ，用 j 之前的数据重建当时预测，得到整日残差 $\varepsilon_{j,t}^X = X_{j,t} - \hat{X}_{j \rightarrow j,t}$ ， $X \in \{L, V\}$ 。取最近至多 30 个历史起点，以同一日期配对负载和光伏残差，构造

$$L_{s,t} = [\hat{L}_{d,t} + \varepsilon_{s,t}^L]_+, \quad V_{s,t} = [\hat{V}_{d,t} + \varepsilon_{s,t}^V]_+, \quad (9)$$

并令光伏点预测为零时对应的光伏场景也为零。记 $N_{s,t} = L_{s,t} - V_{s,t}$ ，所有场景共享 q_t 与库存增量 x_t ，紧急购电 $e_{s,t}$ 随场景变化。S 的日前问题写为

$$\begin{aligned}
\min \quad & \sum_t p_t q_t + \frac{1}{K} \sum_{s,t} 5p_t e_{s,t} - \lambda y, \\
\text{s.t.} \quad & e_{s,t} \geq N_{s,t} + x_t / \eta_c - q_t, \\
& e_{s,t} \geq N_{s,t} + \eta_d x_t - q_t, \quad e_{s,t} \geq 0, \quad q_t \geq 0, \\
& -\min(B, \min_s L_{s,t}) / \eta_d \leq x_t = S_{t+1} - S_t \leq \eta_c B, \\
& S_{\min} \leq S_t \leq S_{\max}, \quad S_0 \text{ 为实际日初库存}, \\
& 0 \leq y \leq \bar{S}, \quad y \leq S_{144}.
\end{aligned} \tag{10}$$

由于紧急电价格为正，最优时 $e_{s,t} = [N_{s,t} + \phi(x_t) - q_t]_+$ ；增量下界则保证恢复后的放电不超过任一场景负载。沿用式(2)，即可保持互斥、状态和费用，故式(10)是**给定有限场景、共享储能路径及终端约束下子问题的精确 LP 表示**。

D 将全部场景替换为单条点预测，其余结构相同；S 与 F-fast 共用上述日前模型，差别在于日内是否重新确定储能动作。无储能对照令 $c = b = 0$ ，逐格采用样本净负载的非负经验 80% 分位数制定额度。

5.3 连接相邻日期的库存估值

若只计算当日购电费，日末留存电量的后续用途未进入目标。本文在每天零点以同一起点信息预测当天及次日，求解一个末端库存不低于当前 S_0 的两日确定性 LP。记其第一日末库存为 $\bar{S}$ ，两日库存连接等式的对偶乘子为 μ ，按该等式方向定义

$$\lambda = \max(0, -\mu), \quad \mathcal{V}(S) = \lambda \min(S, \bar{S}). \tag{11}$$

λ 以元/内部 kWh 计，衡量连接点附近的库存价值， $\bar{S}$ 限定这一线性近似的作用范围。式(10)中的辅助变量 y 实现 $-\mathcal{V}$ 的线性化。各策略按自身真实日初库存估价，同一天内固定 $(\lambda, \bar{S})$ ；两日问题只用于估价，正式购电承诺仍覆盖当天。对偶符号和局部扰动核对集中列于第八章。

这一估值把当天与次日通过库存连接起来。当 $S_{144} < \bar{S}$ 时，多保留 1 内部 kWh 可使目标中的估值项减少 λ 元，是否保留仍取决于当日取得这部分库存的费用；超过 $\bar{S}$ 后，额外库存不再获得同样的线性奖励。次日到来时，再以实际末库存重新计算估值，避免把当天的局部影子价格不加限制地沿用到后续日期。

5.4 逐时反馈与执行保护

李嘉伟等将分布鲁棒优化与模型预测控制结合用于多时间尺度调度 [4]。本文采用历史配对残差场景与库存反馈，F-fast 在日内固定已经封存的 q ，根据剩余时段的信息更新储能路径，**每次只执行首步**。控制所用观测遵循图2的约定。以日期固定随机种

子，从可用历史路径中无放回抽取至多 10 条；对负载、光伏分别以历史残差估计相邻时段相关系数 $\rho_X \in [0, 0.999]$ 。若观测锚点为 r ，则 $h \geq t$ 的场景为

$$\tilde{X}_{s,h}^{(t)} = \left[\hat{X}_{d,h} + \rho_X^{h-r} (X_{d,r} - \hat{X}_{d,r}) + \varepsilon_{s,h}^X - \rho_X^{h-r} \varepsilon_{s,r}^X \right]_+ . \quad (12)$$

该式先移除历史路径自身锚点的衰减影响，再加入当天观测偏差。F-fast 取 $r = t$ ，当前格各场景均为当期量测；未来光伏预测为零的时段保持零光伏。以真实 S_t 为起点，将式(10)限制到 $h = t, \dots, 143$ 重解，首步放电另满足

$$b_t \leq \left[\min_s \{ \tilde{L}_{s,t}^{(t)} - \tilde{V}_{s,t}^{(t)} - q_t \} \right]_+ . \quad (13)$$

F-fast 的当前场景相同，故该式成为当前净缺口上界。

滞后参考对照取 $r = t - 1$ ，首格无当日观测时不作条件化；其参考 LP 不施加式(13)这项额外净缺口界，仍保留功率、库存和场景负载的物理界。D、S 执行日前储能意图，滞后方案执行滚动首步意图，均按当前库存和量测限幅：

$$\begin{aligned} c_t &= \min(c_t^{\text{intent}}, [\min\{B, (S_{\max} - S_t)/\eta_c\}]_+) , \\ b_t &= \min(b_t^{\text{intent}}, [\min\{B, \eta_d(S_t - S_{\min}), L_t - V_t - q_t\}]_+) . \end{aligned} \quad (14)$$

保护仅削减参考意图，保持原有互斥方向；实际动作再按式(5)结算，末库存传递给下一格及下一日。至此，零点额度、日内动作和现金账本形成连续的运行过程。

5.5 334 日连续运行结果

1 月以 F-fast 从 6000.00 kWh 预热，得到 2 月 1 日共同入口 8657.42 kWh。随后各策略独立运行至 12 月 31 日，共 334 日、48096 格；次日计划与库存估值使用各自上一日实际末库存。表5列出现金构成与评价期末库存，全期 Oracle 以全部实测预知为条件提供费用下界。

表 5 334 日现金费用（万元）与期末库存（kWh）

策略	普通计划费	紧急费	现金合计	期末库存
D	1217.17	332.89	1550.06	8008.23
S	1345.87	82.98	1428.85	8221.73
F-fast	1346.44	31.17	1377.60	8264.39
无储能	1731.27	89.97	1821.25	8657.42
全期 Oracle	1222.69	0.00	1222.69	1200.00

分项与合计分别由未舍入值计算，显示末位可能存在舍入差。

D 至 S 的费用下降 7.82%，S 至 F-fast 进一步下降 3.59%。点预测计划的紧急费较高，表明实际偏差会使原来的确定性安排付出额外代价；场景计划增加普通购电余量后，紧急费用明显下降。在 S 与 F-fast 的完整轨迹比较中，普通计划费只增加 0.57 万元，紧急费减少 51.81 万元，**主要现金变化来自缺口费用下降**。两者采用同类日前模型，日内动作和跨日库存各自演化，后续日期的计划也随之变化。

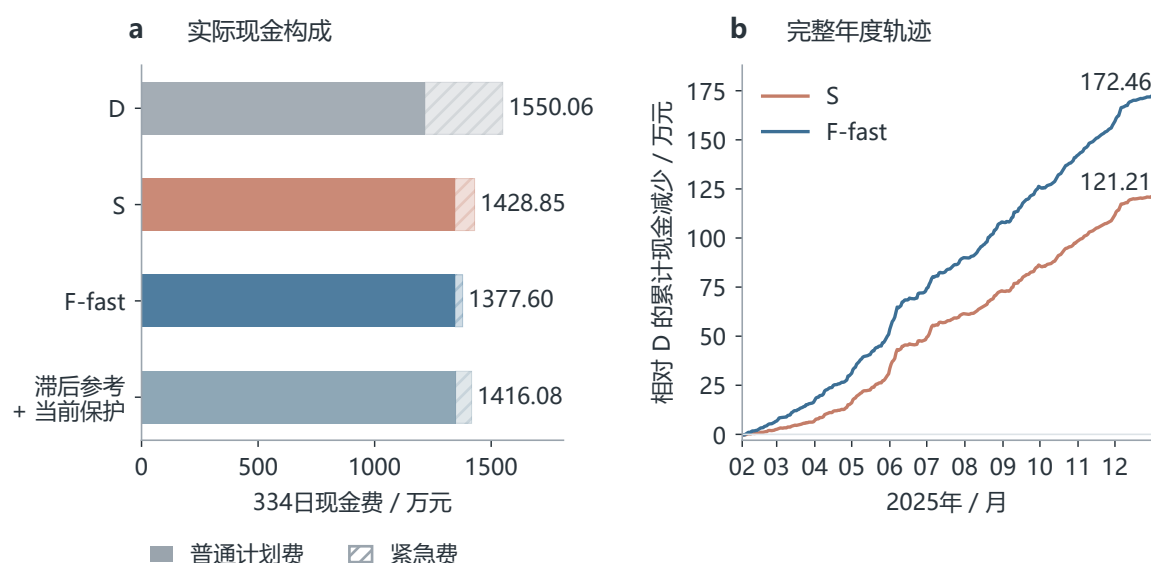


图4 334日费用构成与累计现金变化。累计曲线以点预测固定调度D为基准

F-fast 全年紧急购电为 69634.10 kWh，紧急费占现金费用的 2.26%；已付款但未调用的额度为 1433997.23 kWh。可见高价缺口受到抑制时，计划冗余仍然存在，储能反馈并未消除提前承诺的全部代价。相对无储能对照，F-fast 现金费用下降 24.36%。因此，在本组连续回放中，储能既提供确定性问题中的电量转移，又参与管理承诺额度与实际需求之间的偏差。

滞后参考加当前量测保护的费用为 1416.08 万元，末库存 8173.27 kWh；按未舍入现金计算，比 F-fast 高 2.79%。这一对照、Oracle 的终端匹配、年度物理核验及策略比较的解释范围集中于第八章。四个指定日期的计划、分块执行和连续紧急事件见第5.6节，均取自同一 F-fast 连续轨迹。问题二的调整始终发生在既定合同之内：信息更新后，储能可以改变用电时机，已经买下的额度却保持不变。问题三进一步允许未来合同本身被修订，从而把新增信息传递到购电承诺层。

5.6 问题二指定日期的计划与执行

以下为同一年度主策略在 2025 年四个指定日期的结果；电量单位为 kWh，费用为元。三类表依次对应题目表 1、表 2 和表 3，数值均由未舍入账本计算。

表 6 问题二指定购电计划与全天费用（对应题目表 1）

日期	10:00	12:00	14:00	16:00	18:00	20:00	全天量
03-20	0.00	570.00	0.00	489.34	87.55	60.28	64645.03
06-21	0.00	0.00	0.00	228.54	0.00	99.57	44721.36
09-23	0.00	507.99	0.00	576.67	142.97	195.63	70513.45
12-21	0.00	1003.34	0.00	853.45	95.99	49.66	99197.63

日期	计划费	紧急费	现金合计	实际电网电量
03-20	39959.66	877.46	40837.12	62601.98
06-21	25098.11	0.00	25098.11	40826.24
09-23	44156.10	0.00	44156.10	67029.38
12-21	63981.85	0.00	63981.85	93329.86

表中合同量为零点原计划 q ；时刻为十分钟区间起点。实际电网电量为 $a + e$ 。现金为计划费与紧急费之和；分项分别舍入，末位可能有差异。

表 7 问题二四小时充放电与日初日末库存（对应题目表 2）

区间/时刻	03-20		06-21		09-23		12-21	
	充电	放电	充电	放电	充电	放电	充电	放电
00:00–04:00	2001.00	83.18	1731.85	58.57	1960.66	18.65	2413.29	40.03
04:00–08:00	879.72	5166.21	992.39	1730.28	736.01	5692.05	942.34	1404.83
08:00–12:00	5462.43	2777.38	2544.85	0.00	4714.51	1896.62	5253.00	6835.23
12:00–16:00	4380.93	375.30	5407.62	0.00	4844.01	536.22	6850.22	2195.09
16:00–20:00	227.11	6108.86	259.70	5618.83	233.11	5880.05	38.55	5982.88
20:00–24:00	3677.00	2627.40	7018.79	2077.34	7813.79	2643.94	7822.84	2570.10
00:00 库存	8550.00		2944.83		8550.00		8526.13	
24:00 库存	4472.77		8565.60		8302.41		8371.94	

块内充放电均为正表示不同十分钟格先后动作，非同格充放。库存为电池内部储电量。

表 8 问题二连续紧急购电事件（对应题目表 3）

03-20		06-21		09-23		12-21	
连续时段	电量	连续时段	电量	连续时段	电量	连续时段	电量
12:30–12:40	54.48	无	0.00	无	0.00	无	0.00
20:10–20:20	114.24	—	—	—	—	—	—

完整 144 格合同、全部四小时块及全年连续事件见result2.xlsx。其中计划表全天费用为含紧急费的最终现金。

六、 问题三：预报更新与购电调整

6.1 合同费用与生效区段

光伏预报在日内多次发布，使零点购电计划有了修订依据。记原计划为 q_t ，交付前正式确定的合同额度为 m_t ，则实际调用满足 $a_t \leq m_t$ ，未调用额度为 $u_t = m_t - a_t$ 。为同时描述增购与退订，按交付时段价格 p_t 采用 B 口径合同费用

$$\begin{aligned} K_B(q_t, m_t) &= p_t \min(q_t, m_t) + 0.5p_t(q_t - m_t)_+ + 1.5p_t(m_t - q_t)_+, \\ C &= \sum_t \{K_B(q_t, m_t) + 5p_t e_t\}. \end{aligned} \quad (15)$$

下调部分退还原价并支付 50% 违约金，上调部分按原价的 150% 计费； K_B 已经包含原计划的结算，不能再叠加 $p_t q_t$ 。

对一个交付时段，在其之前最后一个启用的发布时间确定 m_t 。启用 6、12、18 点时，三次分别提交 6–12、12–18、18–24 点合同，优化中更远时段的额度只作为候选。该规则依赖题目未规定提前申报期限、额度上限或额外交易费，且费用不随申报时刻变化：晚决定可忽略新增信息，复制任一较早决定，因此不会缩小可行策略集合。下文据此区分本次生效合同和未来候选合同。

6.2 剩余窗口校准与融合预报

官方预报和历史预测在不同发布时间的误差并不相同，故分别拟合二者的组合权重。附件 3 的整点功率线性插值至十分钟末端，再乘 1/6 换算为区间电量；发布至首个整点之间，以发布时已取得的实测端点为锚。仅 1 月 1 日零点无前日实测，使用附件 1 模板端点冷启动。该规则保持与物理模型相同的离散时标。

设 $h \in \{0, 6, 12, 18\}$ ，当天剩余目标数为 $n_h = 144 - 6h$ 。记 $H_{j,h,k}$ 、 $O_{j,h,k}$ 为历史与官方光伏预报， $V_{j,h,k}$ 为对应实测电量，构造

$$\hat{V}_{j,h,k} = (1 - w_h)H_{j,h,k} + w_h O_{j,h,k}, \quad 0 \leq w_h \leq 1. \quad (16)$$

以 $\mathcal{H}_h(\tau)$ 表示拟合时刻 τ 之前当天目标已全部结束的一月发布日，选择

$$w_h(\tau) \in \arg \min_{0 \leq w \leq 1} \sum_{j \in \mathcal{H}_h(\tau)} \sum_{k=1}^{n_h} |(1 - w)H_{j,h,k} + w O_{j,h,k} - V_{j,h,k}|. \quad (17)$$

拟合目标止于当天 24:00，与当天合同优化窗口一致。一月逐次使用当时可得样本，2 月 1 日零点冻结表 9 中的权重；无样本时取 $w = 0.5$ ，存在多个最优权重时取最优区间中点。

表9 各发布时间的冻结融合权重及后续评价误差

发布时间	目标数/日	一月校准权重/%		二至十二月 MAE	
		官方	历史	官方	历史
00:00	144	38.11	61.89	32.9152	25.0247
06:00	108	72.51	27.49	26.4194	33.1706
12:00	72	93.99	6.01	12.5421	24.1365
18:00	36	100.00	0.00	1.6692	0.3675

权重使用一月共 31 天的当天剩余目标校准，2 月 1 日冻结；MAE 为其后 334 日相同口径的误差，单位为 kWh/十分钟目标。两列误差用于评价冻结后的表现。

表9同时显示校准决定与后续表现：12 点官方预报的评价误差低于历史预测，而 18 点虽在一月得到 100% 官方权重，后续误差反而高于历史预测。权重反映校准期的最优组合，评价期表现仍需另行检验；更新时点的费用价值将在完整策略中考察。

融合预报形成场景中心，叠加过去至多 30 条同发布时间的负载与光伏配对误差路径，再截为非负。历史残差等完整 24 小时目标实现后纳入，其光伏中心按历史原点当时的权重重建。一月权重冻结与全年研究设计的关系，以及各发布时间的完整误差检验，见第八章。

6.3 共享路径与单次信号追索

零点规划采用两种结构。Z1 沿用问题二的共享储能路径：各场景使用同一 q 和状态增量 $x_t = S_{t+1} - S_t$ ，而紧急电按场景补足。令 $m_t = q_t$ ，其目标为 $p^\top q + 5p^\top \bar{e} - \lambda \min(S_{144}, \bar{S})$ 。改约时则以 $K_B(q_t, m_t)$ 替换合同费用，求解剩余窗口；两种情形均使用

$$\begin{aligned} e_{s,t} &\geq L_{s,t} - V_{s,t} - m_t + x_t/\eta_c, \\ e_{s,t} &\geq L_{s,t} - V_{s,t} - m_t + \eta_d x_t, \quad e_{s,t} \geq 0. \end{aligned} \quad (18)$$

库存、功率与不倾倒地电池能量的约束继承问题二，充放电仍按式(2)恢复。日末估值参数由零点两日确定性问题产生：当天光伏采用融合预报，次日采用历史点预测。

Z2 进一步在零点预留一次 6 点信号追索。历史信号取同一日 6–24 点光伏总量的“6 点融合预报减零点融合预报”，依过去样本三分位阈值划分至多三组。所有场景共享零点 q 和 6 点前的储能状态，6 点后同组共享 m_g, S_g 。以 π_g 为组频率、 K 为场景数，目标为

$$\begin{aligned} \min \quad & \sum_g \pi_g \sum_t K_B(q_t, m_{g,t}) + \frac{1}{K} \sum_{s,t} 5p_t e_{s,t} - \sum_g \pi_g \lambda \min(S_{g,144}, \bar{S}), \\ \text{s.t.} \quad & m_{g,t} = q_t \quad (t < 36). \end{aligned} \quad (19)$$

这些分组表示零点尚未揭示的随机分支，组加权动作仅作为规划参考；实际执行在信息到达后重新优化负责区段的合同与储能，并逐步传递真实库存。不启用 6 点更

新时，Z2 不设置该分叉而退化为 Z1；只更新信息而不改约时，各组所有时段均固定 $m_q = q$ 。

日内每次发布先优化剩余合同与储能，仅提交负责区段。随后每十分钟固定当前有效合同，以 F-fast 控制器更新首步动作；未生效候选仍由原 q 代替。控制场景从可用配对路径无放回抽取至多 10 条，种子固定为“日索引 +365h”，并沿用问题二的历史相关系数校正与量测保护。这样，预报更新影响规划，实际执行仍通过逐步反馈连接。

6.4 固定储能下的合同分位结构

合同调整为何可能保持不动，可从同一次改约 LP 的条件子问题解释。固定其返回的储能路径，令 $N_s = L_s + c^* - b^* - V_s$ ，则单时段目标为 $H(m; q) = K_B(q, m) + 5pK^{-1} \sum_s (N_s - m)_+$ ，约束 $m \geq 0$ 。记 $F(x^-) = K^{-1} \#\{N_s < x\}$ 、 $F(x) = K^{-1} \#\{N_s \leq x\}$ 。B 口径在 $m < q$ 与 $m > q$ 的边际合同费分别为 $0.5p$ 、 $1.5p$ ；由左右导数跨过零点，得到

$$\begin{aligned} 0 < m < q: & F(m^-) \leq 0.9 \leq F(m), \\ m > q: & F(m^-) \leq 0.7 \leq F(m), \\ m = q > 0: & F(q^-) \leq 0.9, \quad F(q) \geq 0.7. \end{aligned} \tag{20}$$

由此可解释合同为何不会随每次预测变化而连续追踪需求。增购的边际费用较高，退订又只能部分回收原支出，二者在原计划 q 附近形成一个保持合同仍可能最优的范围。若 $F(q) < 0.7$ ，原额度偏低，需要上调；若 $q > 0$ 且 $F(q^-) > 0.9$ ，原额度偏高，需要下调。中间情形按式(20)判断，离散场景可能形成平坦最优区间，同一费用可对应多个合同额度。

边界 $m = 0 < q$ 只需 $F(0) \geq 0.9$ ； $m = q = 0$ 只需 $F(0) \geq 0.7$ ，此时相对调整比 $(m - q)/q$ 无定义。上述条件只描述**固定同次储能路径后的 B 口径合同子问题**，左右分布同时保留相等样本的影响；相应核验集中于第八章。

6.5 连续运行结果与时点比较

在本题数据、信息条件和已比较配置下，**应引入 6、12、18 点预报，并联合调整购电合同与储能控制**。启用三个时点的 Z2/111 在 334 日回放中现金费用最低，为 1350.32 万元。这支持所考察的完整更新方案，不意味着每次预报更新均能降低费用，也不表示未经比较策略的全局最优。

在共同入口 8657.42 kWh 下连续回放 334 日，Z2/111 的现金费用为 1350.32 万元，在比较配置中最低，末库存为 8303.87 kWh。其原计划费、调整净额与紧急费分别为 1326.07、6.46、17.79 万元，相对问题二 F-fast 减少 27.29 万元。三位开关依次表示 6、12、18 点的更新与改约，Z1/000 仅使用零点融合而不改约。

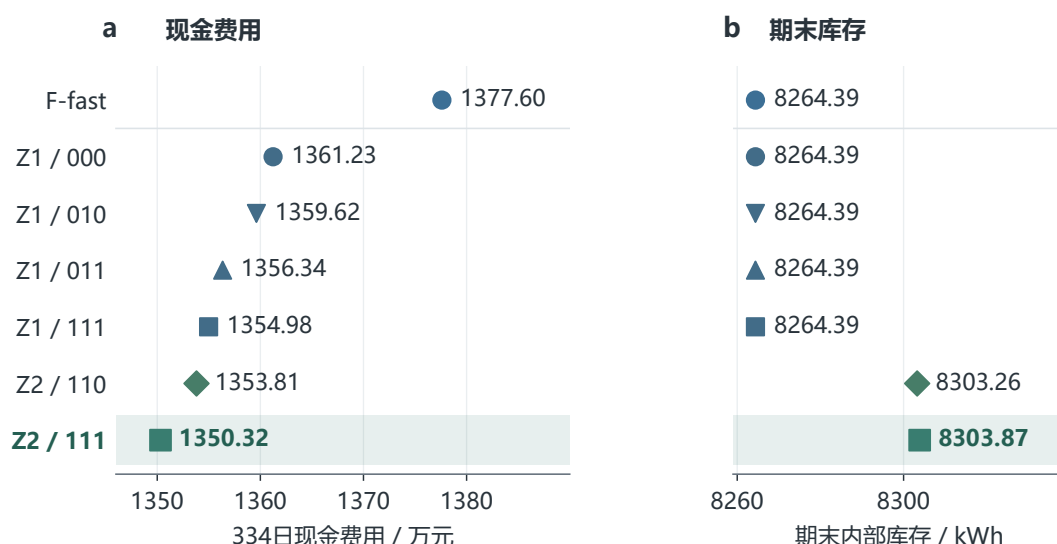


图 5 各完整配置的 334 日现金费用与期末库存。每个点对应一条连续运行轨迹，E0 为问题二 F-fast

表 10 指定配置的现金费用与期末库存差额

配置变化	现金差额/万元	库存差额/kWh
零点融合基线相对 E0	-16.37	0.00
Z1: 000 → 010	-1.61	0.00
Z1: 010 → 011	-3.28	0.00
Z1: 011 → 111	-1.36	0.00
111: Z1 → Z2	-4.66	+39.48
Z2: 110 → 111	-3.49	+0.60

差额为变化后减变化前，均由未舍入值计算；负值表示现金费用减少。

表10采用两类有明确起终配置的比较。在 Z1 结构内，先加入 12 点、再加入 18 点，现金费用依次减少 1.61 万元和 3.28 万元；这两个数沿同一配置序列计算，起点随步骤改变。在相同 111 开关下，零点加入信号追索后再减少 4.66 万元，末库存高 39.48 kWh，说明两类零点规划形成了不同的后续运行轨迹。

对主配置，Z2/111 较 Z2/110 少支出 3.49 万元，末库存高 0.60 kWh。结合表9，18 点的官方光伏预测虽在评价期较差，启用该时点后的完整策略仍有较低费用。更新同时作用于信息、场景、合同及随后控制，预测误差本身不足以决定这一联合结果。比较的库存口径与研究设计归属见第八章，各配置绝对金额见表32，指定日期的三类完整结果见第6.6节。

至此，日前额度与日内合同已能响应负载、光伏信息，但结算仍使用固定分时价格。下一问保留两类策略结构，考察每天变化的价格如何进入决策，又如何与计费权重共同决定最终现金。

6.6 问题三指定日期的计划与执行

以下为同一年度主策略在 2025 年四个指定日期的结果；电量单位为 kWh，费用为元。三类表依次对应题目表 1、表 2 和表 3，数值均由未舍入账本计算。

表 11 问题三指定购电计划与全天费用（对应题目表 1）

日期	合同	10:00	12:00	14:00	16:00	18:00	20:00	全天量
03-20	q	0.00	585.88	0.00	568.77	118.18	60.28	64881.30
03-20	m	0.00	557.83	0.00	568.77	118.18	60.28	64550.62
06-21	q	0.00	0.00	0.00	216.78	0.00	99.57	42587.27
06-21	m	0.00	0.00	0.00	161.10	0.00	99.57	42437.24
09-23	q	0.00	453.76	0.00	604.83	215.26	188.70	70372.07
09-23	m	0.00	421.04	0.00	604.83	212.30	188.70	69833.53
12-21	q	0.00	971.87	0.00	848.48	138.78	57.91	98249.23
12-21	m	0.00	971.87	0.00	848.48	138.78	57.91	97748.17

日期	计划费	调整净额	紧急费	现金合计	实际电网电量
03-20	40218.21	57.98	292.16	40568.35	62378.18
06-21	23993.36	-51.48	0.00	23941.88	39308.11
09-23	44086.35	-43.86	0.00	44042.49	66532.00
12-21	63305.01	-158.98	0.00	63146.04	93300.60

原计划 q 与最终生效额度 m 分别列示；时刻为十分钟区间起点。实际电网电量为 $a + e$ 。现金为计划费、调整净额与紧急费之和；分项分别舍入，末位可能有差异。

表 12 问题三四小时充放电与日初日末库存（对应题目表 2）

区间/时刻	03-20		06-21		09-23		12-21	
	充电	放电	充电	放电	充电	放电	充电	放电
00:00–04:00	2001.00	83.18	523.80	58.57	1993.45	14.71	2386.77	40.03
04:00–08:00	679.88	5616.99	477.73	1730.35	655.60	5761.31	960.64	1448.18
08:00–12:00	3867.77	2777.38	4088.75	0.00	4839.91	1896.62	5265.06	6767.52
12:00–16:00	6333.94	254.72	5642.57	0.00	4896.82	416.01	6876.02	2284.28
16:00–20:00	382.44	5910.57	366.05	5596.11	168.97	5854.37	37.23	5976.18
20:00–24:00	3665.25	2740.19	6997.91	2097.20	7779.62	2683.51	7818.63	2565.35
00:00 库存	8550.00		2798.67		8516.82		8554.77	
24:00 库存	4472.77		8550.00		8343.85		8362.98	

块内充放电均为正表示不同十分钟格先后动作，非同格充放。库存为电池内部储电量。

表 13 问题三连续紧急购电事件（对应题目表 3）

03-20		06-21		09-23		12-21	
连续时段	电量	连续时段	电量	连续时段	电量	连续时段	电量
12:30–12:50	118.59	无	0.00	无	0.00	无	0.00

完整 144 格合同、全部四小时块及全年连续事件见result3.xlsx。其中计划表全天费用为原计划费，调整表全天费用为最终现金，不能相加。

七、 问题四：日际电价变化下的决策与结算

问题四将固定分时电价替换为附件 4 的日际变化电价。4-2 继承问题二 F-fast，4-3 继承问题三 Z2/111，包括零点信号追索及 6、12、18 点预报更新与改约；光伏权重和其余策略设置保持不变。由此，价格变化进入同一物理模型，**决策依据与实际现金则分别核算**。

7.1 价格信息与现金模型

电价在这里承担两种作用：零点计划需要一个决策依据，交付后的现金核算则使用真实价格。附件 4 提供全年价格记录，但未规定当日价格何时公布，因此分别设置“已知价假设”和“前日价预测”。记实际电价为 $p_{d,t}$ ，两种口径在零点冻结

$$\tilde{p}_{d,t} = \begin{cases} p_{d,t}, & \text{当日完整价格日前已知,} \\ p_{d-1,t}, & \text{以前一日曲线预测当日价格,} \end{cases} \quad \hat{p}_{d+1|d,t} = \tilde{p}_{d,t}. \quad (21)$$

两种口径日内均不更新决策价格，次日延续也使用同一冻结曲线。两日确定性问题据此计算 $(\lambda_d, \bar{S}_d)$ ，实际次日价格不进入估值或决策。

4-2 将问题二目标中的价格替换为 $\tilde{p}$ ，日前优化

$$\min \sum_t \tilde{p}_{d,t} q_t + \frac{1}{K} \sum_{s,t} 5\tilde{p}_{d,t} e_{s,t} - \lambda_d y, \quad 0 \leq y \leq \min(S_{144}, \bar{S}_d). \quad (22)$$

日内固定 q ，只更新储能首步。对 4-3，令 $\kappa_B(q, m) = \min(q, m) + 0.5(q - m)_+ + 1.5(m - q)_+$ ，在问题三的前日追索和改约目标中分别使用 $\tilde{p}\kappa_B$ 与 $5\tilde{p}e$ 。储能约束、非预见性结构和控制规则不变。

执行后，两问都按交付区间真实价格记账：

$$\begin{aligned} C_d^{4-2} &= \sum_t p_{d,t} (q_{d,t} + 5e_{d,t}), \\ C_d^{4-3} &= \sum_t p_{d,t} \{ \kappa_B(q_{d,t}, m_{d,t}) + 5e_{d,t} \}. \end{aligned} \quad (23)$$

在前日价预测口径下，零点优化使用前一日曲线，式(23)使用交付时真实价格，二者分别对应决策和评价。图6给出评价期价格变化及其与日净负载的对应关系。

7.2 两种价格口径下的运行结果

表14并列报告两问 334 日的现金与库存。所有方案使用共同入口，完整轨迹持续至评价期末。

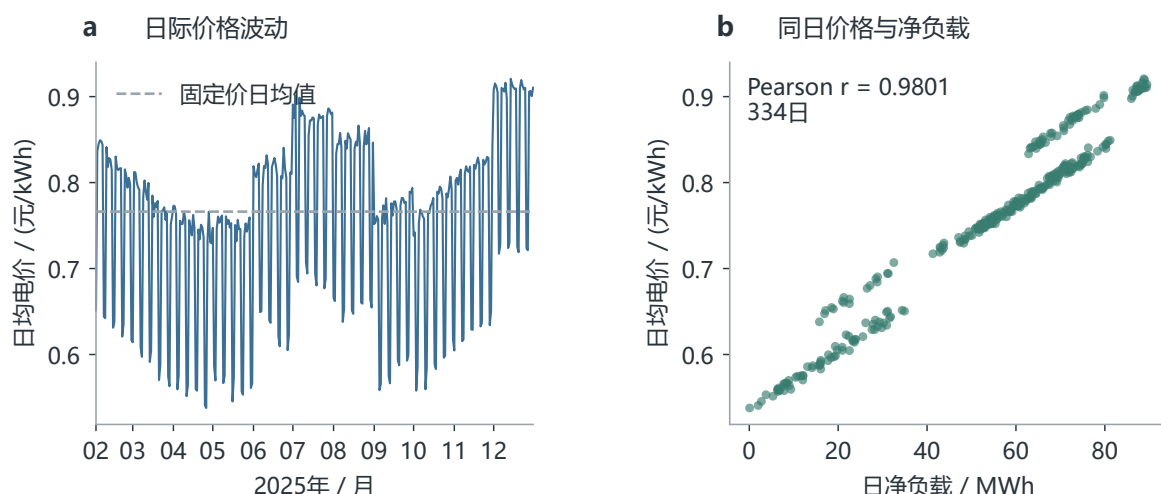


图 6 评价期的日际电价与同日净负载。左图为 334 日日均实际电价，虚线为原固定电价的日均值；右图净负载为实测负载减可用光伏，不作正部截断

表 14 问题四两种价格口径的费用与库存

指标	4-2 已知价	4-2 前日价	4-3 已知价	4-3 前日价
原计划费 / 万元	1409.79	1425.96	1388.64	1404.80
调整净额 / 万元	0.00	0.00	7.07	6.67
紧急费 / 万元	33.54	34.38	19.12	20.54
现金合计 / 万元	1443.33	1460.35	1414.83	1432.01
初库存 / kWh	8657.42	8657.42	8657.42	8657.42
末库存 / kWh	6027.22	8711.14	6134.92	8700.00

同一价格口径下，4-3 相对 4-2 在已知价假设中少支出 28.50 万元、末库存高 107.71 kWh，在前日价预测中少支出 28.34 万元、末库存低 11.14 kWh。两种价格条件均保留了问题三完整策略的现金优势；这一对照共同包含融合预测、日前追索、反馈和改约，反映的是两条完整策略的结果差异。

由已知价假设改为前日价预测，4-2 现金增加 17.02 万元、末库存增加 2683.93 kWh，4-3 现金增加 17.18 万元、末库存增加 2565.08 kWh。现金上升与库存路径变化同时发生，因而表中的两项指标需要共同阅读；详细比较口径及价格预测误差见第八章。年度电量见表33，指定日期的计划与执行结果见第7.4、7.5节。

7.3 固定动作换价的费用分解

为了分开价格变化与重新决策，先冻结问题二、三的原合同和全部动作，用附件 4 价格重算，再与 4-2、4-3 的已知价结果比较。记原固定电价为 p_t^0 ，相应计费权重为 $W_{d,t} = q_{d,t} + 5e_{d,t}$ 或 $W_{d,t} = \kappa_B(q_{d,t}, m_{d,t}) + 5e_{d,t}$ 。W 包含合同计费和紧急电倍数，不是实际购电总量。

对每个固定十分钟时段 t ，在 $D = 334$ 日上计算均值 $\bar{p}_t, \bar{W}_t$ 及以 D 为分母的总体协方差，得到

$$\begin{aligned}\Delta C_{\text{换价}} &= \sum_{d,t} (p_{d,t} - p_t^0) W_{d,t} \\ &= D \underbrace{\sum_t (\bar{p}_t - p_t^0) \bar{W}_t}_{\text{时段均价变化项}} + D \underbrace{\sum_t \text{Cov}_d(p_{d,t}, W_{d,t})}_{\text{同一时段跨日协方差项}}.\end{aligned}\quad (24)$$

这是**固定动作下的会计恒等式**，均值和协方差均按固定 t 跨日计算。图7将两个分解项与重新决策的变化依次连接。

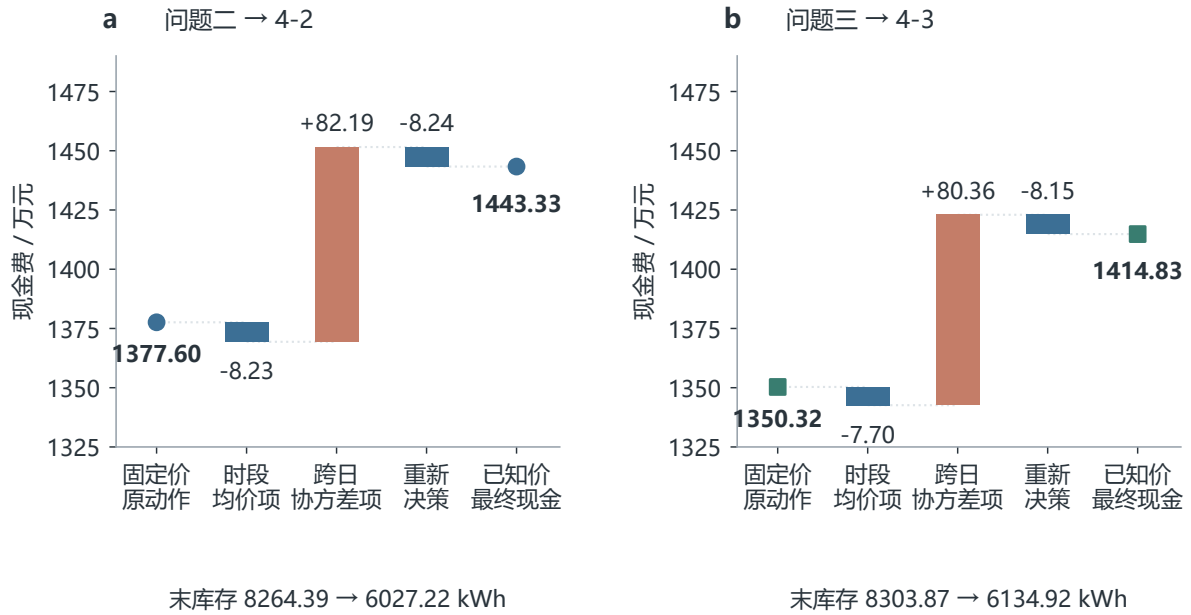


图7 固定动作换价与重新决策的费用变化。均价项与协方差项按同一十分钟时段跨 334 日计算；最后一步为已知价假设下重新决策相对换价账本的变化

问题二原动作换价后增加 73.96 万元，其中时段均价项为 -8.23 万元，协方差项为 82.19 万元；问题三对应增加 72.66 万元，分解为 -7.70 万元和 80.36 万元。按各十分钟时段的跨日平均价格计算，两条轨迹原本都应减少费用；加入价格与计费权重的对应关系后，正协方差项超过了这部分减少。因此，平均电价并不能单独说明年度支出，更高价格是否恰好出现在计费权重较大的日期同样重要。式(24)准确分解这两种会计构成，其比较对象始终是固定的原动作。

已知价下重新决策进一步减少 8.24、8.15 万元现金，但末库存分别由 8264.39、8303.87 kWh 降至 6027.22、6134.92 kWh。故重新决策项单独列示，不与固定动作换价项混为同一作用；完整金额和库存分解保留于表34。

7.4 4-2 指定日期的计划与执行

以下为同一年度主策略在 2025 年四个指定日期的结果；电量单位为 kWh，费用为元。当日完整电价在日前已知为本组结果的价格假设。三类表依次对应题目表 1、表 2 和表 3，数值均由未舍入账本计算。

表 15 4-2 指定购电计划与全天费用（对应题目表 1）

日期	10:00	12:00	14:00	16:00	18:00	20:00	全天量
03-20	0.00	570.00	0.00	489.34	54.52	60.28	68815.34
06-21	0.00	529.09	0.00	228.54	0.00	99.57	38732.07
09-23	0.00	507.99	0.00	576.67	142.97	626.01	68624.24
12-21	0.00	1003.34	0.00	853.45	728.24	49.66	98141.67

日期	计划费	紧急费	现金合计	实际电网电量
03-20	42370.26	767.41	43137.66	66464.66
06-21	18923.85	0.00	18923.85	33126.02
09-23	44670.80	0.00	44670.80	64989.47
12-21	74581.42	0.00	74581.42	92286.96

表中合同量为零点原计划 q ；时刻为十分钟区间起点。实际电网电量为 $a + e$ 。现金为计划费与紧急费之和；分项分别舍入，末位可能有差异。

表 16 4-2 四小时充放电与日初日末库存（对应题目表 2）

区间/时刻	03-20		06-21		09-23		12-21	
	充电	放电	充电	放电	充电	放电	充电	放电
00:00–04:00	2849.92	83.18	1368.27	1876.26	4147.49	18.65	3152.96	40.03
04:00–08:00	3373.48	5130.52	2801.70	1730.28	2514.00	5686.61	781.31	1397.38
08:00–12:00	5786.56	2777.38	3371.21	0.00	4230.40	1896.62	5286.46	6862.33
12:00–16:00	4049.86	375.30	5833.29	0.00	5398.45	594.47	6153.89	2220.11
16:00–20:00	220.85	6779.88	259.70	5618.83	290.20	6367.51	732.25	5298.81
20:00–24:00	4267.40	1983.07	0.00	2386.81	1903.66	2272.72	6195.70	3234.73
00:00 库存	5508.16		2536.51		4950.00		7997.07	
24:00 库存	4968.85		1904.84		2878.46		6898.95	

块内充放电均为正表示不同十分钟格先后动作，非同格充放。库存为电池内部储电量。

表 17 4-2 连续紧急购电事件（对应题目表 3）

03-20		06-21		09-23		12-21	
连续时段	电量	连续时段	电量	连续时段	电量	连续时段	电量
20:10–20:20	120.10	无	0.00	无	0.00	无	0.00

完整 144 格合同、全部四小时块及全年连续事件见result4-2.xlsx。其中计划表全天费用为含紧急费的最终现金。

7.5 4-3 指定日期的计划与执行

以下为同一年度主策略在 2025 年四个指定日期的结果；电量单位为 kWh，费用为元。当日完整电价在日前已知为本组结果的价格假设。三类表依次对应题目表 1、表 2 和表 3，数值均由未舍入账本计算。

表 18 4-3 指定购电计划与全天费用（对应题目表 1）

日期	合同	10:00	12:00	14:00	16:00	18:00	20:00	全天量
03-20	q	0.00	585.88	0.00	568.77	54.31	60.28	68984.23
03-20	m	0.00	557.83	0.00	568.77	54.31	60.28	68663.79
06-21	q	0.00	0.00	0.00	216.78	0.00	99.57	36000.45
06-21	m	0.00	0.00	0.00	183.81	0.00	99.57	35837.97
09-23	q	0.00	453.76	0.00	604.83	136.21	745.82	68403.96
09-23	m	0.00	421.04	0.00	604.83	136.21	745.82	67789.93
12-21	q	0.00	971.87	0.00	859.07	719.49	57.91	97231.86
12-21	m	0.00	971.87	0.00	859.07	719.49	57.91	96845.84

日期	计划费	调整净额	紧急费	现金合计	实际电网电量
03-20	42625.04	90.72	570.75	43286.51	66247.47
06-21	18068.51	-43.94	0.00	18024.57	32724.93
09-23	44608.36	-99.18	0.00	44509.18	64292.93
12-21	73778.34	-165.32	0.00	73613.02	92339.09

原计划 q 与最终生效额度 m 分别列示；时刻为十分钟区间起点。实际电网电量为 $a + e$ 。现金为计划费、调整净额与紧急费之和；分项分别舍入，末位可能有差异。

表 19 4-3 四小时充放电与日初日末库存（对应题目表 2）

区间/时刻	03-20		06-21		09-23		12-21	
	充电	放电	充电	放电	充电	放电	充电	放电
00:00–04:00	2811.02	83.18	1302.55	1813.46	4148.27	18.65	3159.35	40.03
04:00–08:00	3173.64	5465.43	2234.34	1730.35	2462.01	5796.64	781.31	1421.61
08:00–12:00	3850.85	2777.38	4088.75	0.00	4495.81	1896.62	5319.02	6811.22
12:00–16:00	6200.89	254.72	5611.26	0.00	5155.12	416.01	6288.88	2284.28
16:00–20:00	365.33	6579.90	397.35	5663.20	216.80	6446.36	613.54	5235.33
20:00–24:00	4262.81	2087.39	0.00	2421.20	1905.11	2192.08	6163.08	3231.38
00:00 库存	5543.17		2466.74		4950.00		7991.33	
24:00 库存	4976.81		1817.34		2865.52		6946.36	

块内充放电均为正表示不同十分钟格先后动作，非同格充放。库存为电池内部储电量。

表 20 4-3 连续紧急购电事件（对应题目表 3）

03-20		06-21		09-23		12-21	
连续时段	电量	连续时段	电量	连续时段	电量	连续时段	电量
12:30-12:50	154.06	无	0.00	无	0.00	无	0.00
20:10-20:20	17.00	—	—	—	—	—	—

完整 144 格合同、全部四小时块及全年连续事件见result4-3.xlsx。其中计划表全天费用为原计划费，调整表全天费用为最终现金，不能相加。

八、模型检验与灵敏度分析

8.1 评价设计与比较口径

检验围绕两类问题展开：一是模型与程序是否遵守物理约束、信息顺序和现金定义，二是新增的决策层在连续运行中产生了怎样的经验结果。年度评价使用 2025 年 2 月至 12 月的连续历史回放，问题二沿用前期探索形成的预测规则，其评价前的冻结时点尚无法追溯；问题三的数值权重仅用一月拟合，但剩余窗口校准方案是在全年诊断后修订，主配置也经完整轨迹比较选出。故本文将**年度结果界定为回顾性策略比较**，一月校准与后续目标的时间隔离另行保持。

主策略从共同入口独立跨日运行，现金比较同时列出末库存。表10的相邻差额沿指定配置序列计算，合计对应起终两条完整轨迹；每一步包含合同、控制和库存路径的共同变化。18 点比较同时更新预报、负载场景和合同，滞后参考的比较还涉及控制规则与量测条件，因而这些差额采用条件配置比较的解释。第四问直接沿用相同预热入口，评价期间再引入附件 4 价格。

全期 Oracle 预先取得全部实测，并允许联合安排整个评价期。表5的 Oracle 以 1200.00 kWh 结束，低于 F-fast 的 8264.39 kWh；直接相减会把终端可用能量的差异纳入费用差。将 Oracle 末库存约束为与 F-fast 相同后，重新优化所得下界为 1223.02 万元，F-fast 仍高于该下界 12.64%。这一比较统一了末端库存要求，但 Oracle 保留完全预知信息，只用于衡量所实现策略与确定性下界的距离。

式(12)以历史相关系数修正残差锚点，在保留历史路径后续形态的同时近似当前条件场景；这并非已知的真实条件分布。预测误差检验与控制费用检验因此分别进行：前者衡量指定目标上的误差，后者评估有限场景、库存估值与执行规则共同形成的运行结果。

8.2 局部模型与年度账本核验

问题一的主目标、对偶下界、物理平衡和首尾恢复误差均在各自单位下的 10^{-6} 以内；14 种已保存情形通过物理账本及显式互斥对照，固定种子的构造例补充检查边界行为。单独放宽库存上界 10 kWh、下界 10 kWh 和充电功率 100 kW 时，费用分别减少

6.55、6.59、5.65 元；放电功率增加 100 kW 不降费，与当前峰值 4293.92 kW 未触及上限相符。对偶预测与重算差小于 10^{-6} 元，完整结果见表24。设备及离散解释的响应见图10，其中 90% 改按往返效率解释时费用为 33801.50 元；设备容量翻倍的比较同时改变库存边界与初值，反映这一组合设置的费用响应。

问题二在三个真实日初库存下，仅扰动两日 LP 连接等式 ± 0.1 内部 kWh，正、反向有限差分均与 $\lambda = -\mu$ 一致，最大差为 1.984×10^{-11} 元/内部 kWh（表25），核对了局部影子价格的符号与单位。10 组年度结果通过**逐格现金与物理重算**，18 个构造例的状态增量 LP 与显式互斥模型同值；改变尚不可用的未来实测后，**日前计划及此前动作保持不变**。F-fast 有 99 格同时出现紧急电与充电，紧急电合计 21803.43 kWh；固定其他当期量去掉充电后，缺口减少 20132.37 kWh，仍余 1671.06 kWh。这一分解量化当期充电与紧急需求的重叠，年度可避免支出还取决于后续库存与重新决策，未由该局部计算确定。详见附录B。

问题三两类规划器与另写的显式互斥 MILP 在 48 个小例中最大目标差为 7.11×10^{-15} 元，两者共用 HiGHS 求解库。五条当前轨迹共 240480 格通过费用、物理、计划封存、合同生效和日际库存核验，电量/库存最大差 1.27×10^{-11} kWh、费用最大差 0.00 元。主配置沿真实日初和发布时库存重解 1002 次改约 LP，覆盖 72144 个发布窗口位置及 36072 个正式生效位置。固定同次 LP 的储能动作后，逐点核对合同子问题的左右斜率夹住零，正式生效段与年度合同误差小于 10^{-6} 。窗口重叠不作为独立样本，4-3 两种价格情形另见本章核验。

8.3 场景、设备与惩罚参数的响应

F-fast 的对照中，直接使用原始历史日费用增加 24.58%，而打乱日内场景次序后费用降低 0.15%；本数据未表现出保留整日相关性的额外费用收益。自由、硬约束和影子价格三种终端处理分别产生 1380.89、1389.07、1377.60 万元现金费，末库存分别为 1490.19、9613.97、8264.39 kWh。硬终端要求日末不低于日初，使跨日边界库存单调不降；影子价格则以有限奖励保留后续库存用途。滞后参考加当前量测保护的费用比 F-fast 高 2.79%，超过所采用的 2% 容忍度，本文据此保留 F-fast，并沿用图2的当前区间量测条件。各完整对照见表26至表28。

表 21 S 策略六组扫描的全部 19 个设置与 334 日现金费

因素	设置（依次对应）	现金费/万元（依次对应）
终端处理	自由、影子价格、硬终端	1431.13、1428.85、1441.39
场景数 K	10、20、30、50	1443.21、1434.80、1428.85、1427.70
单程效率	0.85、0.90、0.95	1477.43、1428.85、1380.89
库存上界/kWh	7200、10800、14400*	1545.05、1428.85、1346.09
充放电功率/kW	2500、5000、7500	1474.23、1428.85、1421.35
紧急购电倍率	2、5、8	1341.20、1428.85、1470.88

各组按自身设置完成一月预热，比较包含入口库存效应；现金费不含预热费、不扣终值。* 表示假设设备扩容至 16000 kWh，库存下界仍为 1200 kWh。各组扰动幅度不同，费用响应仅在各自设置内比较；逐项指标见表29。

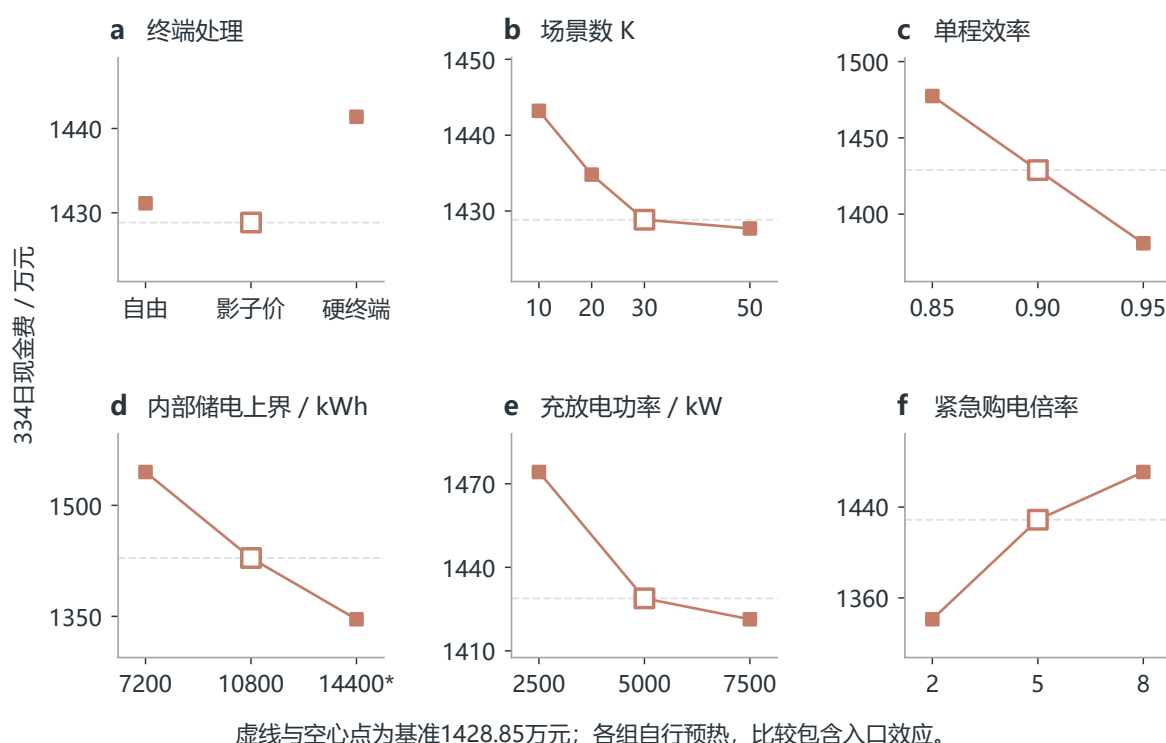


图 8 S 策略六组参数扫描的现金响应。各点对应表21的已运行设置，空心点为组内基准；面板采用各自纵轴。

在已测水平上，增加场景数、效率、库存上界或功率上限均对应较低现金费；紧急倍率从 2 增至 5、8 时，紧急电量依次为 682811.5、218870.8、140282.3 kWh。惩罚提高后，策略使用更少紧急电，现金却因计价倍数与决策共同变化而上升。终端组的日末库存中位数依次为 1621、8274、10800 kWh，描述 334 个日末状态的中心水平，与评价期最后一天的末库存是不同统计量。**14 个不同设置均从原始附件重算**；重复基准共用同一轨迹，构成表中 19 点，费用在表列精度下逐项一致。结论限于这些设置的联合响应，未计设备投资与寿命成本。

8.4 预测误差与 18 点更新的负结果

校准目标从完整 24 小时修订为当天剩余窗口后，改约所关心的目标得到更直接的拟合，但更远目标的预测可能发生变化。在完全相同目标上，修订后的冻结评价期当天剩余 MAE 由 19.1413 降至 18.1569 kWh/十分钟目标，完整 24 小时 MAE 却由 21.5739 升至 29.5197。剩余窗口各发布时间有 334 日，非零时刻的完整路径有 333 日；重叠目标用于误差汇总，不视为独立样本。逐项配对见表30。完整 24 小时残差仍参与 AR 参数估计，其误差变化也可能通过场景构造影响控制。

分发布时间看，12 点剩余窗口 MAE 从 17.7663 降至 12.3355，18 点却从 0.4802 升至 1.6692；18 点权重为 1，同期历史预测 MAE 仅 0.3675。18–24 点实际光伏日均 116.79 kWh、中位数 4.18 kWh，日均绝对预测误差和 60.09 kWh。图9并列报告两类窗口，保留这种改善与退化并存的结果。

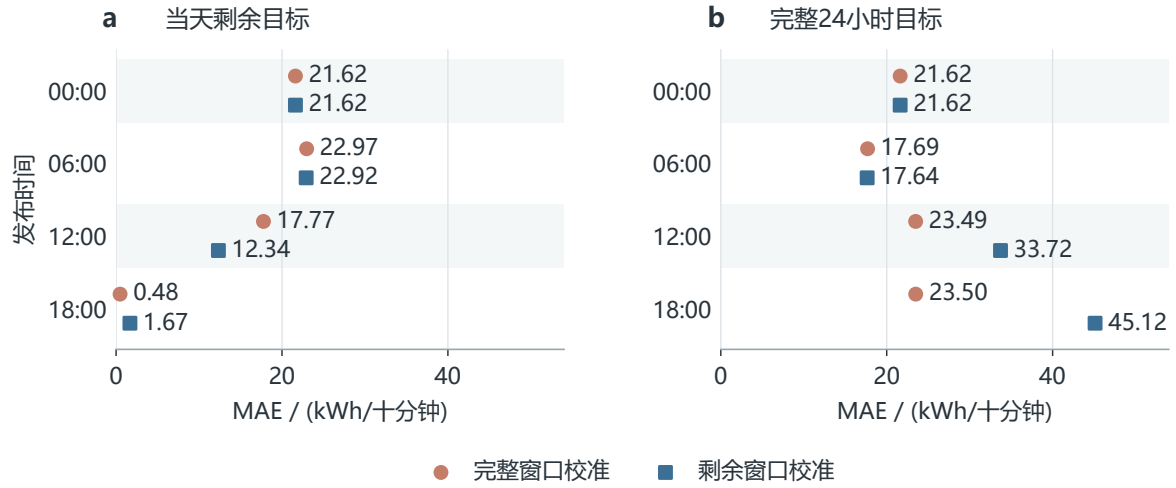


图 9 两种校准口径在共同目标上的预测误差。同组点纵向错开以避免遮挡；两面板横轴同尺，权重仅用一月数据拟合

为考察晚间误差的费用量级，固定主轨迹的合同 m 、负载及充放电动作，记 $e(V) = [L + c - b - m - V]_+$ 。由正部函数的 1-Lipschitz 性质，

$$\left| \sum_d \sum_{t=108}^{143} 5p_t \{e(\hat{V}_{d,t}) - e(V_{d,t})\} \right| \leq \sum_d \sum_{t=108}^{143} 5p_t |\hat{V}_{d,t} - V_{d,t}| \quad (25)$$

$$= 126143.02 \text{ 元.}$$

该界在 334 日的 12024 格上核对成立，约为主方案现金费的 0.93%，但高于加入 18 点后的综合现金减少 34904.57 元。因此，晚间光伏量级较小并不足以说明误差影响可以忽略。该界的适用对象是固定合同与动作下的紧急费用变化；重新优化和后续库存影响在范围之外，18 点光伏误差对最终费用的独立贡献仍未量化。

8.5 价格检验与综合证据边界

相同 48096 格上，前一日曲线与附件 1 固定曲线预测实际电价的 MAE 分别为 0.084326、0.096323 元/kWh。该描述性诊断说明两种简单价格参照在本期数据中的误差，既定规则未据此重选。式(24)按固定十分钟时段跨 334 日展开，均价项与协方差项之和复现换价费用差，重新决策项则另列库存变化（表34）。价格预测和控制效果据此分开评价。

第四问两类策略均完成全年物理与现金核对。4-3 两条轨迹共 96192 格通过标量费用、库存连续、计划封存及合同生效检查；两种电价口径的固定储能分位核验各覆盖 334 日 1002 次 LP。已有信息测试中，改变尚不可用的未来价格不改变决策，固定动作只改结算价仅改变现金，恢复附件 1 价格在三个完整日复现第三问。这些结果连接了数学子问题、程序动作和实际账本。全年策略仍采用有限场景、共享路径和逐步重解近似，其结论是所实现策略在已说明历史条件下的运行表现；全局随机最优性与实际控制时延下的现场效果未由上述核验确立。

九、 模型评价与推广

本文以统一的能量平衡和现金定义连接四问，在储能调度上依次增加不确定性反馈、合同修订与日际价格信息。结果表明，日内反馈和合同更新分别调整物理动作与购电额度，二者须通过库存状态配合；储能的节费空间同时受转换损耗和可用容量限制。**预测误差与运行费用也应分别评价：**18 点误差的负结果未对应所有相关策略费用上升，平均电价下降则可被价格与计费权重的正向协变超过。

模型仍采用有限历史场景和局部续期价值近似，年度证据适用于第八章所述的连续历史回放，**末库存与现金费用须共同阅读**。推广时，可先按实际量测延迟调整控制时序，在统一终端条件和新的独立数据上检验稳定性，再引入电池寿命成本与可用价格预测，评估设备容量和功率配置，将运行费用比较延伸为包含投资与退化的长期决策。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于候选建模方法与局部推导、程序实现、实验与数值核验、数值图与文稿草案、通用设备示意素材生成、可编辑概念图及排版、审阅意见核对，详细使用情况见支撑材料。

参考文献

- [1] 王成山, 李鹏. 分布式发电、微网与智能配电网的发展与挑战 [J]. 电力系统自动化, 2010, 34(2): 10–14, 23.
- [2] Boyd S, Vandenberghe L. *Convex Optimization*. Cambridge University Press, 2004.
- [3] Joshi A, Kebriaei H, Mariani V, Glielmo L. A Sufficient Condition to Guarantee Non-Simultaneous Charging and Discharging of Household Battery Energy Storage. arXiv:2104.06267, 2021.
- [4] 李嘉伟, 巨云涛, 张璐, 等. 基于分布鲁棒模型预测控制的微电网多时间尺度优化调度 [J]. 电力工程技术, 2024, 43(4): 45–55. DOI: 10.12158/j.2096-3203.2024.04.005.

附录 A 结果文件使用

问题一指定时段已列于正文；问题二、三、4-2 和 4-3 的指定日期结果分别见第5.6、6.6、7.4、7.5节，不再在附录重复列数。五份结果表按原模板填写，所有主结果取自相应连续轨迹。

result1.xlsx 为问题一；result2.xlsx、result3.xlsx、result4-2.xlsx、result4-3.xlsx 分别保留其余问题完整 144 格合同、六个四小时块及全部连续紧急事件。问题二和 4-2 计划表的全天费用为最终现金；问题三和 4-3 计划表为原计划费、调整表为最终现金，两表费用不能相加。第四问主表使用当日完整价格日前已知的假设。

附录 B 必要推导与局部核验

B.1 时标及最优解的非唯一性

附件末端标签 00:10 对应物理区间 00:00–00:10、模型索引 0；标签 10:10 对应 10:00–10:10、索引 60；标签 0:00+1 对应 23:50–24:00、索引 143。结果表保留原模板文字并按位置填写，不能把更换标签当作重新优化。若采用左端点周期解释并同步重排价格、负载和光伏，重新求解后的六格购电依次为 0.00、486.40、0.00、394.93、636.98、0.00 kWh，全天费用 35126.85 元。

表 22 附件标签、物理区间与模型位置的对应

附件末端标签	物理十分钟区间	模型索引	结果表位置
00:10	00:00–00:10	0	第 1 格
10:10	10:00–10:10	60	第 61 格
12:10	12:00–12:10	72	第 73 格
0:00+1	23:50–24:00	143	第 144 格

B.2 问题一最优面的显式展开

在主费用最优面内，对 289 个购电与库存坐标逐个求上下界，两个购电坐标和两个中间状态可变，其余坐标在 10^{-6} 以内固定。以交付计划为起点，在 23:10–23:20 增加购电 θ ，在 23:30–23:40 等量减少，可写为

$$\begin{aligned} g_{139}(\theta) &= 569.35863333 + \theta, \\ g_{141}(\theta) &= 904.94871667 - \theta, \quad 0 \leq \theta \leq 1000/3 \text{ kWh}. \end{aligned} \quad (26)$$

23:20 与 23:30 两个边界的库存各增加 0.9θ ，至多 300 kWh，其余动作与状态不变。两格同价，费用与充放总量均不变。因此先最小化费用、再最小化充放总量仍不能消除该组多解；六个指定时段购电值固定，也不意味着整日调度唯一。端点调度可由所附程序重新生成。

B.3 问题一各参数设置的完整费用

表 23 解释与参数对照：各情形均重新优化

情形及主要变化	购电费/元	较基准变化/元
往返效率 90%	33801.50	-1325.45
左端点周期映射	35126.85	-0.10
相邻点均值	35205.89	+78.94
初始储电可优化	35118.60	-8.35
无损储能	32377.09	-2749.86
充放电功率减半	36390.84	+1263.89
充放电功率翻倍	34960.59	-166.36
可用储电窗口减半	40020.40	+4893.46
设备容量翻倍	30145.96	-4980.99

设备容量翻倍一项同时调整名义容量、库存边界与初值，相邻点均值是离散近似；各设置均在同一预测日重新优化，不计投资或衰减。

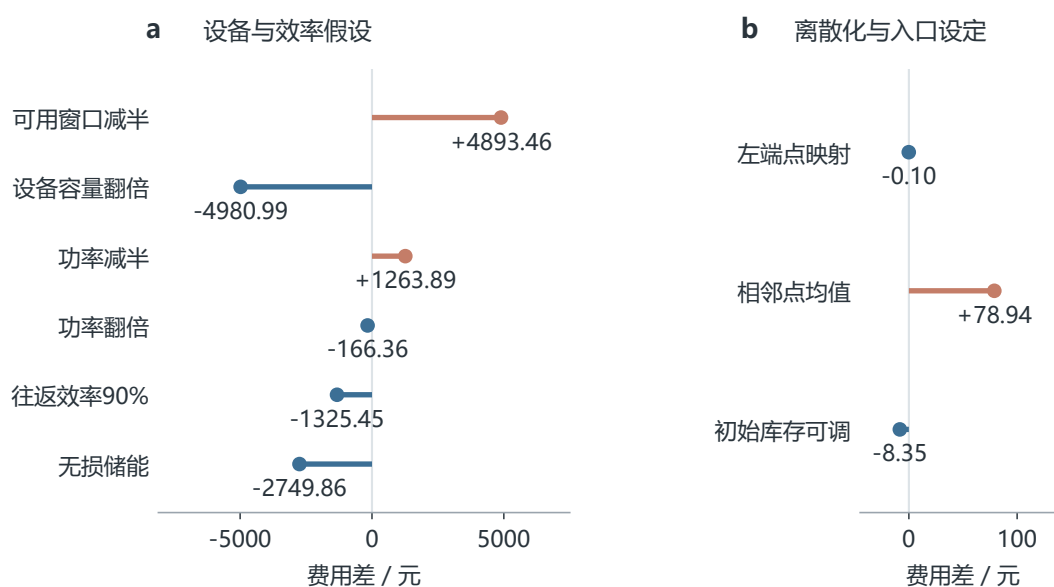


图 10 问题一设备假设与离散解释的费用响应。各设置重新优化，两个面板的尺度及扰动幅度不同；容量翻倍同时调整关联边界与初值

B.4 问题一边界扰动的对偶核对

表中每次独立改变一项边界；未舍入的对偶预测与重算差在 10^{-6} 元以内。活跃约束改变后局部斜率不必保持，不能将其外推为全年投资排序。

表 24 局部边界变化的对偶预测与实际重算

独立改变的参数	预测费用变化/元	重算费用变化/元
储电上限增加 10 kWh	-6.55	-6.55
储电下限减少 10 kWh	-6.59	-6.59
充电功率增加 100 kW	-5.65	-5.65
放电功率增加 100 kW	0.00	0.00

B.5 两日库存价值的有限差分

在 2025 年 2 月 1 日、4 月 11 日及 9 月 22 日的真实入口库存下，仅用当天零点前的信息预测两日负载和光伏。固定入口、预测、价格、两日末端下界及其余约束，只改变两日连接等式右端 ± 0.1 内部 kWh。令 $J(\delta)$ 为扰动目标，则

$$D_+ = \frac{J(0) - J(+0.1)}{0.1}, \quad D_- = \frac{J(-0.1) - J(0)}{0.1}. \quad (27)$$

表 25 两日库存价值的局部扰动核对；斜率单位为元/内部 kWh

日期	日初 S_0/kWh	$\lambda = -\mu$	D_+	D_-
2025-02-01	8657.42	0.472000000	0.472000000	0.472000000
2025-04-11	3596.45	0.466444444	0.466444444	0.466444444
2025-09-22	8550.00	0.472000000	0.472000000	0.472000000

附录 C 补充对照

C.1 问题二场景与终端处理

表 26 F-fast 与滞后参考加当前量测保护的选型对照

方案	334 日现金费/元	入口 S_0/kWh	末库存/kWh
F-fast	13776041.14	8657.42	8264.39
滞后参考加当前量测保护	14160835.20	8657.42	8173.27

表 27 F-fast 的场景与可用观测对照；费用单位万元

方案	总费用	相对主模型	状态
F-fast（影子价格）	1377.60	0.00%	334 天可行
G0 原始历史日	1716.28	24.58%	334 天可行
G2 打乱日内路径	1375.47	-0.15%	334 天可行
旧滞后：保留额外首步界	1656.08	20.21%	历史 334 天可行

G0 直接使用历史原始曲线；G2 逐格打乱场景编号并保持同格负载、光伏配对。旧滞后行保留式(13)的额外首步净缺口界，与当前“滞后参考+当前量测保护”不是同一策略，其 20.21% 差额不能解释为纯观测时差的代价。

表 28 F-fast 的三种日终处理：费用单位万元，末内部储电量单位 kWh

处理	总费用	评价期末内部储电量	状态
自由终端	1380.89	1490.19	334 天可行
硬终端 $S_{144} \geq S_0$	1389.07	9613.97	334 天可行
影子价格	1377.60	8264.39	334 天可行

表 29 S 策略扫描的附加指标：费用及全部 19 个设置见正文表21

因素	设置（依次对应）	附加指标（依次对应）
终端	自由、影子价格、硬终端	日末中位库存 1621、8274、10800 kWh
效率	0.85、0.90、0.95	往返效率 0.7225、0.8100、0.9025
库存上界	7200、10800、14400 kWh	可用范围 6000、9600、13200 kWh
功率	2500、5000、7500 kW	每格上限 416.67、833.33、1250 kWh
紧急倍率	2、5、8	紧急电 682811.5、218870.8、140282.3 kWh

各组按自身设置从 6000 kWh 完成一月预热；比较包含入口库存效应。日末中位数与评价期最后一天的末库存属于不同统计量。

C.2 问题三同目标预测误差的完整配对

较早方案采用完整窗口校准，本文采用当天剩余窗口校准。误差单位为 kWh/十分钟目标；版本差额仅对表中共同目标计算。

表 30 校准窗口修订前后在完全相同目标上的 MAE

期间	窗口	共同目标数	修订前	修订后	后减前
一月逐次回测	完整 24 小时	17424	17.8160	22.5007	4.6847
一月逐次回测	当天剩余	11160	15.4087	15.1170	-0.2918
二至十二月冻结评价	完整 24 小时	191952	21.5739	29.5197	7.9458
二至十二月冻结评价	当天剩余	120240	19.1413	18.1569	-0.9844

表 31 冻结评价期各发布时间的当天剩余 MAE (kWh / 十分钟目标)

发布时间	历史预测	官方预测	融合预测
0:00	25.0247	32.9152	21.6201
6:00	33.1706	26.4194	22.9162
12:00	24.1365	12.5421	12.3355
18:00	0.3675	1.6692	1.6692

完整预测接口与历史残差仍为 144 格。剩余窗口在冻结评价期各有 334 日；非零发布时间的完整窗口因跨年目标缺失各有 333 日。误差比较按共同目标配对，重叠目标不视为独立样本。

C.3 问题三各配置的现金与末库存

三位开关依次代表 6、12、18 点，表中均为完整连续运行，不能逐日混用。

表 32 第三问各配置的 334 日现金费与末库存

配置	现金费 / 万元	末库存 / kWh
Z1/000	1361.23	8264.39
Z1/010	1359.62	8264.39
Z1/011	1356.34	8264.39
Z1/111	1354.98	8264.39
Z2/110	1353.81	8303.26
Z2/111	1350.32	8303.87

C.4 问题四电量与换价分解

表 33 问题四年度电量 (万 kWh; 生效额度在 4-2 等于原计划)

指标	4-2 已知价	4-2 前日价	4-3 已知价	4-3 前日价
原计划量	2210.82	2210.40	2177.01	2176.61
生效额度	2210.82	2210.40	2166.63	2165.83
紧急电	7.14	7.55	4.14	4.59
未调用额度	155.01	152.55	116.28	114.30
弃光	145.55	148.74	135.94	138.14
充电	651.17	645.97	657.20	652.95
放电	527.68	523.23	532.56	528.88

表 34 固定动作换价与重新决策的现金核算（元）

项目	问题二至 4-2	问题三至 4-3
固定价下原现金	13776041.14	13503176.37
原动作按附件 4 换价	14515649.28	14229792.29
换价费用差	739608.14	726615.92
其中：时段均价项	-82286.75	-77014.25
其中：跨日协方差项	821894.89	803630.17
已知价下重新决策相对换价账本	-82393.33	-81512.27
原动作末库存 / kWh	8264.39	8303.87
重新决策末库存 / kWh	6027.22	6134.92

附录 D 程序说明与支撑文件

D.1 计算入口

计算采用 Python、NumPy、SciPy 和 openpyxl。先安装requirements.txt 所列版本，在支撑目录执行 `python run.py --case all`，按官方附件重新计算问题一、共同预热、年度配置和 S 策略扫描。问题四的 4-3 配置由本次问题三的五条完整年度结果选定。`python verify.py` 完成相应数学核验，`python export.py` 生成五份模板结果表。年度计算需要一定时间，`python run.py --list` 可列出可单独运行的配置。

原始附件保持只读；输出写入output/。每条轨迹都检查十分钟物理平衡、费用重算、计划封存和跨日库存连续。运行入口使用本次计算得到的一月末库存，不以保存的年度账本代替优化。文件哈希、数值核对和运行范围见verification.json。

D.2 支撑材料逐文件清单

支撑压缩包共 60 个文件，逐项列于下表。INPUTS.json 记录官方输入哈希，SUPPORT_MANIFEST.json 记录包内文件哈希；results_summary.json 保留全部配置及数学核验的汇总。完整逐时轨迹在运行后生成。

figures/ 包含论文全部 10 图的复现入口、图源数据、配色、图号对应表及选定素材。安装该目录的依赖后，在支撑包解压目录运行 `python figures/build.py`，即可生成figures/rendered/ 中的图形及排版副本。图一、图二复制既定素材，其余 8 图由 CSV 绘制；绘图不重新运行年度优化。完整绘图程序随支撑包提供。

序号	文件相对路径	序号	文件相对路径
1	AI 工具使用详情.pdf	31	model/explicit_check.py
2	INPUTS.json	32	model/fusion.py
3	README_复现说明.md	33	model/history.py
4	SUPPORT_MANIFEST.json	34	model/lagged.py
5	configs/q1_v2.json	35	model/oracle.py
6	export.py	36	model/planning.py
7	figures/FIGURE_MAP.csv	37	model/prices.py

序号	文件相对路径（续）	序号	文件相对路径（续）
8	figures/INPUT_MANIFEST.json	38	model/publication.py
9	figures/README.md	39	model/q1.py
10	figures/assets/F01_system_supplied.png	40	model/remaining.py
11	figures/assets/F02_questions_info.png	41	model/settlement.py
12	figures/build.py	42	problem/attachments/C 题/附件 1.xlsx
13	figures/data/F04_q1_dispatch.csv	43	problem/attachments/C 题/附件 2.xlsx
14	figures/data/F05_q1_sensitivity.csv	44	problem/attachments/C 题/附件 3.xlsx
15	figures/data/F06_q2_annual.csv	45	problem/attachments/C 题/附件 4.xlsx
16	figures/data/F08_s_scans.csv	46	requirements.txt
17	figures/data/F09_frozen_weights.csv	47	result1.xlsx
18	figures/data/F09_q3_forecast.csv	48	result2.xlsx
19	figures/data/F10_q3_policies.csv	49	result3.xlsx
20	figures/data/F11_q4_prices.csv	50	result4-2.xlsx
21	figures/data/F12_q4_costs.csv	51	result4-3.xlsx
22	figures/palette.json	52	results_summary.json
23	figures/requirements.txt	53	run.py
24	model/__init__.py	54	templates/result1.xlsx
25	model/amendment_prices.py	55	templates/result2.xlsx
26	model/amendments.py	56	templates/result3.xlsx
27	model/contract_cash.py	57	templates/result4-2.xlsx
28	model/contracts.py	58	templates/result4-3.xlsx
29	model/control.py	59	verification.json
30	model/dynamic_prices.py	60	verify.py

附录 E 完整计算程序

以下完整列出支撑包中的计算、数学核验和结果导出程序。不同问题共用预测、规划与结算模块。文件相对路径与支撑清单一致；空的model/___init__.py 仅在清单列示。

run.py: 统一计算入口

```
1 """Recompute the paper from the four official workbooks (no saved results read)."""
2 import argparse
3 from concurrent.futures import ProcessPoolExecutor, as_completed
4 from dataclasses import asdict, replace
5 import hashlib
6 import json
7 import os
8 from pathlib import Path
9 from time import perf_counter
10
11 import numpy as np
12 from model.history import ForecastArchive, load_data
13 from model.planning import Physical
14 from model import control as q2
15 from model import lagged as lagged
16 from model.remaining import FusedForecastArchive
17 from model import amendments as q3
18 from model import dynamic_prices as q42
19 from model.prices import load_prices
20 from model.amendment_prices import build_price_inputs
21
22 ROOT = Path(__file__).resolve().parent
23
24 def save(path, value):
25     path = Path(path)
26     path.parent.mkdir(parents=True, exist_ok=True)
27     path.write_text(json.dumps(value, ensure_ascii=False, indent=2,
28                               default=lambda x: x.tolist() if isinstance(x, np.ndarray) else float(x)),
29                     encoding='utf-8')
30
31 def fingerprints():
32     paths = [Path(__file__), *sorted((ROOT/'model').rglob('*.*py')),
33           *sorted((ROOT/'problem').rglob('*.*xlsx')), ROOT/'configs/q1_v2.json']
34     return {p.relative_to(ROOT).as_posix(): hashlib.sha256(p.read_bytes()).hexdigest() for p
35           in paths}
36
37 def designs():
38     cases = {'q2/{k}': ('q2', config, Physical(), False)
39             for k, config in q2.PRIMARY_CONFIGS.items() if k != 'F_lagged_guard'}
40     cases['q2/lagged_reference'] = ('lagged',
41                                     lagged.PRIMARY_CONFIGS['F_lagged_no_min_guard'], Physical(), False)
42     for label, options in [('G0_raw', dict(scenario_method='raw')),
43                           ('G2_shuffled', dict(scenario_method='shuffled')),
44                           ('terminal_free', dict(terminal='free')),
45                           ('terminal_cyclic', dict(terminal='cyclic'))]:
46         cases[f'q2/{label}'] = ('q2', replace(q2.PRIMARY_CONFIGS['F_fast'], **options),
47                                 Physical(), False)
48     for label, model, hours in [('Z1_000', 'Z1', ()), ('Z1_010', 'Z1', (12,)),
49                               ('Z1_011', 'Z1', (12,18)), ('Z1_111', 'Z1', (6,12,18)),
50                               ('Z2_110', 'Z2', (6,12)), ('Z2_111', 'Z2', (6,12,18))]:
51         config = q3.Config(label, model, hours, bool(hours), controller='fast')
52         cases[f'q3/{label}'] = ('q3', config, Physical(), False)
53     for mode in ('known_day_ahead', 'previous_day_forecast'):
54         cases[f'q4_2/{mode}'] = ('q42', mode, Physical(), False)
```

```

51     # The selected Q3 configuration is filled from the fresh annual comparison.
52     cases[f'q4_3/{mode}'] = ('q43', mode, Physical(), False)
53     variants = [('base', {}, {})]
54     variants += [(f'terminal_{x}', dict(terminal=x), {}) for x in ('free', 'cyclic')]
55     variants += [(f'K_{k}', dict(scenario_count=k), {}) for k in (10,20,50)]
56     variants += [(f'eta_{v}', {}, dict(eta_c=v, eta_d=v)) for v in (.85,.95)]
57     variants += [(f'upper_{v}', {}, dict(s_max=v)) for v in (7200.,14400.)]
58     variants += [(f'power_{v}', {}, dict(power_energy=v/6)) for v in (2500.,7500.)]
59     variants += [(f'emergency_{v}', {}, dict(emergency_multiple=v)) for v in (2.,8.)]
60     for label, config_values, physical_values in variants:
61         cases[f'S_scan/{label}'] = ('q2', replace(q2.PRIMARY_CONFIGS['S'], **config_values),
62                                     replace(Physical(), **physical_values), True)
63     return cases
64
65 def one_day(kind, day, stock, archive, config, physical, prices=None, conditional=None):
66     if kind in ('q2', 'lagged'):
67         plan, _ = q2.make_plan(day, stock, archive, config, physical)
68         plan.q.setflags(write=False)
69         sealed = plan.q.copy()
70         engine = lagged if kind == 'lagged' else q2
71         ledger, execution = engine.execute_day(day, stock, plan, archive, config, physical)
72     elif kind == 'q42':
73         origin = prices.at_origin(day, config)
74         plan, _, view = q42.make_plan(day, stock, archive, origin, physical)
75         plan.q.setflags(write=False)
76         sealed = plan.q.copy()
77         actions, execution = q42.execute_actions(day, stock, plan, view, physical)
78         ledger, audit = q42.settle_actual(plan, actions, archive.data.load[day],
79                                         archive.data.pv[day], stock, prices.settle_price(day), physical)
80         execution['independent_recalculation'] = audit
81     else:
82         plan = q3.make_plan(day, stock, archive, prices, config, physical)
83         plan.q.setflags(write=False)
84         sealed = plan.q.copy()
85         ledger, execution = q3.execute_day(day, stock, plan, archive, prices, config,
86                                           physical, conditional=conditional)
87     assert np.array_equal(sealed, plan.q), 'The zero-time plan was changed'
88     assert execution['independent_recalculation']['passed'], 'Independent settlement failed'
89     return ledger, execution
90
91 def window(name, kind, config, physical, start, stop, initial_soc, destination):
92     data = load_data(ROOT)
93     archive = ForecastArchive(data, cold_start='scaled')
94     prices = conditional = None
95     if kind in ('q3', 'q43'):
96         archive = FusedForecastArchive(data, root=ROOT, load_archive=archive)
97         conditional = q3.ConditionalArchive(archive)
98         prices = q3.PriceInputs.q3(data)
99         if kind == 'q43':
100             mode, config = config
101             prices = build_price_inputs(data, load_prices(ROOT, data.dates), mode)
102     elif kind == 'q42':
103         prices = load_prices(ROOT, data.dates)
104     folder = Path(destination)/name
105     folder.mkdir(parents=True, exist_ok=True)
106     identity = dict(name=name, engine=kind, config=asdict(config) if not
107                  ↪ isinstance(config,str) else config,
108                  physical=asdict(physical), start=start, stop=stop,
109                  ↪ initial_soc=initial_soc,
110                  input_and_source_hashes=fingerprints())
111     manifest = folder/'manifest.json'
112     if manifest.exists():
113         assert json.loads(manifest.read_text('utf-8')) == identity, 'Existing run has
114             ↪ different inputs or code'
115         completed = folder/'summary.json'

```

```

113         if completed.exists():
114             return json.loads(completed.read_text('utf-8'))
115     save(manifest, identity)
116     rows = []
117     stock = float(initial_soc)
118     guards = 0
119     began = perf_counter()
120     for day in range(start, stop):
121         ledger, execution = one_day(kind, day, stock, archive, config, physical, prices,
122             ↪ conditional)
123         arrays = {k: v for k,v in ledger.items() if isinstance(v, np.ndarray) and v.shape ==
124             ↪ (144,)}
125         arrays.update(load=data.load[day], pv=data.pv[day])
126         if kind in ('q3','q43'):
127             arrays['price'] = prices.settlement[day]
128         elif kind == 'q42':
129             origin = prices.at_origin(day, config)
130             arrays.update(decision_price=origin.decision,
131                 ↪ settlement_price=prices.settle_price(day),
132                     nextday_price_forecast=origin.next_day_forecast)
133         rows.append(arrays)
134         stock = float(ledger['soc_end'][-1])
135         guards += execution['guard_count']
136         # Residual and controller caches are keyed by the current day only.
137         # Historical point forecasts stay cached; expired day buffers can go.
138         for owner in (archive, getattr(archive,'load_archive',None), conditional):
139             for key in ('_errors','_residuals','_conditional_bases','_base'):
140                 cache = getattr(owner,key,None)
141                 if isinstance(cache,dict): cache.clear()
142         if day == start or (day+1)%30 == 0 or day+1 == stop:
143             progress = dict(case=name, completed_days=len(rows), total_days=stop-start,
144                 ↪ cash_yuan=float(sum(r['cash_cost'].sum() for r in rows)),
145                     final_soc=stock, seconds=perf_counter()-began)
146             save(folder/'progress.json', progress)
147             print(json.dumps(progress), flush=True)
148         combined = {k: np.stack([r[k] for r in rows]) for k in rows[0]}
149         combined['d'] = np.arange(start,stop)
150         assert np.max(np.abs(combined['soc_end'][:-1,-1]-combined['soc_start'][:1,0]),
151             ↪ initial=0.) < 1e-6
152         np.savez_compressed(folder/'ledger.npz', **combined)
153         summary = dict(status='COMPLETED', days=stop-start, initial_soc=initial_soc,
154             ↪ final_soc=stock,
155                 totals={k:float(v.sum()) for k,v in combined.items() if k not in
156                     ↪ ('d','soc_start','soc_end')},
157                     daily_terminal_quantiles=np.quantile(combined['soc_end'][:, -1],
158                     ↪ [0,.05,.25,.5,.75,.95,1]),
159                     guard_count=guards, independent_recalculation_all_passed=True,
160                     interday_soc_continuous=True, sealed_plan_unchanged=True,
161                     ↪ seconds=perf_counter()-began)
162         save(folder/'summary.json', summary)
163     return summary
164
165 def run_case(name, destination, initial_soc, stop=365):
166     kind, config, physical, own_warmup = designs()[name]
167     if own_warmup:
168         warm = window(name+'warmup', kind, config, physical, 0,31,6000.,destination)
169         initial_soc = warm['final_soc']
170     if kind == 'q43':
171         selected = json.loads((Path(destination)/'q3/selection.json').read_text('utf-8'))['s']
172         ↪ elected']
173         config = (config, designs()[selected][1])
174     return window(name, kind, config, physical, 31,stop,initial_soc,destination)
175
176 def run_q1(destination):
177     from model import q1 as model

```

```

169 base = json.loads((ROOT/'configs/q1_v2.json').read_text('utf-8'))
170 values, _ = model.read_inputs(base)
171 summaries = {}
172 for name in model.DEFAULT_CASES + ['upper_plus_10', 'lower_minus_10', 'charge_power_plus_1'
↳ 00', 'discharge_power_plus_100']:
173     config = model.case_config(base, name)
174     data = model.interpreted(values, config['time_interpretation'])
175     price, load, pv = data[:,0], data[:,1]*config['step_hours'],
↳ data[:,2]*config['step_hours']
176     solved = model.solve(price, load, pv, config)
177     folder = Path(destination)/'q1'/name
178     folder.mkdir(parents=True, exist_ok=True)
179     np.savez_compressed(folder/'solution.npz', **solved['solution'], price=price,
↳ load=load, pv=pv)
180     summaries[name] = model.metrics(solved['solution'], price, load, pv, config)
181     save(folder/'summary.json', summaries[name])
182 save(Path(destination)/'q1/comparison.json', summaries)
183 return summaries
184
185 def select_q3(destination):
186     names = [name for name in designs() if name.startswith('q3/') and name != 'q3/Z1_000']
187     costs = {name: json.loads((Path(destination)/name/'summary.json').read_text('utf-8'))['t'
↳ otals']['cash_cost'] for name in
↳ names}
188     result = dict(selected=min(costs, key=costs.get), annual_cash_yuan=costs,
189                     rule='minimum verified complete annual cash among the five fixed
↳ configurations')
190     save(Path(destination)/'q3/selection.json', result)
191     return result
192
193 def main():
194     parser = argparse.ArgumentParser(description=__doc__)
195     parser.add_argument('--case', default='all', help='all, warmup, q1, or a case listed by
↳ --list')
196     parser.add_argument('--list', action='store_true')
197     parser.add_argument('--workers', type=int, default=4)
198     parser.add_argument('--out', type=Path, default=ROOT/'output')
199     parser.add_argument('--stop', type=int, default=365, help='exclusive day index; 365 is
↳ the full evaluation')
200     args = parser.parse_args()
201     save(args.out/'process.json', dict(pid=os.getpid(), case=args.case, workers=args.workers))
202     if args.list:
203         print('\n'.join(['q1', 'warmup', *designs()]))
204         return
205     if args.case in ('all', 'q1'):
206         run_q1(args.out)
207         if args.case == 'q1':
208             return
209     warm = window('warmup', 'q2', q2.PRIMARY_CONFIGS['F_fast'], Physical(),
↳ 0, 31, 6000., args.out)
210     if args.case == 'warmup':
211         return
212     if args.case != 'all':
213         run_case(args.case, args.out, warm['final_soc'], args.stop)
214         return
215     cases = designs()
216     # Q4-3 starts only after the five fresh Q3 annual results determine its configuration.
217     for batch in ([n for n in cases if not n.startswith('q4_3/')], [n for n in cases if
↳ n.startswith('q4_3/')]):
218         with ProcessPoolExecutor(max_workers=args.workers) as pool:
219             jobs = {pool.submit(run_case, n, args.out, warm['final_soc'], args.stop): n for n in
↳ batch}
220             for future in as_completed(jobs):
221                 summary = future.result()

```

```

222         print(json.dumps(dict(finished=jobs[future],cash_yuan=summary['totals']['cas_
        ↪ h_cost'])),flush=True)
223     if not batch[0].startswith('q4_3/'):
224         select_q3(args.out)
225     save(args.out/'complete.json', dict(status='COMPLETED', cases=len(cases),
226         input_and_source_hashes=fingerprints(), evaluation_days=[31,args.stop]))
227
228 if __name__ == '__main__':
229     main()

```

model/q1.py: 问题一单日状态模型

```

1 """Exact Q1 LP using storage increments, with an optional lexicographic tie rule."""
2 from __future__ import annotations
3
4 import argparse
5 import csv
6 import hashlib
7 import json
8 import math
9 import platform
10 from datetime import datetime, time, timezone
11 from pathlib import Path
12 from time import perf_counter
13
14 import numpy as np
15 import scipy
16 from openpyxl import load_workbook
17 from scipy.optimize import linprog
18 from scipy.sparse import csr_matrix, hstack, lil_matrix, vstack
19
20 ROOT = Path(__file__).resolve().parents[1]
21 DEFAULT_CASES = ["base", "roundtrip90", "left_endpoint", "interval_average",
22     "free_initial", "ideal_efficiency", "power_half", "power_double",
23     "usable_half", "usable_double"]
24
25 def digest(path):
26     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
27
28 def label(minutes):
29     return "24:00" if minutes == 1440 else f"{minutes//60:02d}:{minutes%60:02d}"
30
31 def read_inputs(config):
32     source = ROOT / config["source_workbook"]
33     if digest(source) != config["source_sha256"]:
34         raise ValueError("Raw workbook hash differs from the audited source")
35     workbook = load_workbook(source, data_only=False, read_only=True, keep_links=False)
36     rows = list(workbook[config["source_sheet"]].iter_rows(values_only=True))
37     workbook.close()
38     if rows[0] != (" 时间", " 电价", " 小区负载", " 光伏发电预测功率") or len(rows) != 145:
39         raise ValueError("Unexpected source schema or row count")
40     raw_labels = [r[0].strftime("%H:%M") if isinstance(r[0], time) else r[0] for r in
        ↪ rows[1:]]
41     if raw_labels != [label(m) for m in range(10, 1440, 10)] + ["0:00+1"]:
42         raise ValueError("Unexpected source timestamps")
43     values = np.asarray([r[1:] for r in rows[1:]], dtype=float)
44     if not np.isfinite(values).all():
45         raise ValueError("Source includes nonfinite values")
46     return values, raw_labels
47
48 def interpreted(values, mode):
49     if mode == "right_endpoint_representative":
50         return values.copy()

```

```

51     if mode == "left_endpoint_periodic":
52         return np.roll(values, 1, axis=0)
53     if mode == "trapezoid_periodic":
54         # A discrete interval-average approximation, not the exact integral of
55         # a continuously varying price multiplied by continuously varying power.
56         return (values + np.roll(values, 1, axis=0))/2
57     raise ValueError(f"Unknown interpretation: {mode}")
58
59 def case_config(base, case):
60     cfg = dict(base)
61     if case == "base":
62         pass
63     elif case == "roundtrip90":
64         cfg.update(eta_charge=math.sqrt(.9), eta_discharge=math.sqrt(.9))
65     elif case == "left_endpoint":
66         cfg["time_interpretation"] = "left_endpoint_periodic"
67     elif case == "interval_average":
68         cfg["time_interpretation"] = "trapezoid_periodic"
69     elif case == "free_initial":
70         cfg["initial_soc_kwh"] = None
71     elif case == "ideal_efficiency":
72         cfg.update(eta_charge=1., eta_discharge=1.)
73     elif case == "power_half":
74         cfg.update(max_charge_power_kw=2500., max_discharge_power_kw=2500.)
75     elif case == "power_double":
76         cfg.update(max_charge_power_kw=10000., max_discharge_power_kw=10000.)
77     elif case == "usable_half":
78         cfg.update(soc_min_kwh=3600., soc_max_kwh=8400.)
79     elif case == "usable_double":
80         cfg.update(capacity_kwh=24000., soc_min_kwh=2400., soc_max_kwh=21600.,
81                    ↪ initial_soc_kwh=12000.)
82     elif case == "upper_plus_10":
83         cfg["soc_max_kwh"] += 10.
84     elif case == "lower_minus_10":
85         cfg["soc_min_kwh"] -= 10.
86     elif case == "charge_power_plus_100":
87         cfg["max_charge_power_kw"] += 100.
88     elif case == "discharge_power_plus_100":
89         cfg["max_discharge_power_kw"] += 100.
90     else:
91         raise ValueError(f"Unknown case: {case}")
92     cfg["case"] = case
93     return cfg
94
95 def formulate(price, load, pv, config):
96     """All energies in kWh; ordering x=[g_1..g_n,S_0..S_n]."""
97     price, load, pv = [np.asarray(a, dtype=float) for a in (price, load, pv)]
98     n = len(price)
99     if not n or price.ndim != 1 or load.shape != price.shape or pv.shape != price.shape:
100         raise ValueError("Expected nonempty, equal-length one-dimensional arrays")
101     if not all(np.isfinite(a).all() for a in (price, load, pv)):
102         raise ValueError("Nonfinite model input")
103     if (price <= 0).any() or (load < 0).any() or (pv < 0).any():
104         raise ValueError("Exact epigraph model requires positive prices and nonnegative
105                    ↪ load/PV")
106     ec, ed = config["eta_charge"], config["eta_discharge"]
107     if not (0 < ec <= 1 and 0 < ed <= 1):
108         raise ValueError("Each efficiency must lie in (0,1]")
109     dt = config["step_hours"]
110     low, high, initial = config["soc_min_kwh"], config["soc_max_kwh"],
111                    ↪ config["initial_soc_kwh"]
112     if dt <= 0 or low > high or (initial is not None and not low <= initial <= high):
113         raise ValueError("Invalid time step, state bounds, or initial state")
114     cmax, dmax = config["max_charge_power_kw"]*dt, config["max_discharge_power_kw"]*dt
115     if cmax < 0 or dmax < 0:

```

```

113         raise ValueError("Power limits must be nonnegative")
114     if (config.get("grid_export_allowed", False) or config.get("battery_dump_allowed", False)
115         or not config.get("pv_curtailment_allowed", True)):
116         raise ValueError("This formulation implements free PV curtailment, no export and no
        ↪ battery dumping")
117     size = 2*n+1
118     objective = np.r_[price, np.zeros(n+1)]
119     lower = np.r_[np.zeros(n), np.full(n+1, low)]
120     upper = np.r_[np.full(n, np.inf), np.full(n+1, high)]
121     if initial is not None:
122         lower[n] = upper[n] = lower[-1] = upper[-1] = initial
123     aub = lil_matrix((4*n, size), dtype=float)
124     bub = np.zeros(4*n)
125     for t in range(n):
126         for row, slope in ((t, 1/ec), (n+t, ed)):
127             aub[row, t] = -1
128             aub[row, n+t] = -slope
129             aub[row, n+t+1] = slope
130             bub[row] = pv[t]-load[t]
131         aub[2*n+t, n+t] = -1
132         aub[2*n+t, n+t+1] = 1
133         bub[2*n+t] = ec*cmax
134         aub[3*n+t, n+t] = 1
135         aub[3*n+t, n+t+1] = -1
136         # Under exclusivity, no export and no battery dumping imply d_t<=load_t.
137         bub[3*n+t] = min(dmax, load[t])/ed
138     aeq = lil_matrix((1, size), dtype=float)
139     aeq[0, n] = -1; aeq[0, -1] = 1
140     return dict(n=n, c=objective, lower=lower, upper=upper,
141               aub=aub.tocsr(), bub=bub, aeq=aeq.tocsr(), beq=np.zeros(1))
142
143 def run_lp(model, method="highs-ds"):
144     start = perf_counter()
145     result = linprog(model["c"], A_ub=model["aub"], b_ub=model["bub"],
146                     A_eq=model["aeq"], b_eq=model["beq"],
147                     bounds=list(zip(model["lower"], model["upper"])), method=method,
148                     options={"primal_feasibility_tolerance": 1e-9,
149                             "dual_feasibility_tolerance": 1e-9,
150                             "ipm_optimality_tolerance": 1e-10})
151     seconds = perf_counter()-start
152     if not result.success:
153         raise RuntimeError(f"LP failed ({result.status}): {result.message}")
154     return result, seconds
155
156 def optimal_face(model, optimum, tolerance=1e-9):
157     """Use complementary slackness, avoiding an equality to a rounded optimum."""
158     result = dict(model)
159     active = np.flatnonzero(optimum.ineqlin.marginals < -tolerance)
160     result["aeq"] = vstack([model["aeq"], model["aub"][active]]).tocsr()
161     result["beq"] = np.r_[model["beq"], model["bub"][active]]
162     result["lower"] = model["lower"].copy()
163     result["upper"] = model["upper"].copy()
164     at_lower = np.flatnonzero(optimum.lower.marginals > tolerance)
165     at_upper = np.flatnonzero(optimum.upper.marginals < -tolerance)
166     result["upper"][at_lower] = result["lower"][at_lower]
167     result["lower"][at_upper] = result["upper"][at_upper]
168     result["face_constraints"] = dict(tight_inequality_rows=active.tolist(),
169                                     tight_lower_variables=at_lower.tolist(),
170                                     tight_upper_variables=at_upper.tolist(), dual_tolerance)
171     ↪ =tolerance)
172
173     return result
174
175 def throughput_model(primary_model, optimum, config):
176     """Stage two: minimize exact bus throughput on the primary optimal face."""
177     n = primary_model["n"]; old_size = len(primary_model["c"])

```

```

176 extra = lil_matrix((2*n, old_size+n), dtype=float)
177 for t in range(n):
178     # h_t >= max(delta/eta_c, -eta_d*delta) = c_t+d_t after recovery.
179     for row, slope in ((t, 1/config["eta_charge"]), (n+t, -config["eta_discharge"])):
180         extra[row, n+t] = -slope
181         extra[row, n+t+1] = slope
182         extra[row, old_size+t] = -1
183 face = optimal_face(primary_model, optimum)
184 aub = vstack([hstack([face["aub"], csr_matrix((4*n, n))]), extra]).tocsr()
185 aeq = hstack([face["aeq"], csr_matrix((face["aeq"].shape[0], n))]).tocsr()
186 return dict(n=n, c=np.r_[np.zeros(old_size), np.ones(n)],
187             lower=np.r_[face["lower"], np.zeros(n)],
188             upper=np.r_[face["upper"], np.full(n, np.inf)],
189             aub=aub, bub=np.r_[primary_model["bub"], np.zeros(2*n)],
190             aeq=aeq, beq=face["beq"], face_constraints=face["face_constraints"])
191
192 def recover(x, n, load, pv, config):
193     grid = x[:n].copy()
194     soc = x[n:2*n+1].copy()
195     change = np.diff(soc)
196     charge = np.maximum(change, 0)/config["eta_charge"]
197     discharge = np.maximum(-change, 0)*config["eta_discharge"]
198     spill = grid + pv + discharge - load - charge
199     # Remove only solver roundoff; do not change substantive optimized values.
200     for array in (grid, charge, discharge, spill):
201         array[np.abs(array) < 1e-9] = 0.
202     floor = np.maximum(load-pv+charge-discharge, 0.)
203     if np.max(np.abs(grid-floor)) > 1e-6:
204         raise AssertionError("The grid epigraph is not tight")
205     if np.min(spill) < -1e-6 or np.max(spill-pv) > 1e-6:
206         raise AssertionError("Recovered unused PV lies outside [0,PV]")
207     return dict(g=grid, c=charge, d=discharge, spill=spill, soc=soc, delta=change)
208
209 def solve(price, load, pv, config, select=True):
210     model = formulate(price, load, pv, config)
211     primary, t1 = run_lp(model)
212     selected, second_model, t2 = primary, None, 0.
213     if select:
214         second_model = throughput_model(model, primary, config)
215         selected, t2 = run_lp(second_model)
216     solution = recover(selected.x, len(price), load, pv, config)
217     if abs(float(np.dot(price, solution["g"]))-primary.fun) > 1e-6:
218         raise AssertionError("Selection changed the primary objective")
219     return dict(model=model, primary=primary, selected=selected,
220               ↪ secondary_model=second_model,
221               solution=solution, primary_seconds=t1, secondary_seconds=t2)
222
223 def metrics(solution, price, load, pv, config):
224     total = lambda a: math.fsum(float(v) for v in a)
225     g,c,d,s,soc = [solution[k] for k in ("g","c","d","spill","soc")]
226     no_battery = np.maximum(load-pv, 0)
227     cost, reference = total(g*price), total(no_battery*price)
228     return dict(cost_yuan=cost, grid_kwh=total(g), load_kwh=total(load),
229               ↪ pv_available_kwh=total(pv),
230               pv_curtailed_kwh=total(s), charge_bus_kwh=total(c),
231               ↪ discharge_bus_kwh=total(d),
232               throughput_kwh=total(c+d), storage_loss_kwh=total((1-config["eta_charge"])*c]
233               ↪ +(1/config["eta_discharge"]-1)*d),
234               initial_soc_kwh=float(soc[0]), terminal_soc_kwh=float(soc[-1]),
235               soc_min_kwh=float(min(soc)), soc_max_kwh=float(max(soc)),
236               max_charge_power_kw=float(max(c)/config["step_hours"]),
237               max_discharge_power_kw=float(max(d)/config["step_hours"]),
238               charge_slots=int(sum(c>1e-6)), discharge_slots=int(sum(d>1e-6)),
239               simultaneous_charge_discharge_slots=int(sum((c>1e-6)&(d>1e-6))),
240               no_battery_grid_kwh=total(no_battery), no_battery_cost_yuan=reference,

```

```

237         no_battery_curtailed_kwh=total(np.maximum(pv-load, 0)),
238         cost_saving_yuan=reference-cost,
239         cost_saving_fraction=(reference-cost)/reference if reference else None)

```

model/history.py: 历史预测与残差场景

```

1  """Past-information forecasts and whole-day historical-origin scenarios."""
2  from __future__ import annotations
3
4  from dataclasses import dataclass
5  from datetime import date, datetime, timedelta
6  from pathlib import Path
7  from typing import Literal
8
9  import numpy as np
10 from openpyxl import load_workbook
11
12 T = 144
13 DT = 1 / 6
14 START_DATE = date(2025, 1, 1)
15
16 @dataclass(frozen=True)
17 class DailyData:
18     price: np.ndarray
19     load: np.ndarray
20     pv: np.ndarray
21     dates: tuple[date, ...]
22     load_template: np.ndarray
23     pv_template: np.ndarray
24
25 def load_data(root: str | Path | None = None) -> DailyData:
26     """Load source workbooks; labels identify interval ends, position is retained."""
27     root = Path(root) if root is not None else Path(__file__).resolve().parents[1]
28     folder = root / "problem" / "attachments" / "C 题"
29     book = load_workbook(folder / "附件 1.xlsx", read_only=True, data_only=True)
30     try:
31         rows = list(book.worksheets[0].iter_rows(min_row=2, values_only=True))
32         a1 = np.asarray([r[1:4] for r in rows], dtype=float)
33     finally:
34         book.close()
35     book = load_workbook(folder / "附件 2.xlsx", read_only=True, data_only=True)
36     try:
37         payload = []
38         date_sets = []
39         for name in ("小区负载", "光伏发电实际功率"):
40             rows = list(book[name].iter_rows(min_row=2, values_only=True))
41             payload.append(np.asarray([r[1:] for r in rows], dtype=float) * DT)
42             date_sets.append(tuple(r[0].date() if isinstance(r[0], datetime) else r[0]
43                                 for r in rows))
44     finally:
45         book.close()
46     if a1.shape != (T, 3) or any(x.shape != (365, T) for x in payload):
47         raise ValueError("Official source workbook shapes differ from 144/365x144")
48     expected_dates = tuple(START_DATE + timedelta(days=j) for j in range(365))
49     if not date_sets[0] == date_sets[1] == expected_dates:
50         raise ValueError("Source dates are not aligned daily dates in 2025")
51     arrays = [a1[:, 0].copy(), *payload, a1[:, 1] * DT, a1[:, 2] * DT]
52     for x in arrays:
53         if not np.isfinite(x).all() or (x < 0).any():
54             raise ValueError("Source contains nonfinite or negative values")
55         x.setflags(write=False)
56     return DailyData(arrays[0], arrays[1], arrays[2], expected_dates, arrays[3], arrays[4])
57

```

```

58 def _lowday(index: int) -> bool:
59     return (START_DATE + timedelta(days=int(index))).weekday() in (4, 5)
60
61 def ar1_coef(residual: np.ndarray) -> float:
62     """Pooled adjacent-slot correlation, estimated solely from historical errors."""
63     x, y = residual[:, :-1].ravel(), residual[:, 1:].ravel()
64     if x.std() < 1e-9 or y.std() < 1e-9:
65         return 0.0
66     return float(np.clip(np.corrcoef(x, y)[0, 1], 0.0, 0.999))
67
68 class ForecastArchive:
69     def __init__(self, data: DailyData, cold_start: Literal["reference", "scaled"] =
    ↪ "reference"):
70         if cold_start not in ("reference", "scaled"):
71             raise ValueError("cold_start must be reference or scaled")
72         if data.load.shape != data.pv.shape or data.load.ndim != 2 or data.load.shape[1] !=
    ↪ T:
73             raise ValueError("load/PV must be aligned day-by-slot arrays")
74         self.data = data
75         self.cold_start = cold_start
76         self._points: dict[tuple[int, int], tuple[np.ndarray, np.ndarray]] = {}
77         self._residuals: dict[tuple[int, int], tuple[np.ndarray, np.ndarray, np.ndarray]] =
    ↪ {}
78         self._conditional_bases: dict[tuple[int, int, str], tuple] = {}
79
80     def _origin(self, origin: int) -> int:
81         if not isinstance(origin, (int, np.integer)) or not 0 <= origin <=
    ↪ len(self.data.load):
82             raise ValueError("origin must be a nonnegative index with its past available")
83         return int(origin)
84
85     def point(self, origin: int, target: int | None = None) -> tuple[np.ndarray, np.ndarray]:
86         """Forecast target day using only actual records strictly before origin."""
87         origin = self._origin(origin)
88         target = origin if target is None else int(target)
89         if target < 0:
90             raise ValueError("target must be nonnegative")
91         key = (origin, target)
92         if key in self._points:
93             return self._points[key]
94         same = [i for i in range(origin - 1, -1, -1) if _lowday(i) == _lowday(target)][:3]
95         if same:
96             load_hat = self.data.load[same].mean(axis=0)
97         else:
98             scale = 78400 / 111025 if _lowday(target) else 124000 / 111025
99             if self.cold_start == "reference" and origin == target == 0:
100                 scale = 1.0
101             load_hat = self.data.load_template * scale
102         if origin == 0:
103             pv_hat = self.data.pv_template.copy()
104         else:
105             count = min(14, origin)
106             weights = np.asarray([0.7 ** j for j in range(count)])
107             rows = [origin - 1 - j for j in range(count)]
108             pv_hat = (weights[:, None] * self.data.pv[rows]).sum(axis=0) / weights.sum()
109         output = (np.maximum(load_hat, 0), np.maximum(pv_hat, 0))
110         for x in output:
111             x.setflags(write=False)
112         self._points[key] = output
113         return output
114
115     def residuals(self, origin: int, K: int = 30) -> tuple[np.ndarray, np.ndarray,
    ↪ np.ndarray]:
116         """Errors from the forecasts actually available at each historical origin."""
117         origin = self._origin(origin)

```

```

118         if K < 1:
119             raise ValueError("K must be positive")
120         key = (origin, int(K))
121         if key in self._residuals:
122             return self._residuals[key]
123         indices = np.arange(max(0, origin - K), origin, dtype=int)
124         if len(indices):
125             e_load, e_pv = [], []
126             for j in indices:
127                 lh, ph = self.point(int(j))
128                 e_load.append(self.data.load[j] - lh)
129                 e_pv.append(self.data.pv[j] - ph)
130             result = (np.asarray(e_load), np.asarray(e_pv), indices)
131         else:
132             result = (np.zeros((1, T)), np.zeros((1, T)), np.array([-1], dtype=int))
133         for x in result:
134             x.setflags(write=False)
135         self._residuals[key] = result
136         return result
137
138     def _metadata(self, origin: int, indices: np.ndarray, method: str) -> dict:
139         return {'origin': origin, 'origin_date': (START_DATE +
140             ↪ timedelta(days=origin)).isoformat(), 'latest_actual_day_index': origin - 1,
141             ↪ 'historical_indices': indices.tolist(), 'method': method, 'cold_start':
142             ↪ self.cold_start}
143
144     def scenarios(self, origin: int, K: int = 30, method: str = "residual") ->
145     ↪ tuple[np.ndarray, np.ndarray, dict]:
146         origin = self._origin(origin)
147         lh, ph = self.point(origin)
148         e_load, e_pv, indices = self.residuals(origin, K)
149         metadata = self._metadata(origin, indices, method)
150         if method == "raw" and origin:
151             loads, pvs = self.data.load[indices].copy(), self.data.pv[indices].copy()
152         elif method in ("residual", "shuffled", "raw"):
153             loads, pvs = np.maximum(lh[None, :] + e_load, 0), np.maximum(ph[None, :] + e_pv,
154             ↪ 0)
155             pvs[:, ph <= 1e-9] = 0.0
156             if method == "shuffled":
157                 # Permute scenario labels independently at each slot; retain
158                 # each slot's joint L/PV marginal while destroying path links.
159                 rng = np.random.default_rng(1729 + origin)
160                 for t in range(T):
161                     order = rng.permutation(len(loads))
162                     loads[:, t] = loads[order, t]
163                     pvs[:, t] = pvs[order, t]
164                 metadata["shuffle_seed"] = 1729 + origin
165                 metadata["shuffle_scope"] = "independent scenario permutation by slot; paired
166                 ↪ L/PV"
167             else:
168                 raise ValueError("method must be residual, raw, or shuffled")
169         return loads, pvs, metadata
170
171     def _conditional_base(self, origin: int, K: int, method: str) -> tuple:
172         key = (origin, int(K), method)
173         if key in self._conditional_bases:
174             return self._conditional_bases[key]
175         lh, ph = self.point(origin)
176         e_load, e_pv, indices = self.residuals(origin, 30)
177         if method != "residual":
178             loads, pvs, _ = self.scenarios(origin, 30, method)
179             e_load, e_pv = loads - lh[None, :], pvs - ph[None, :]
180         # A shuffled-path negative control must not inherit temporal
181         # persistence from the original residual archive. Fit rho to the
182         # actual path representation being supplied to this controller.

```

```

177     rho_load, rho_pv = ar1_coef(e_load), ar1_coef(e_pv)
178     order = np.arange(len(e_load))
179     if len(order) > K:
180         order = np.random.default_rng(origin).choice(len(order), K, replace=False)
181     result = (lh, ph, e_load[order], e_pv[order], rho_load, rho_pv, indices[order])
182     self._conditional_bases[key] = result
183     return result
184
185     def conditional(self, origin: int, t: int, observed_load: np.ndarray,
186                   observed_pv: np.ndarray, observe_current: bool = True,
187                   K: int = 10, lag_anchor: str = "reference",
188                   method: str = "residual") -> tuple[np.ndarray, np.ndarray, dict]:
189         """Remaining-day conditional scenarios from an explicitly bounded prefix."""
190         origin = self._origin(origin)
191         if not isinstance(t, (int, np.integer)) or not 0 <= t < T or K < 1:
192             raise ValueError("t must be an interval index and K must be positive")
193         if lag_anchor not in ("reference", "corrected"):
194             raise ValueError("lag_anchor must be reference or corrected")
195         allowed = int(t) + int(bool(observe_current))
196         observed_load = np.asarray(observed_load, dtype=float)
197         observed_pv = np.asarray(observed_pv, dtype=float)
198         if observed_load.shape != (allowed,) or observed_pv.shape != (allowed,):
199             raise ValueError(f"Only an observation prefix of exactly {allowed} slots is
200                               ↳ allowed")
201         if not np.isfinite(observed_load).all() or not np.isfinite(observed_pv).all():
202             raise ValueError("Observation prefix contains nonfinite values")
203         lh, ph, e_load, e_pv, rho_load, rho_pv, indices = self._conditional_base(origin, K,
204                                       ↳ method)
205         metadata = self._metadata(origin, indices, method)
206         metadata.update(t=int(t), observe_current=bool(observe_current),
207                       ↳ last_observed_slot=allowed - 1, lag_anchor=lag_anchor, rho_load=rho_load,
208                       ↳ rho_pv=rho_pv, scenario_seed=origin)
209         if method == "shuffled":
210             metadata["shuffle_seed"] = 1729 + origin
211         if not observe_current and t == 0 and lag_anchor == "corrected":
212             loads = np.maximum(lh[None, :] + e_load, 0)
213             pvs = np.maximum(ph[None, :] + e_pv, 0)
214             metadata["residual_anchor_slot"] = None
215         else:
216             anchor_observed = t if observe_current else t - 1
217             err_load = observed_load[anchor_observed] - lh[anchor_observed] if allowed else
218             ↳ 0.0
219             err_pv = observed_pv[anchor_observed] - ph[anchor_observed] if allowed else 0.0
220             steps = np.arange(T - t) + (0 if observe_current else 1)
221             factor_load, factor_pv = rho_load ** steps, rho_pv ** steps
222             anchor_residual = t if observe_current or lag_anchor == "reference" else t - 1
223             loads = np.maximum(lh[t:][None, :] + factor_load[None, :] * err_load
224                               ↳ + e_load[:, t:] - factor_load[None, :] * e_load[:,
225                               ↳ ↳ [anchor_residual]], 0)
226             pvs = np.maximum(ph[t:][None, :] + factor_pv[None, :] * err_pv
227                               ↳ + e_pv[:, t:] - factor_pv[None, :] * e_pv[:, [anchor_residual]],
228                               ↳ ↳ 0)
229             metadata["residual_anchor_slot"] = int(anchor_residual)
230         pvs[:, ph[t:] <= 1e-9] = 0.0
231         if observe_current:
232             loads[:, 0] = observed_load[-1]
233             pvs[:, 0] = observed_pv[-1]
234         return loads, pvs, metadata

```

model/publication.py: 官方预报与发布时间

```

1 """Official PV forecast paths and causally available paired historical errors."""
2 from __future__ import annotations

```

```

3
4 from dataclasses import dataclass
5 from datetime import date, datetime, time, timedelta
6 import hashlib
7 from pathlib import Path
8
9 import numpy as np
10 from openpyxl import load_workbook
11
12 from model.history import DT, START_DATE, T, DailyData, ForecastArchive
13
14 ISSUE_HOURS = (0, 6, 12, 18)
15 _EPOCH = datetime.combine(START_DATE, time.min)
16
17 def _readonly(array: np.ndarray) -> np.ndarray:
18     array.setflags(write=False)
19     return array
20
21 def _time_at_boundary(boundary: int) -> str:
22     return (_EPOCH + timedelta(minutes=10 * int(boundary))).isoformat(timespec="minutes")
23
24 def _integer(value: int, name: str) -> int:
25     if isinstance(value, (bool, np.bool_)) or not isinstance(value, (int, np.integer)):
26         raise ValueError(f"{name} must be an integer")
27     return int(value)
28
29 @dataclass(frozen=True)
30 class OfficialForecastTable:
31     """Read-only official table; the leading dimensions are day and issue hour."""
32
33     hourly_kw: np.ndarray
34     dates: tuple[date, ...]
35     source_hashes: dict[str, str]
36     sheet_name: str = "Sheet1"
37
38 def load_official_forecasts(root: str | Path | None = None) -> OfficialForecastTable:
39     """Read attachment 3 without modifying it, validating every date/publication."""
40     root = Path(root) if root is not None else Path(__file__).resolve().parents[1]
41     folder = root / "problem" / "attachments" / "C 题"
42     workbook = load_workbook(folder / "附件 3.xlsx", read_only=True, data_only=True)
43     try:
44         if len(workbook.worksheets) != 1:
45             raise ValueError("Attachment 3 must contain one forecast sheet")
46         sheet = workbook.worksheets[0]
47         rows = list(sheet.iter_rows(values_only=True))
48         if len(rows) != 1461 or len(rows[0]) != 26:
49             raise ValueError("Attachment 3 differs from 365 days x 4 issues x 24 leads")
50         if tuple(rows[0][2:]) != tuple(f"预报 {h} 小时" for h in range(1, 25)):
51             raise ValueError("Attachment 3 lead-hour column labels are not 1 through 24")
52         dates = tuple(START_DATE + timedelta(days=d) for d in range(365))
53         hourly = np.empty((365, 4, 24), dtype=float)
54         for d, expected_date in enumerate(dates):
55             for issue_index, issue_hour in enumerate(ISSUE_HOURS):
56                 row = rows[1 + 4 * d + issue_index]
57                 raw_date = row[0]
58                 if isinstance(raw_date, datetime):
59                     row_date = raw_date.date()
60                 elif isinstance(raw_date, date):
61                     row_date = raw_date
62                 elif raw_date in (None, ""):
63                     row_date = None
64                 else:
65                     row_date = datetime.strptime(str(raw_date).strip(), "%Y-%m-%d").date()
66                 if (issue_index == 0 and row_date != expected_date) or (
67                     issue_index > 0 and row_date not in (None, expected_date)

```

```

68         ):
69             raise ValueError(f"Unexpected date in attachment 3 row {2 + 4*d +
        ↪ issue_index}")
70         raw_hour = row[1]
71         if isinstance(raw_hour, (datetime, time)):
72             parsed_hour = raw_hour.hour if raw_hour.minute == raw_hour.second == 0
        ↪ else -1
73         else:
74             parts = str(raw_hour).strip().split(":")
75             parsed_hour = int(parts[0]) if len(parts) == 2 and int(parts[1]) == 0
        ↪ else -1
76         if parsed_hour != issue_hour:
77             raise ValueError("Attachment 3 publication order differs from
        ↪ 00/06/12/18")
78         hourly[d, issue_index] = np.asarray(row[2:], dtype=float)
79         if not np.isfinite(hourly).all() or (hourly < 0).any():
80             raise ValueError("Official forecast powers must be finite and nonnegative")
81         sheet_name = sheet.title
82     finally:
83         workbook.close()
84     hashes = {
85         f"problem/attachments/C 题/附件 {n}.xlsx": hashlib.sha256(
86             (folder / f"附件 {n}.xlsx").read_bytes()
87         ).hexdigest()
88         for n in (1, 2, 3)
89     }
90     return OfficialForecastTable(_readonly(hourly), dates, hashes, sheet_name)
91
92 class OfficialForecastArchive:
93     """Official PV centers plus Q2 load centers at the same historical origin."""
94
95     def __init__(self, data: DailyData, root: str | Path | None = None,
96                  load_archive: ForecastArchive | None = None,
97                  official: OfficialForecastTable | None = None):
98         self.data = data
99         self.official = official if official is not None else load_official_forecasts(root)
100         self.load_archive = load_archive if load_archive is not None else
        ↪ ForecastArchive(data)
101         if data.load.shape != data.pv.shape or data.load.ndim != 2 or data.load.shape[1] !=
        ↪ T:
102             raise ValueError("Actual load/PV must be aligned day-by-slot arrays")
103         if self.official.hourly_kw.shape != (len(self.official.dates), 4, 24):
104             raise ValueError("Official forecasts must have shape (days, 4, 24)")
105         expected_dates = tuple(START_DATE + timedelta(days=d)
106                                for d in range(len(self.official.dates)))
107         if self.official.dates != expected_dates:
108             raise ValueError("Official forecast dates must be consecutive from START_DATE")
109         if not np.isfinite(self.official.hourly_kw).all() or (self.official.hourly_kw <
        ↪ 0).any():
110             raise ValueError("Official forecast powers must be finite and nonnegative")
111         self._points: dict[tuple[int, int], tuple[np.ndarray, np.ndarray, dict]] = {}
112         self._errors: dict[tuple[int, int, int], tuple[np.ndarray, np.ndarray, np.ndarray,
        ↪ dict]] = {}
113
114     def _origin(self, day: int, issue_hour: int,
115                as_of_abs_slot: int | None = None) -> tuple[int, int, int]:
116         day, issue_hour = _integer(day, "day"), _integer(issue_hour, "issue_hour")
117         if not 0 <= day < len(self.official.dates) or issue_hour not in ISSUE_HOURS:
118             raise ValueError("Need an available official forecast day and issue hour
        ↪ 0/6/12/18")
119         boundary = day * T + issue_hour * 6
120         as_of = boundary if as_of_abs_slot is None else _integer(as_of_abs_slot,
        ↪ "as_of_abs_slot")
121         if as_of < boundary:

```

```

122         raise ValueError("Requested forecast has not been published at this information
123         ↪ boundary")
124     if boundary > self.data.load.size:
125         raise ValueError("The completed-interval anchor is absent from the supplied
126         ↪ actual history")
127     # A later as_of permits retrieval of an old publication, but never moves
128     # that publication's actual-data fitting or residual-availability cutoff.
129     return day, issue_hour, boundary
130
131 def _metadata(self, day: int, issue_hour: int, boundary: int) -> dict:
132     source_row = 2 + day * 4 + ISSUE_HOURS.index(issue_hour)
133     return {'origin_day': day, 'issue_hour': issue_hour, 'published_at':
134     ↪ _time_at_boundary(boundary), 'forecast_origin_abs_slot': boundary,
135     ↪ 'latest_target_actual_abs_slot': None, 'latest_actual_abs_slot_used': boundary -
136     ↪ 1 if boundary else None, 'historical_days': []}
137
138 def residuals(self, day: int, issue_hour: int = 0, K: int = 30, *,
139     as_of_abs_slot: int | None = None) -> tuple[np.ndarray, np.ndarray,
140     ↪ np.ndarray, dict]:
141     """Latest at most 30 same-publication complete 24-hour forecast errors."""
142     day, issue_hour, boundary = self._origin(day, issue_hour, as_of_abs_slot)
143     K = _integer(K, "K")
144     if not 1 <= K <= 30:
145         raise ValueError("K must be between 1 and the fixed 30-path history limit")
146     key = day, issue_hour, K
147     if key not in self._errors:
148         offset = issue_hour * 6
149         indices = np.arange(max(0, day - K), day, dtype=int)
150         load_errors, pv_errors = [], []
151         release_boundaries, completed_boundaries = [], []
152         for j in indices:
153             start = int(j) * T + offset
154             end = start + T
155             if end > boundary or end > self.data.load.size:
156                 raise ValueError("A historical 24-hour target path is not fully
157                 ↪ observable")
158             load_hat, pv_hat, _ = self.point(int(j), issue_hour)
159             load_errors.append(self.data.load.reshape(-1)[start:end] - load_hat)
160             pv_errors.append(self.data.pv.reshape(-1)[start:end] - pv_hat)
161             release_boundaries.append(start)
162             completed_boundaries.append(end)
163             e_load = np.asarray(load_errors, dtype=float).reshape((-1, T))
164             e_pv = np.asarray(pv_errors, dtype=float).reshape((-1, T))
165             meta = self._metadata(day, issue_hour, boundary)
166             meta.update(historical_days=indices.tolist(),
167                 ↪ latest_target_actual_abs_slot=max(completed_boundaries) - 1 if len(indices)
168                 ↪ else None, sample_count=len(indices))
169             self._errors[key] = _readonly(e_load), _readonly(e_pv), _readonly(indices), meta
170     e_load, e_pv, indices, meta = self._errors[key]
171     return e_load, e_pv, indices, dict(meta)
172
173 def scenarios(self, day: int, issue_hour: int = 0, K: int = 30, *,
174     as_of_abs_slot: int | None = None) -> tuple[np.ndarray, np.ndarray, dict]:
175     """Add paired historical errors to the current center and clip at zero."""
176     load_hat, pv_hat, point_meta = self.point(day, issue_hour,
177     ↪ as_of_abs_slot=as_of_abs_slot)
178     e_load, e_pv, _, residual_meta = self.residuals(day, issue_hour, K,
179     ↪ as_of_abs_slot=as_of_abs_slot)
180     meta = {**point_meta, **residual_meta, "scenario_method": "paired additive residuals
181     ↪ with nonnegative clipping"}
182     return np.maximum(load_hat[None, :] + e_load, 0), np.maximum(pv_hat[None, :] + e_pv,
183     ↪ 0), meta
184
185 def z2_signal_pairs(self, day: int, K: int = 30, *,
186     as_of_abs_slot: int | None = None) -> dict:

```

```

174     """Past 00:00 errors paired with that same day's later 06:00 signal."""
175     day, _, boundary = self._origin(day, 0, as_of_abs_slot)
176     e_load, e_pv, indices, base_meta = self.residuals(day, 0, K,
177     ↪ as_of_abs_slot=as_of_abs_slot)
178     signals, pv00_remaining, pv06_remaining = [], [], []
179     for j in indices:
180         hist_signal_boundary = int(j) * T + 36
181         if hist_signal_boundary > boundary or (int(j) + 1) * T > boundary:
182             raise ValueError("Z2 signal and paired target residual must both already be
183             ↪ historical")
184         forecast00 = self.point(int(j), 0)[1]
185         forecast06 = self.point(int(j), 6)[1]
186         earlier, later = float(forecast00[36:].sum()), float(forecast06[:108].sum())
187         pv00_remaining.append(earlier)
188         pv06_remaining.append(later)
189         signals.append(later - earlier)
190     signal_array = np.asarray(signals, dtype=float)
191     thresholds = np.quantile(signal_array, [1/3, 2/3]) if len(signals) else np.empty(0)
192     groups = np.searchsorted(thresholds, signal_array, side="right").astype(int)
193     counts = [int(np.sum(groups == g)) for g in range(3)]
194     meta = dict(base_meta)
195     pass
196     return {
197         "load_residuals": e_load, "pv_residuals": e_pv,
198         "historical_days": indices, "signals": _readonly(signal_array),
199         "thresholds": _readonly(np.asarray(thresholds)), "groups": _readonly(groups),
200         "pv00_remaining_kwh": _readonly(np.asarray(pv00_remaining)),
201         "pv06_remaining_kwh": _readonly(np.asarray(pv06_remaining)),
202         "metadata": meta,
203     }

```

model/fusion.py: 完整窗口融合基线

```

1  """Causal historical/official PV fusion with January calibration only."""
2  from __future__ import annotations
3
4  from copy import deepcopy
5  from datetime import timedelta
6  import hashlib
7  import math
8  from pathlib import Path
9
10 import numpy as np
11
12 from model.history import DT, START_DATE, T, DailyData, ForecastArchive
13 from model.publication import ISSUE_HOURS, OfficialForecastArchive, OfficialForecastTable,
14     ↪ _integer, _readonly, _time_at_boundary
15
16 JANUARY_DAYS = 31
17 FREEZE_BOUNDARY = JANUARY_DAYS * T
18 COLD_START_WEIGHT = 0.5
19
20 def fit_convex_l1_weight(history: np.ndarray, official: np.ndarray,
21     ↪ actual: np.ndarray) -> dict:
22     """Exact piecewise-linear minimizer of  $\sum |(1-w)h + w \cdot o - y|$ ,  $0 \leq w \leq 1$ ."""
23     history, official, actual = (np.asarray(x, dtype=float) for x in (history, official,
24     ↪ actual))
25     if history.shape != official.shape or history.shape != actual.shape or not history.size:
26         raise ValueError("L1 calibration needs nonempty equally shaped arrays")
27     if not all(np.isfinite(x).all() for x in (history, official, actual)):
28         raise ValueError("L1 calibration inputs must be finite")
29     history, official, actual = history.ravel(), official.ravel(), actual.ravel()
30     delta = official - history

```

```

29     active = delta != 0
30     if not np.isfinite(delta).all():
31         raise ValueError("Calibration differences must be finite")
32     if active.any():
33         with np.errstate(over="ignore", divide="ignore"):
34             points = np.clip((actual[active] - history[active]) / delta[active], 0.0, 1.0)
35             masses = np.abs(delta[active])
36             order = np.argsort(points, kind="stable")
37             points, masses = points[order], masses[order]
38             ratios = [float(m).as_integer_ratio() for m in masses]
39             denominator = max(pair[1] for pair in ratios)
40             integer_masses = [num * (denominator // den) for num, den in ratios]
41             total, cumulative = sum(integer_masses), 0
42             for index, mass in enumerate(integer_masses):
43                 cumulative += mass
44                 if 2 * cumulative >= total:
45                     low = float(points[index])
46                     high = float(points[index + 1]) if 2*cumulative == total and index+1 <
47                         ↪ len(points) else low
48                     break
49             distinct = int(np.unique(points).size)
50     else:
51         low, high, distinct = 0.0, 1.0, 0
52     weight = (low + high) / 2.0
53     errors = (1.0-weight)*history + weight*official - actual
54     sae = math.fsum(np.abs(errors).tolist())
55     return {
56         "official_weight": weight, "history_weight": 1.0-weight,
57         "optimal_weight_interval": [low, high], "nonunique_optimum": high > low,
58         "selection_rule": "midpoint of the complete optimal interval in [0,1]",
59         "solver": "weighted median of clipped L1 breakpoints; exact binary-float mass sums",
60         "target_count": int(actual.size), "active_breakpoint_count": int(active.sum()),
61         "distinct_clipped_breakpoints": distinct,
62         "zero_difference_target_count": int((~active).sum()),
63         "minimum_sae_kwh": sae, "minimum_mae_kwh_per_interval": sae / actual.size,
64     }
65
66 def _array_hash(*arrays: np.ndarray) -> str:
67     digest = hashlib.sha256()
68     for array in arrays:
69         array = np.ascontiguousarray(array, dtype="<f8")
70         digest.update(str(array.shape).encode("ascii"))
71         digest.update(array.tobytes())
72     return digest.hexdigest()
73
74 class FusedForecastArchive(OfficialForecastArchive):
75     """V1-compatible archive with causal convex PV fusion and fixed load model."""
76
77     def __init__(self, data: DailyData, root: str | Path | None = None,
78                 load_archive: ForecastArchive | None = None,
79                 official: OfficialForecastTable | None = None):
80         super().__init__(data, root=root, load_archive=load_archive, official=official)
81         self.root = Path(root) if root is not None else Path(__file__).resolve().parents[1]
82         self._components: dict[tuple[int, int], tuple[np.ndarray, np.ndarray, np.ndarray,
83             ↪ dict]] = {}
84         self._weight_fits: dict[tuple[int, int], dict] = {}
85
86     def components(self, day: int, issue_hour: int = 0, *,
87                 as_of_abs_slot: int | None = None) -> tuple[np.ndarray, np.ndarray,
88             ↪ np.ndarray, dict]:
89         """Load, historical PV and official PV from this publication's history."""
90         day, issue_hour, boundary = self._origin(day, issue_hour, as_of_abs_slot)
91         key = day, issue_hour
92         if key not in self._components:
93             offset = issue_hour * 6

```

```

91     load_today, pv_today = self.load_archive.point(day, day)
92     if offset:
93         load_next, pv_next = self.load_archive.point(day, day + 1)
94         load_hat = np.r_[load_today[offset:], load_next[:offset]]
95         historical = np.r_[pv_today[offset:], pv_next[:offset]]
96     else:
97         load_hat, historical = load_today.copy(), pv_today.copy()
98     if boundary:
99         anchor = float(self.data.pv.reshape(-1)[boundary - 1] / DT)
100        source = "attachment2 actual PV at last completed interval endpoint"
101    else:
102        anchor = float(self.data.pv_template[-1] / DT)
103        source = "Assumed cold start: attachment1 24:00 PV template endpoint"
104    powers = self.official.hourly_kw[day, ISSUE_HOURS.index(issue_hour)]
105    official_pv = np.interp(np.arange(1, T + 1, dtype=float) * DT,
106                           np.arange(25, dtype=float), np.r_[anchor, powers]) * DT
107    if not all(np.isfinite(x).all() for x in (load_hat, historical, official_pv)):
108        raise ValueError("A forecast consumed nonfinite available data")
109    meta = self._metadata(day, issue_hour, boundary)
110    pass
111    self._components[key] = (_readonly(load_hat), _readonly(historical),
112                           _readonly(official_pv), meta)
113    loads, historical, official_pv, meta = self._components[key]
114    return loads, historical, official_pv, dict(meta)
115
116 def _fit_at(self, cutoff: int, issue_hour: int) -> dict:
117     """Fit only January paths fully observable strictly before cutoff."""
118     cutoff = min(int(cutoff), FREEZE_BOUNDARY)
119     key = cutoff, issue_hour
120     if key not in self._weight_fits:
121         offset = issue_hour * 6
122         count = max(0, min(JANUARY_DAYS, (cutoff - offset - T) // T + 1))
123         days = list(range(count))
124         ends = [(j+1)*T+offset for j in days]
125         if ends and max(ends) > min(cutoff, self.data.pv.size):
126             raise ValueError("The requested calibration needs unavailable target
127                               ↪ actuals")
128         if count:
129             centers = [self.components(j, issue_hour) for j in days]
130             history = np.vstack([value[1] for value in centers])
131             official = np.vstack([value[2] for value in centers])
132             actual = np.vstack([self.data.pv.reshape(-1)[j*T+offset:(j+1)*T+offset] for j
133                               ↪ in days])
134             record = fit_convex_l1_weight(history, official, actual)
135             record["classification"] = "Derived from completed January historical-origin
136                               ↪ forecasts"
137         else:
138             history = official = actual = np.empty((0, T))
139             record = {
140                 "official_weight": COLD_START_WEIGHT, "history_weight":
141                 ↪ 1-COLD_START_WEIGHT,
142                 "optimal_weight_interval": [0.0, 1.0], "nonunique_optimum": True,
143                 "selection_rule": "Assumed cold-start weight 0.5; no realized calibration
144                 ↪ target",
145                 "solver": "none: no complete January calibration paths",
146                 "target_count": 0, "active_breakpoint_count": 0,
147                 "distinct_clipped_breakpoints": 0, "zero_difference_target_count": 0,
148                 "minimum_sae_kwh": None, "minimum_mae_kwh_per_interval": None,
149                 "classification": "Assumed cold start",
150             }

```

```

146         record.update(issue_hour=issue_hour, fit_cutoff_abs_slot_exclusive=cutoff,
↪      sample_count=count, historical_days=days,
↪      latest_target_actual_abs_slot=max(ends) - 1 if ends else None,
↪      calibration_arrays_sha256=_array_hash(history, official, actual),
↪      objective='sum of absolute errors over all 144 ten-minute targets of every
↪      eligible path')
147         self._weight_fits[key] = record
148     return deepcopy(self._weight_fits[key])
149
150     def weight_at(self, day: int, issue_hour: int = 0, *, as_of_abs_slot: int | None = None)
↪      -> dict:
151         """The historical weight actually available at this publication time."""
152         day, issue_hour, boundary = self._origin(day, issue_hour, as_of_abs_slot)
153         result = self._fit_at(min(boundary, FREEZE_BOUNDARY), issue_hour)
154         result.update(origin_day=day, forecast_origin_abs_slot=boundary,
↪      regime='january_walk_forward' if boundary < FREEZE_BOUNDARY else
↪      'february_frozen')
155         return result
156
157     def point(self, day: int, issue_hour: int = 0, *,
158               as_of_abs_slot: int | None = None) -> tuple[np.ndarray, np.ndarray, dict]:
159         day, issue_hour, _ = self._origin(day, issue_hour, as_of_abs_slot)
160         key = day, issue_hour
161         if key not in self._points:
162             loads, historical, official, meta = self.components(day, issue_hour)
163             fit = self.weight_at(day, issue_hour)
164             weight = fit["official_weight"]
165             fused = (1-weight)*historical + weight*official
166             meta.update(official_weight=weight, history_weight=1 - weight)
167             self._points[key] = loads, _readonly(fused), meta
168         loads, fused, meta = self._points[key]
169         return loads, fused, dict(meta)
170

```

model/remaining.py: 当天剩余窗口校准

```

1  """Remaining-day calibration with the unchanged complete 24h forecast API."""
2  from __future__ import annotations
3
4  from copy import deepcopy
5  from datetime import timedelta
6  import hashlib
7
8  import numpy as np
9
10 from model.history import START_DATE, T
11 from model.publication import ISSUE_HOURS, _time_at_boundary
12 from model.fusion import COLD_START_WEIGHT, FREEZE_BOUNDARY, JANUARY_DAYS,
↪      FusedForecastArchive as _FullWindowFusedArchive, _array_hash, fit_convex_l1_weight
13
14 FIT_WINDOW = "remaining_same_day"
15 REVISION_STATUS = "post_annual_diagnostics_revision_with_january_only_numeric_calibration"
16
17 class RemainingWindowFusedForecastArchive(_FullWindowFusedArchive):
18     """V2-compatible centers/residuals with remaining-day January L1 weights."""
19
20     def _fit_at(self, cutoff: int, issue_hour: int) -> dict:
21         """Use January same-day windows with end=(j+1)*144 <= cutoff."""
22         cutoff = min(int(cutoff), FREEZE_BOUNDARY)
23         key = cutoff, issue_hour
24         if key not in self._weight_fits:
25             offset = issue_hour*6
26             length = T-offset

```

```

27     count = max(0, min(JANUARY_DAYS, cutoff//T))
28     days = list(range(count))
29     starts = [j*T+offset for j in days]
30     ends = [(j+1)*T for j in days]
31     if ends and max(ends) > min(cutoff, self.data.pv.size):
32         raise ValueError("The requested calibration needs unavailable remaining-day
33             ↪ actuals")
34     if count:
35         components = [self.components(j, issue_hour) for j in days]
36         history = np.vstack([value[1][:length] for value in components])
37         official = np.vstack([value[2][:length] for value in components])
38         actual = np.vstack([self.data.pv.reshape(-1)[start:end] for start, end in
39             ↪ zip(starts, ends)])
40         record = fit_convex_l1_weight(history, official, actual)
41         record["classification"] = "Derived from completed January historical-origin
42             ↪ remaining-day forecasts"
43     else:
44         history = official = actual = np.empty((0, length))
45         record = {
46             "official_weight": COLD_START_WEIGHT, "history_weight":
47             ↪ 1-COLD_START_WEIGHT,
48             "optimal_weight_interval": [0.0, 1.0], "nonunique_optimum": True,
49             "selection_rule": "Assumed cold-start weight 0.5; no realized calibration
50             ↪ target",
51             "solver": "none: no complete January remaining-day calibration windows",
52             "target_count": 0, "active_breakpoint_count": 0,
53             "distinct_clipped_breakpoints": 0, "zero_difference_target_count": 0,
54             "minimum_sae_kwh": None, "minimum_mae_kwh_per_interval": None,
55             "classification": "Assumed cold start",
56         }
57     record.update(issue_hour=issue_hour, fit_cutoff_abs_slot_exclusive=cutoff,
58         ↪ sample_count=count, historical_days=days,
59         ↪ latest_target_actual_abs_slot=max(ends) - 1 if ends else None,
60         ↪ calibration_arrays_sha256=array_hash(history, official, actual),
61         ↪ objective=f'sum of absolute errors over all {length} remaining-day ten-minute
62         ↪ targets of every eligible publication')
63     self._weight_fits[key] = record
64     return deepcopy(self._weight_fits[key])
65
66 # Compatible imports for the existing runtime and the new runner.
67 FusedForecastArchive = RemainingWindowFusedForecastArchive

```

model/planning.py: 储能状态规划

```

1  """Sparse Q2 planning models, explicit battery formulation and state LP."""
2  from __future__ import annotations
3
4  from dataclasses import dataclass
5  from functools import lru_cache
6  from time import perf_counter
7
8  import numpy as np
9  from scipy import sparse
10 from scipy.optimize import Bounds, LinearConstraint, linprog, milp
11
12 @dataclass(frozen=True)
13 class Physical:
14     eta_c: float = 0.9
15     eta_d: float = 0.9
16     s_min: float = 1200.0
17     s_max: float = 10800.0
18     power_energy: float = 5000.0/6.0
19     emergency_multiple: float = 5.0

```

```

20
21 @dataclass
22 class Block:
23     load: np.ndarray
24     pv: np.ndarray
25     price: np.ndarray
26
27     def __post_init__(self):
28         self.load = np.atleast_2d(np.asarray(self.load, float))
29         self.pv = np.atleast_2d(np.asarray(self.pv, float))
30         self.price = np.asarray(self.price, float)
31         if self.load.shape != self.pv.shape or self.price.shape != (self.load.shape[1],):
32             raise ValueError('Inconsistent scenario/price shapes')
33         if not np.isfinite(self.load).all() or not np.isfinite(self.pv).all():
34             raise ValueError('Nonfinite scenario')
35         if np.min(self.load) < 0 or np.min(self.pv) < 0 or np.min(self.price) <= 0:
36             raise ValueError('Nonphysical input or nonpositive price')
37
38 @dataclass
39 class Plan:
40     q: np.ndarray
41     c: np.ndarray
42     b: np.ndarray
43     soc: np.ndarray
44     objective: float
45     expected_cash: float
46     lam: float
47     end_target: float
48     seconds: float
49     diagnostics: dict
50     result: object = None
51     formulation: dict | None = None
52
53 @lru_cache(maxsize=450)
54 def _layout(shapes: tuple, eta_c: float, eta_d: float, has_value: bool):
55     """Matrix topology depends on shapes only; scenario values go in RHS."""
56     offsets=[]; cursor=0
57     for k,n in shapes:
58         offsets.append(dict(q=cursor,c=cursor+n,b=cursor+2*n,s=cursor+3*n,
59                             e=cursor+4*n,r=cursor+4*n+k*n))
60         cursor += 4*n+2*k*n
61     v_idx=cursor if has_value else None
62     nv=cursor+int(has_value)
63     rowparts=[]; colparts=[]; valparts=[]; rhs_slices=[]; link_rows=[]
64     row=0; previous=None
65     for (k,n),o in zip(shapes, offsets):
66         t=np.arange(n)
67         link_rows.append(row)
68         #  $S_t - \eta_c c_t + b_t / \eta_d - S_{t-1} = 0$ .
69         rowparts.extend([row+t,row+t,row+t])
70         colparts.extend([o['s']+t,o['c']+t,o['b']+t])
71         valparts.extend([np.ones(n),np.full(n,-eta_c),np.full(n,1/eta_d)])
72         if n>1:
73             rowparts.append(row+t[1:]); colparts.append(o['s']+t[:-1]);
74             ↪ valparts.append(-np.ones(n-1))
75         if previous is not None:
76             rowparts.append(np.array([row])); colparts.append(np.array([previous])); valparts.
77             ↪ append(np.array([-1.]))
78         row+=n
79         #  $q+b+e-c-r = \text{load-pv}$ , scenario-major indexing.
80         times=np.tile(t,k); rids=row+np.arange(k*n)
81         rowparts.extend([rids]*5)
82         colparts.extend([o['q']+times,o['b']+times,o['e']+np.arange(k*n),
83                             o['c']+times,o['r']+np.arange(k*n)])
84         valparts.extend([np.ones(k*n),np.ones(k*n),np.ones(k*n),-np.ones(k*n),-np.ones(k*n)])

```

```

83     rhs_slices.append(slice(row,row+k*n)); row+=k*n
84     previous=o['s']+n-1
85     mat=sparse.coo_matrix((np.concatenate(valparts),(np.concatenate(rowparts),np.concatenate(
    ↪ (colparts))),shape=(row,nv)).tocsr())
86     return offsets,nv,mat,tuple(rhs_slices),tuple(link_rows),previous,v_idx
87
88 def formulate(blocks: list[Block], s0: float, *, terminal='free', lam=0.,
89             target=None, fixed_q=None, physical=Physical(), strict_delivery=False,
90             no_battery_dump=False, first_charge_cap=None, first_discharge_cap=None,
91             explicit_mutual=False, enforce_m0=False):
92     """Build explicit LP, or a small-case MILP when mutual/M0 is requested."""
93     if terminal not in ('free','cyclic','equal','value','floor'):
94         raise ValueError(terminal)
95     if not physical.s_min-1e-7 <= s0 <= physical.s_max+1e-7:
96         raise ValueError('Initial storage outside bounds')
97     shapes=tuple(b.load.shape for b in blocks)
98     offsets,nv,eq,ranges,links,last,vid=_layout(shapes,physical.eta_c,physical.eta_d,terminal
    ↪ l='value')
99     obj=np.zeros(nv);lo=np.zeros(nv);hi=np.full(nv,np.inf);rhs=np.zeros(eq.shape[0]);rhs[0]=
    ↪ s0
100    for block,(k,n),o,rs in zip(blocks,shapes,offsets,ranges):
101        obj[o['q']:o['q']+n]=block.price
102        obj[o['e']:o['e']+k*n]=np.tile(physical.emergency_multiple*block.price/k,k)
103        hi[o['c']:o['c']+n]=physical.power_energy
104        hi[o['b']:o['b']+n]=physical.power_energy
105        if no_battery_dump:
106            hi[o['b']:o['b']+n]=np.minimum(physical.power_energy,block.load.min(axis=0))
107        lo[o['s']:o['s']+n]=physical.s_min;hi[o['s']:o['s']+n]=physical.s_max
108        if strict_delivery: hi[o['r']:o['r']+k*n]=block.pv.ravel()
109        rhs[rs]=(block.load-block.pv).ravel()
110    first=offsets[0];n0=shapes[0][1]
111    if fixed_q is not None:
112        q=np.asarray(fixed_q,float)
113        if q.shape!=(n0,) or np.min(q)<-1e-7: raise ValueError('Invalid fixed q')
114        lo[first['q']:first['q']+n0]=q;hi[first['q']:first['q']+n0]=q
115    if first_charge_cap is not None:
116        hi[first['c']]=min(physical.power_energy,max(0.,float(first_charge_cap)))
117    if first_discharge_cap is not None:
118        hi[first['b']]=min(physical.power_energy,max(0.,float(first_discharge_cap)))
119    ur=[];uc=[];uv=[];ub=[]
120    if terminal in ('cyclic','equal','floor'):
121        level=s0 if target is None else float(target)
122        if terminal=='equal': lo[last]=hi[last]=level
123        else: ur=[0];uc=[last];uv=[-1.];ub=[-level]
124    elif terminal=='value':
125        if target is None: target=physical.s_max
126        if lam<0: raise ValueError('Negative continuation credit')
127        obj[vid]=-lam;hi[vid]=float(target)
128        ur=[0,0];uc=[vid,last];uv=[1.,-1.];ub=[0.]
129    integrality=None
130    if explicit_mutual or enforce_m0:
131        # A mode z_t=1 allows charging and forces discharge/emergency to zero.
132        # M for each e uses a known per-scenario physical bound; no q cap.
133        n_modes=sum(n for _,n in shapes);base_nv=nv;nv+=n_modes
134        obj=np.r_[obj,np.zeros(n_modes)];lo=np.r_[lo,np.zeros(n_modes)];hi=np.r_[hi,np.ones(
    ↪ n_modes)]
135        integrality=np.r_[np.zeros(base_nv,int),np.ones(n_modes,int)]
136        eq=sparse.hstack([eq,sparse.csr_matrix((eq.shape[0],n_modes))],format='csr')
137        iz=base_nv
138        for block,(k,n),o in zip(blocks,shapes,offsets):
139            for t in range(n):
140                r=len(ub);ur.extend([r,r]);uc.extend([o['c']+t,iz+t]);uv.extend([1.,-physical
    ↪ 1.power_energy]);ub.append(0.)
141                r=len(ub);ur.extend([r,r]);uc.extend([o['b']+t,iz+t]);uv.extend([1.,physical
    ↪ .power_energy]);ub.append(physical.power_energy)

```

```

142         if enforce_m0:
143             # z=1 (charging):  $q-c \geq \max_s(\text{load}_s - \text{pv}_s)$ .
144             # z=0:  $c=0$  and the inequality reduces to  $q \geq 0$ .
145             # With positive emergency cost, this forces every  $e_s=0$ 
146             # whenever  $c>0$ , without a large arbitrary energy bound.
147             worst_net=float(np.max(block.load[:,t]-block.pv[:,t]))
148             r=len(ub);ur.extend([r,r,r]);uc.extend([o['c']+t,o['q']+t,iz+t]);uv.extend(
                ↪ nd([1.,-1.,worst_net]);ub.append(0.)
149         iz+=n
150     ineq=None if not ub else sparse.coo_matrix((uv,(ur,uc)),shape=(len(ub),nv)).tocsr()
151     return dict(objective=obj,lower=lo,upper=hi,eq=eq,rhs=rhs,ub=ineq,
152                ub_rhs=None if not ub else np.asarray(ub),integrality=integrality,
153                offsets=offsets,shapes=shapes,link_rows=links,value_idx=vid,
154                terminal=terminal,lam=lam,target=target,physical=physical)
155
156 def solve(blocks: list[Block], s0: float, *, retain_problem=False, **options) -> Plan:
157     f=formulate(blocks,s0,**options);tic=perf_counter()
158     if f['integrality'] is None:
159         result=linprog(f['objective'],A_ub=f['ub'],b_ub=f['ub_rhs'],
160                       A_eq=f['eq'],b_eq=f['rhs'],bounds=np.c_[f['lower'],f['upper']],method)
161         ↪ ='highs')
162     else:
163         constraints=[LinearConstraint(f['eq'],f['rhs'],f['rhs'])]
164         if f['ub'] is not None:
165             ↪ constraints.append(LinearConstraint(f['ub'],-np.inf,f['ub_rhs']))
166         result=milp(f['objective'],integrality=f['integrality'],bounds=Bounds(f['lower'],f['upper']),
167                   ↪ constraints=constraints,options={'mip_rel_gap':1e-9,'time_limit':120.})
168     elapsed=perf_counter()-tic
169     if not result.success:
170         raise RuntimeError(f"Q2 solve failed: status={result.status} {result.message}")
171     o=f['offsets'][0];k,n=f['shapes'][0];x=result.x
172     arrays={name:x[o[name]:o[name]+n].copy() for name in ('q','c','b','s')}
173     credit=0. if f['value_idx'] is None else f['lam']*x[f['value_idx']]
174     marginal=0.
175     if f['integrality'] is None:
176         if len(blocks)>1: marginal=max(0.,-float(result.eqlin.marginals[f['link_rows'][1]]))
177         elif f['terminal'] in ('cyclic','floor'):
178             marginal=max(0.,-float(result.ineqlin.marginals[0]))
179         elif f['terminal']=='value': marginal=f['lam']
180     e=x[o['e']:o['e']+k*n].reshape(k,n)
181     diag={'variables':len(x),'equalities':len(f['rhs']),'scenario_count':k,
182          ↪ 'max_simultaneous_charge_discharge':float(np.max(np.minimum(arrays['c'],arrays['b']
183          ↪ ))),
184          ↪ 'emergency_and_charge_scenario_slots':int(np.sum((e>1e-6)&(arrays['c'])[None,:]>1e-
185          ↪ 6))),
186          ↪ 'm0_enforced':f['integrality'] is not None and options.get('enforce_m0',False),
187          ↪ 'solver_status':int(result.status),'solver_message':result.message,
188          ↪ 'primal_equality_error':float(np.max(np.abs(f['eq']@x-f['rhs']))),
189          ↪ 'min_bound_slack':float(min(np.min(x-f['lower']),np.min(f['upper']-x)))}
190     if f['integrality'] is not None:
191         diag.update(mip_gap=float(result.mip_gap),mip_dual_bound=float(result.mip_dual_bound)
192                   ↪ )
193     return Plan(arrays['q'],arrays['c'],arrays['b'],arrays['s'],float(result.fun),
194                float(result.fun+credit),marginal,float(arrays['s'][-1]),elapsed,diag,
195                result if retain_problem else None,f if retain_problem else None)
196
197 def solve_state_lp(load, pv, price, s0, *, fixed_q=None, terminal='free',
198                  target=None, lam=0., physical=Physical(), no_battery_dump=False,
199                  enforce_fixed_q_m0=False, first_discharge_cap=None):
200     """Independently formulated exact piecewise state-increment LP for M1."""
201     load=np.atleast_2d(np.asarray(load,float));pv=np.atleast_2d(np.asarray(pv,float));price=
202     ↪ np.asarray(price,float)
203     k,n=load.shape

```

```

198     if pv.shape!=load.shape or len(price)!=n: raise ValueError('Shapes')
199     # All n+1 states are variables, unlike the explicit formulation.
200     q0=0;sidx=n;eidx=2*n+1;vid=eidx+k*n if terminal=='value' else None
201     nv=eidx+k*n+int(vid is not None)
202     obj=np.zeros(nv);obj[:n]=price;obj[eidx:eidx+k*n]=np.tile(physical.emergency_multiple*pr
    ↪ ice/k,k)
203     lo=np.zeros(nv);hi=np.full(nv,np.inf)
204     lo[sidx:sidx+n+1]=physical.s_min;hi[sidx:sidx+n+1]=physical.s_max
205     lo[sidx]=hi[sidx]=s0
206     if fixed_q is not None:lo[:n]=hi[:n]=np.asarray(fixed_q,float)
207     if enforce_fixed_q_m0 and fixed_q is None:
208         raise ValueError('The convex M0 charge bound requires q already fixed')
209     if terminal in ('cyclic','floor','equal'):
210         level=s0 if target is None else target
211         lo[sidx+n]=level
212         if terminal=='equal':hi[sidx+n]=level
213     if vid is not None:
214         hi[vid]=physical.s_max if target is None else target;obj[vid]=-lam
215     rows=[];cols=[];vals=[];bounds=[]
216     for coefficient in (1/physical.eta_c,physical.eta_d):
217         for s in range(k):
218             for t in range(n):
219                 r=len(bounds); rows.extend([r]*4);cols.extend([t,sidx+t+1,sidx+t,eidx+s*n+t])
220                 vals.extend([-1.,coefficient,-coefficient,-1.]);bounds.append(pv[s,t]-load[s]
    ↪ ,t])
221     for t in range(n):
222         charge_cap=physical.power_energy
223         if enforce_fixed_q_m0:
224             # For fixed q and a common scenario action this bound is exactly
225             # c<=max(0,q-max_s(net_s)), hence e_s*c=0 for every sample.
226             charge_cap=min(charge_cap,max(0.,float(fixed_q[t]-np.max(load[:,t]-pv[:,t]))))
227             r=len(bounds);rows.extend([r,r]);cols.extend([sidx+t+1,sidx+t]);vals.extend([1.,-1.]
    ↪ );bounds.append(physical.eta_c*charge_cap)
228         discharge=physical.power_energy
229         if no_battery_dump:discharge=min(discharge,float(load[:,t].min()))
230         if t==0 and first_discharge_cap is not None:
231             discharge=min(discharge,max(0.,float(first_discharge_cap)))
232             r=len(bounds);rows.extend([r,r]);cols.extend([sidx+t,sidx+t+1]);vals.extend([1.,-1.]
    ↪ );bounds.append(discharge/physical.eta_d)
233     if vid is not None:
234         r=len(bounds);rows.extend([r,r]);cols.extend([vid,sidx+n]);vals.extend([1.,-1.]);bou
    ↪ nds.append(0.)
235     a=sparse.coo_matrix((vals,(rows,cols)),shape=(len(bounds),nv)).tocsr()
236     tic=perf_counter();r=linprog(obj,A_ub=a,b_ub=np.asarray(bounds),bounds=np.c_[lo,hi],meth
    ↪ od='highs');elapsed=perf_counter()-tic
237     if not r.success:raise RuntimeError(r.message)
238     x=np.diff(r.x[sidx:sidx+n+1]);c=np.maximum(x,0)/physical.eta_c;b=np.maximum(-x,0)*physic
    ↪ al.eta_d
239     return dict(q=r.x[:n],c=c,b=b,soc=r.x[sidx:sidx+n+1],e=r.x[eidx:eidx+k*n].reshape(k,n),
240               objective=float(r.fun),seconds=elapsed,result=r)

```

model/contracts.py: 共享合同与信号追索

```

1 """Convex state-increment LPs with final-contract settlement and one signal split."""
2 from functools import lru_cache
3 from time import perf_counter
4
5 import numpy as np
6 from scipy import sparse
7 from scipy.optimize import linprog
8
9 from model.planning import Physical

```

```

10
11 def contract_cost(q, m, price, fee='B'):
12     q, m, p = np.asarray(q,float), np.asarray(m,float), np.asarray(price,float)
13     if fee == 'B':
14         return p*np.minimum(q,m) + .5*p*np.maximum(q-m,0) + 1.5*p*np.maximum(m-q,0)
15     if fee == 'A':
16         return p*q + .5*p*np.maximum(q-m,0) + 1.5*p*np.maximum(m-q,0)
17     raise ValueError('fee must be A or B')
18
19 def _inputs(load,pv,price,s0,physical):
20     load, pv = np.atleast_2d(np.asarray(load,float)), np.atleast_2d(np.asarray(pv,float))
21     p = np.asarray(price,float)
22     if load.shape != pv.shape or p.shape != (load.shape[1],) or load.shape[0] < 1:
23         raise ValueError('scenario/price shape mismatch or empty scenario set')
24     if not all(np.isfinite(a).all() for a in (load,pv,p)) or min(load.min(),pv.min()) < -1e-8
25         ↪ or p.min() <= 0:
26         raise ValueError('nonphysical scenario or nonpositive price')
27     if not physical.s_min-1e-7 <= s0 <= physical.s_max+1e-7:
28         raise ValueError('initial inventory outside bounds')
29     return np.maximum(load,0), np.maximum(pv,0), p
30
31 @lru_cache(maxsize=500)
32 def _shared_topology(k,n,eta_c,eta_d,has_value):
33     qidx=0; sidx=n; eidx=2*n+1; yidx=eidx+k*n if has_value else None
34     nv=eidx+k*n+int(has_value)
35     times=np.tile(np.arange(n),k)
36     rows=[];cols=[];vals=[];cursor=0
37     for alpha in (1/eta_c,eta_d):
38         r=cursor+np.arange(k*n)
39         rows.extend([r]*4)
40         cols.extend([times,sidx+times+1,sidx+times,eidx+np.arange(k*n)])
41         vals.extend([-np.ones(k*n),np.full(k*n,alpha),np.full(k*n,-alpha),-np.ones(k*n)])
42         cursor+=k*n
43     for sign in (1.,-1.):
44         r=cursor+np.arange(n)
45         rows.extend([r,r]);cols.extend([sidx+np.arange(n)+1,sidx+np.arange(n)])
46         vals.extend([np.full(n,sign),np.full(n,-sign)]);cursor+=n
47     if has_value:
48         rows.extend([np.array([cursor]),np.array([cursor])])
49         cols.extend([np.array([yidx]),np.array([sidx+n])])
50         vals.extend([np.array([1.]),np.array([-1.])]);cursor+=1
51     mat=sparse.coo_matrix((np.concatenate(vals),(np.concatenate(rows),np.concatenate(cols))))
52     ↪ , shape=(cursor,nv)).tocsr()
53     return mat,sidx,eidx,yidx,nv
54
55 def solve_shared(load,pv,price,s0,*,fixed_m=None,original_q=None,fee='B',
56     lam=0.,target=None,physical=Physical(),first_discharge_cap=None):
57     """Day-ahead q, adjustable common m, or fixed-m storage subproblem."""
58     if fixed_m is not None and original_q is not None:
59         raise ValueError('choose fixed storage control or contract amendment')
60     if fee not in ('A','B') or lam < 0:
61         raise ValueError('unsupported fee or negative continuation value')
62     load,pv,p=_inputs(load,pv,price,s0,physical)
63     k,n=load.shape;has_value=lam>0
64     mat,sidx,eidx,yidx,nv=_shared_topology(k,n,physical.eta_c,physical.eta_d,has_value)
65     lower=np.zeros(nv);upper=np.full(nv,np.inf);obj=np.zeros(nv)
66     lower[sidx:sidx+n+1]=physical.s_min;upper[sidx:sidx+n+1]=physical.s_max
67     lower[sidx]=upper[sidx]=float(s0)
68     obj[eidx:eidx+k*n]=np.tile(physical.emergency_multiple*p/k,k)
69     if has_value:
70         upper[yidx]=physical.s_max if target is None else float(target)
71         obj[yidx]=-float(lam)
72     dis=np.minimum(physical.power_energy,load.min(axis=0))
73     if first_discharge_cap is not None:dis[0]=min(dis[0],max(0.,float(first_discharge_cap)))
74     rhs=np.r_[(pv-load).ravel(),(pv-load).ravel()],

```

```

73         np.full(n,physical.eta_c*physical.power_energy),dis/physical.eta_d,
74         np.array([0.]) if has_value else np.array([]])
75 if fixed_m is not None:
76     m=np.asarray(fixed_m,float)
77     if m.shape!=(n,) or m.min() < -1e-7:raise ValueError('invalid fixed contract')
78     lower[:n]=upper[:n]=np.maximum(m,0)
79 elif original_q is None:
80     obj[:n]=p
81 else:
82     q=np.asarray(original_q,float)
83     if q.shape!=(n,) or q.min() < -1e-7:raise ValueError('invalid original plan')
84     mat=sparse.hstack((mat,sparse.csr_matrix((mat.shape[0],n))),format='csr')
85     rows=np.r_[np.arange(n),np.arange(n),n+np.arange(n),n+np.arange(n)]
86     cols=np.r_[np.arange(n),nv+np.arange(n),np.arange(n),nv+np.arange(n)]
87     second=.5 if fee=='B' else -.5
88     vals=np.r_[1.5*p,-np.ones(n),second*p,-np.ones(n)]
89     fee_mat=sparse.coo_matrix((vals,(rows,cols)),shape=(2*n,nv+n)).tocsr()
90     mat=sparse.vstack((mat,fee_mat),format='csr')
91     rhs=np.r_[rhs,.5*p*q,(-.5 if fee=='B' else -1.5)*p*q]
92     lower=np.r_[lower,np.zeros(n)];upper=np.r_[upper,np.full(n,np.inf)]
93     obj=np.r_[obj,np.ones(n)]
94     start=perf_counter()
95     result=linprog(obj,A_ub=mat,b_ub=rhs,bounds=np.c_[lower,upper],method='highs')
96     seconds=perf_counter()-start
97     if not result.success:raise RuntimeError(f'Q3 shared LP: {result.status}:
98     ↳ {result.message}')
99     sol=result.x; m=sol[:n].copy();soc=sol[sidx:sidx+n+1].copy();x=np.diff(soc)
100    c=np.maximum(x,0)/physical.eta_c;b=np.maximum(-x,0)*physical.eta_d
101    e=sol[eidx:eidx+k*n].reshape(k,n)
102    if original_q is not None:fee_values=contract_cost(original_q,m,p,fee)
103    elif fixed_m is not None:fee_values=np.zeros(n)
104    else:fee_values=p*m
105    credit=0 if not has_value else lam*sol[yidx]
106    expected=float(fee_values.sum()+physical.emergency_multiple*p@e.mean(axis=0))
107    if abs(float(result.fun)-(expected-credit))>1e-5:
108        raise AssertionError('shared LP objective/independent cost disagreement')
109    return dict(m=m,q=q,c=c,b=b,soc=soc,e=e,objective=float(result.fun),
110        expected_cost=expected,contract_cost=fee_values,terminal_credit=float(credit),
111        seconds=seconds,variables=len(sol),max_primal_violation=float(max(0.,np.max(mat@sol-
112        ↳ rhs))),
113        solver_status=int(result.status),fixed_contract_constant_omitted=fixed_m is not None)
114
115 def solve_signal_recourse(load,pv,price,s0,groups,*,split=36,allow_amend=True,
116     fee='B',lam=0.,target=None,physical=Physical()):
117     """One observed 06:00 signal, with group-shared post-signal decisions."""
118     load,pv,p=_inputs(load,pv,price,s0,physical)
119     k,n=load.shape
120     raw=np.asarray(groups,int)
121     if raw.shape!=(k,) or not 0<=split<=n or fee not in ('A','B') or lam<0:
122         raise ValueError('invalid signal groups, split, or fee')
123     labels,gidx=np.unique(raw,return_inverse=True);ng=len(labels)
124     weights=np.bincount(gidx,minlength=ng)/k
125     q0=0;m0=n;s0idx=m0+ng*n;e0=s0idx+ng*(n+1);z0=e0+k*n
126     y0=z0+ng*n;nv=y0+(ng if lam>0 else 0)
127     lo=np.zeros(nv);hi=np.full(nv,np.inf);obj=np.zeros(nv)
128     for g in range(ng):
129         lo[s0idx+g*(n+1):s0idx+(g+1)*(n+1)]=physical.s_min
130         hi[s0idx+g*(n+1):s0idx+(g+1)*(n+1)]=physical.s_max
131         lo[s0idx+g*(n+1)]=hi[s0idx+g*(n+1)]=float(s0)
132         obj[z0+g*n:z0+(g+1)*n]=weights[g]
133         if lam>0:
134             obj[y0+g]=-lam*weights[g]
135             hi[y0+g]=physical.s_max if target is None else target
136     obj[e0:e0+k*n]=np.tile(physical.emergency_multiple*p/k,k)
137     rr=[];cc=[];vv=[];rhs=[]

```

```

136 def ub(cols,vals,bound):
137     r=len(rhs);rr.extend([r]*len(cols));cc.extend(cols);vv.extend(vals);rhs.append(bound)
138 for s in range(k):
139     g=int(gidx[s]);si=s0idx+g*(n+1)
140     for t in range(n):
141         for alpha in (1/physical.eta_c,physical.eta_d):
142             ub([si+t+1,si+t,m0+g*n+t,e0+s*n+t],
143                [alpha,-alpha,-1.,-1.],pv[s,t]-load[s,t])
144 for g in range(ng):
145     si=s0idx+g*(n+1);minload=load[gidx==g].min(axis=0)
146     for t in range(n):
147         ub([si+t+1,si+t],[1.,-1.],physical.eta_c*physical.power_energy)
148         ub([si+t,si+t+1],[1.,-1.],min(physical.power_energy,minload[t])/physical.eta_d)
149         mi,zi=m0+g*n+t,z0+g*n+t
150         ub([mi,q0+t,zi],[1.5*p[t],-.5*p[t],-1.],0.)
151         if fee=='B':ub([mi,q0+t,zi],[.5*p[t],.5*p[t],-1.],0.)
152         else:ub([mi,q0+t,zi],[-.5*p[t],1.5*p[t],-1.],0.)
153         if lam>0:ub([y0+g,si+n],[1.,-1.],0.)
154 er=[];ec=[];ev=[];eb=[]
155 def eq(cols,vals):
156     r=len(eb);er.extend([r]*len(cols));ec.extend(cols);ev.extend(vals);eb.append(0.)
157 for g in range(ng):
158     for t in range(split if allow_amend else n):eq([m0+g*n+t,q0+t],[1.,-1.])
159     if g:
160         for t in range(1,split+1):eq([s0idx+g*(n+1)+t,s0idx+t],[1.,-1.])
161 mat=sparse.coo_matrix((vv,(rr,cc)),shape=(len(rhs),nv)).tocsr()
162 emat=sparse.coo_matrix((ev,(er,ec)),shape=(len(eb),nv)).tocsr()
163 start=perf_counter()
164 r=linprog(obj,A_ub=mat,b_ub=np.asarray(rhs),A_eq=emat,b_eq=np.asarray(eb),
165           bounds=np.c_[lo,hi],method='highs')
166 seconds=perf_counter()-start
167 if not r.success:raise RuntimeError(f'Q3 signal LP: {r.status}: {r.message}')
168 q=r.x[:n].copy();m=r.x[m0:m0+ng*n].reshape(ng,n).copy()
169 states=r.x[s0idx:s0idx+ng*(n+1)].reshape(ng,n+1).copy()
170 increments=np.diff(states,axis=1);c=np.maximum(increments,0)/physical.eta_c;b=np.maximum
    ↪ (-increments,0)*physical.eta_d
171 e=r.x[e0:e0+k*n].reshape(k,n)
172 fees=contract_cost(q[None,:],m,p[None,:],fee)
173 expected=float(weights@fees.sum(axis=1)+physical.emergency_multiple*p@e.mean(axis=0))
174 credit=float(lam*weights@r.x[y0:y0+ng]) if lam>0 else 0.
175 if abs(r.fun-(expected-credit))>1e-5:raise AssertionError('signal objective
    ↪ double-counting or extraction error')
176 pre_error=float(np.max(np.abs(states[:,split+1]-states[0,:split+1])))
177 if pre_error>1e-6:raise AssertionError('pre-signal storage anticipates future group')
178 return dict(q=q,m_by_group=m,soc_by_group=states,c_by_group=c,b_by_group=b,
179            e=e,group_labels=labels,scenario_groups=gidx,group_weights=weights,
180            objective=float(r.fun),expected_cost=expected,terminal_credit=credit,
181            contract_cost_by_group=fees,pre_signal_state_error=pre_error,
182            seconds=seconds,variables=nv,solver_status=int(r.status),
183            max_primal_violation=float(max(0.,np.max(mat@r.x-np.asarray(rhs))))

```

model/control.py: 问题二计划与实时控制

```

1 """Past-information planning and real-time control for the corrected Q2 run."""
2 from __future__ import annotations
3
4 from dataclasses import asdict, dataclass
5 from time import perf_counter
6
7 import numpy as np
8
9 from model.history import ForecastArchive
10 from model.planning import Block, Physical, Plan, solve, solve_state_lp

```

```

11 from model.settlement import settle_schedule, independent_recalculator
12
13 @dataclass(frozen=True)
14 class Config:
15     name: str
16     plan_kind: str = 'S'
17     controller: str = 'fast'
18     terminal: str = 'value'
19     scenario_count: int = 30
20     control_scenario_count: int = 10
21     scenario_method: str = 'residual'
22     lag_anchor: str = 'corrected'
23     physical_guard: bool = False
24     flat_planning_price: bool = False
25     no_battery: bool = False
26     strict_delivery: bool = False
27     mip_time_limit: float = 120.0
28     mip_gap: float = 1e-8
29
30 PRIMARY_CONFIGS = {
31     'D': Config('D', plan_kind='D', controller='fixed', physical_guard=True),
32     'S': Config('S', controller='fixed', physical_guard=True),
33     'F_fast': Config('F_fast'),
34     'F_lagged': Config('F_lagged', controller='lagged'),
35     'F_lagged_guard': Config('F_lagged_guard', controller='lagged', physical_guard=True),
36     'No_storage':
37     ↪ Config('No_storage', controller='fixed', no_battery=True, physical_guard=True),
37 }
38
39 def continuation_value(day: int, stock: float, archive: ForecastArchive,
40     ↪ physical=Physical()):
41     """Derived local continuation slope, using two-day deterministic LP only."""
42     p=archive.data.price
43     l0,v0=archive.point(day);l1,v1=archive.point(day,day+1)
44     plan=solve([Block(l0,v0,p),Block(l1,v1,p)],stock,
45         terminal='cyclic',no_battery_dump=True,physical=physical)
46     return plan.lam,plan.end_target,dict(source='two_day_deterministic_LP_link_dual',
47         lambda_yuan_per_internal_kwh=plan.lam,target_kwh=plan.end_target,
48         forecast_origin_index=day,future_target_index=day+1,
49         latest_observed_day=day-1,objective=plan.objective,seconds=plan.seconds)
50
51 def make_plan(day: int, stock: float, archive: ForecastArchive, config: Config,
52     physical=Physical()):
53     """Return a sealed day-ahead plan. Never accepts current actual L/PV."""
54     if config.strict_delivery:
55         raise NotImplementedError('Corrected strict delivery is not a primary configuration;
56     ↪ reference replay is labeled separately')
57     if config.flat_planning_price:
58         price=np.full(144,float(archive.data.price.mean()))
59     else:price=archive.data.price
60     lh,vh=archive.point(day)
61     if config.plan_kind=='D':
62         loads,pvs=lh[None,:],vh[None,:]
63         scenario_meta={'method':'point','effective_history_days':None,'latest_observed_day':}
64         ↪ day-1}
65     elif config.plan_kind=='S':
66         loads,pvs,scenario_meta=archive.scenarios(day,config.scenario_count,config.scenario_)
67         ↪ method)
68     else:raise ValueError(config.plan_kind)
69     lam,target,dual_meta=continuation_value(day,stock,archive,physical)
70     term=config.terminal
71     if term=='free': lam=0.;target=None
72     elif term=='cyclic': target=stock
73     elif term!='value':raise ValueError(term)
74     tic=perf_counter()

```

```

71     if config.no_battery:
72         # Exact minimizers of nonnegative empirical newsvendor loss. The
73         # lower 80th percentile is a valid choice within an optimal interval.
74         net=loads-pvs
75         q=np.maximum(0.,np.quantile(net,.8,axis=0,method='inverted_cdf'))
76         c=np.zeros(144);b=np.zeros(144);s=np.full(144,stock)
77         expectation=float(price@q+physical.emergency_multiple*price@np.maximum(net-q,0).mean(
78             ↪ (axis=0))
79         plan=Plan(q,c,b,s,expectation,expectation,0.,stock,perf_counter()-tic,
80             {'formulation':'exact_no_storage_empirical_quantile','m0_enforced':False,
81             'scenario_count':len(loads),'variables':144,'constraints':0})
82     else:
83         r=solve_state_lp(loads,pvs,price,stock,terminal=term,target=target,lam=lam,
84             physical=physical,no_battery_dump=True)
85         e=r['e'];c=r['c'];b=r['b'];s=r['soc'][1:]
86         cash=float(price@r['q']+physical.emergency_multiple*price@e.mean(axis=0))
87         plan=Plan(r['q'],c,b,s,r['objective'],cash,lam,float(s[-1]),r['seconds'],
88             {'formulation':'state_increment_exact_LP','m0_enforced':False,
89             'scenario_count':len(loads),'variables':len(r['result'].x),
90             'max_charge_discharge_product_kwh2':float(np.max(c*b)),
91             'emergency_charge_scenario_slots':int(np.count_nonzero((e>1e-6)&(c[None,: ]
92             ↪ ]>1e-6))))))
93     plan.lam=lam
94     # The terminal credit function is part of the frozen model. Its cap is
95     # NOT the SOC that one particular day-ahead optimum happens to achieve.
96     # Preserve the H2-derived cap (or cyclic floor) for every intraday LP.
97     optimized_end_soc=float(plan.soc[-1])
98     plan.end_target=float(target) if target is not None else physical.s_max
99     meta={'day':day,'config':asdict(config),'initial_soc':stock,
100         'forecast_origin_index':day,'max_actual_day_used':day-1,
101         'scenario_metadata':scenario_meta,'continuation':dual_meta,
102         'objective':plan.objective,'expected_cash':plan.expected_cash,
103         'optimized_plan_end_soc':optimized_end_soc,'controller_terminal_target':plan.end_t
104         ↪ arget,
105         'plan_seconds':plan.seconds,'diagnostics':plan.diagnostics}
106     return plan,meta
107
108 def protect_intent(c: float,b: float,stock: float,q: float,load: float,pv: float,
109     physical=Physical()):
110     """Explicit common physical protection for fixed intents, never hidden."""
111     x=physical.eta_c*c-b/physical.eta_d
112     intended_c=max(0.,x)/physical.eta_c;intended_b=max(0.,-x)*physical.eta_d
113     if x>=0:
114         cap=max(0.,min(physical.power_energy,(physical.s_max-stock)/physical.eta_c))
115         actual_c=min(intended_c,cap);actual_b=0.
116     else:
117         cap=max(0.,min(physical.power_energy,(stock-physical.s_min)*physical.eta_d,
118             load-pv-q))
119         actual_c=0.;actual_b=min(intended_b,cap)
120     delta=abs(actual_c-c)+abs(actual_b-b)
121     return actual_c,actual_b,{'guard_applied':delta>1e-6,'guard_delta_bus_kwh':delta,
122         'charge_power_soc_reduction_kwh':max(0.,intended_c-actual_c),
123         'unneeded_discharge_reduction_kwh':max(0.,intended_b-actual_b)}
124
125 def feedback_action(day: int,t: int,stock: float,plan: Plan,archive: ForecastArchive,
126     observed_load,observed_pv,config: Config,physical=Physical()):
127     """One remaining-horizon LP. Receives ONLY its allowed observation prefix."""
128     fast=config.controller=='fast'
129     if config.controller not in ('fast','lagged'):raise ValueError(config.controller)
130     loads,pvs,meta=archive.conditional(day,t,observed_load,observed_pv,
131         observe_current=fast,K=config.control_scenario_count,
132         lag_anchor=config.lag_anchor,method=config.scenario_method)
133     p=archive.data.price[t:]
134     if config.flat_planning_price:p=np.full_like(p,float(archive.data.price.mean()))
135     # Avoid gratuitous discharge in the current sample set. For fast, all

```

```

133 # current samples are the actual measurement, so this is an exact limit.
134 discharge_cap=max(0.,float(np.min(loads[:,0]-pvs[:,0]-plan.q[t])))
135 r=solve_state_lp(loads,pvs,p,stock,fixed_q=plan.q[t:],terminal=config.terminal,
136                 target=plan.end_target,lam=plan.lam,physical=physical,
137                 no_battery_dump=True,enforce_fixed_q_m0=False,
138                 first_discharge_cap=discharge_cap)
139 c,b=float(r['c'][0]),float(r['b'][0])
140 diagnostic={'t':t,'observed_slots':len(observed_load),'lp_objective':r['objective'],
141            'lp_seconds':r['seconds'],'sample_count':len(loads),
142            'current_load_min':float(loads[:,0].min()),'current_load_max':float(loads[:,0].max()),
143            'current_pv_min':float(pvs[:,0].min()),'current_pv_max':float(pvs[:,0].max()),
144            'planned_end_soc':plan.end_target,'continuation_lambda':plan.lam,
145            'metadata':meta,'physical_projection_count':0}
146 return c,b,diagnostic
147
148 def execute_day(day: int,initial_soc: float,plan: Plan,archive: ForecastArchive,
149               config: Config,physical=Physical(), *, require_feasible=True):
150     """Reveals the current row only after the caller has saved the plan."""
151     load=archive.data.load[day];pv=archive.data.pv[day];price=archive.data.price
152     c=np.empty(144);b=np.empty(144);proposed_c=np.empty(144);proposed_b=np.empty(144)
153     stock=initial_soc;details=[];guard_count=0;guard_energy=0.
154     tic=perf_counter()
155     for t in range(144):
156         if config.controller=='fixed':
157             ct,bt=float(plan.c[t]),float(plan.b[t]);item={'t':t,'observed_slots':0,'lp_seconds':0}
158         else:
159             prefix=t+int(config.controller=='fast')
160             ct,bt,item=feedback_action(day,t,stock,plan,archive,load[:prefix],pv[:prefix],config,physical)
161             proposed_c[t],proposed_b[t]=ct,bt
162             if config.physical_guard:
163                 ct,bt,guard=protect_intent(ct,bt,stock,float(plan.q[t]),float(load[t]),float(pv[t]),physical)
164                 item.update(guard);guard_count+=int(guard['guard_applied']);guard_energy+=guard['guard_delta_bus_kwh']
165             else:item.update(guard_applied=False,guard_delta_bus_kwh=0.)
166             # This is a physical feasibility audit, not an action modification.
167             try:
168                 one=settle_schedule([plan.q[t]],[ct],[bt],[load[t]],[pv[t]],stock,[price[t]],
169                                   eta_charge=physical.eta_c,eta_discharge=physical.eta_d,soc_min=physical.s_min,
170                                   soc_max=physical.s_max,bus_limit=physical.power_energy,
171                                   emergency_multiplier=physical.emergency_multiple,enforce_m0=False)
172             except ValueError as exc:
173                 failure={'day':day,'t':t,'config':asdict(config),'reason':str(exc),
174                        'soc_before':stock,'q':float(plan.q[t]),'c':ct,'b':bt,
175                        'actual_load':float(load[t]),'actual_pv':float(pv[t]),
176                        'proposed_c':float(proposed_c[t]),'proposed_b':float(proposed_b[t]),
177                        'completed_charge_actions':c[:t].tolist(),
178                        'completed_discharge_actions':b[:t].tolist(),
179                        'observation_count':len(load[:t+int(config.controller=='fast')])),
180                        'controller_diagnostics':item,'previous_step_details':details}
181                 error=RuntimeError('Strict runtime physical infeasibility')
182                 error.failure_record=failure
183                 raise error from exc
184             c[t],b[t]=ct,bt;stock=float(one['soc_end'][0]);details.append(item)
185     ledger=settle_schedule(plan.q,c,b,load,pv,initial_soc,price,
186                           eta_charge=physical.eta_c,eta_discharge=physical.eta_d,soc_min=physical.s_min,
187                           soc_max=physical.s_max,bus_limit=physical.power_energy,
188                           emergency_multiplier=physical.emergency_multiple,enforce_m0=False)

```

```

189     recalc=independent_recalculator(plan.q,c,b,load,pv,initial_soc,price,ledger,
190         eta_charge=physical.eta_c,eta_discharge=physical.eta_d,soc_min=physical.s_mi
        ↪ n,
191         soc_max=physical.s_max,bus_limit=physical.power_energy,
192         emergency_multiplier=physical.emergency_multiple,enforce_m0=False)
193     if not recalc['passed']:raise AssertionError(recalc)
194     return ledger,dict(steps=details,proposed_c=proposed_c,proposed_b=proposed_b,
195         independent_recalculation=recalc,execution_seconds=perf_counter()-tic,
196         guard_count=guard_count,guard_energy_kwh=guard_energy,
197         feedback_projection_count=0 if not config.physical_guard else
        ↪ guard_count)

```

model/lagged.py: 滞后控制对照

```

1  """Past-only reference LP; current observations enter execution protection only."""
2  from time import perf_counter
3
4  import numpy as np
5
6  from model.planning import Physical, solve_state_lp
7  from model.control import Config, make_plan, protect_intent
8  from model.settlement import settle_schedule, independent_recalculator
9
10 PRIMARY_CONFIGS = {
11     'F_lagged_no_min_guard': Config('F_lagged_no_min_guard', controller='lagged',
        ↪ physical_guard=True),
12 }
13
14 def feedback_action(day, t, stock, plan, archive, observed_load, observed_pv,
15     config, physical=Physical()):
16     if config.controller != 'lagged' or not config.physical_guard:
17         raise ValueError('This experiment requires lagged planning and measured protection')
18     loads, pvs, meta = archive.conditional(
19         day, t, observed_load, observed_pv, observe_current=False,
20         K=config.control_scenario_count, lag_anchor=config.lag_anchor,
21         method=config.scenario_method)
22     price = archive.data.price[t:]
23     result = solve_state_lp(loads, pvs, price, stock, fixed_q=plan.q[t:],
24         terminal=config.terminal, target=plan.end_target, lam=plan.lam,
25         physical=physical, no_battery_dump=True, enforce_fixed_q_m0=False,
26         first_discharge_cap=None)
27     diagnostic = dict(t=t, observed_slots=len(observed_load),
28         lp_objective=result['objective'], lp_seconds=result['seconds'],
29         sample_count=len(loads), extra_first_discharge_bound=False,
30         old_bound_for_diagnostic=max(0., float(np.min(loads[:,0]-pvs[:,0]-plan.q[t]))),
31         metadata=meta, physical_projection_count=0)
32     return float(result['c'][0]), float(result['b'][0]), diagnostic
33
34 def execute_day(day, initial_soc, plan, archive, config, physical=Physical(), *,
        ↪ require_feasible=True):
35     load, pv, price = archive.data.load[day], archive.data.pv[day], archive.data.price
36     n = len(price)
37     c, b, pc, pb = (np.empty(n) for _ in range(4))
38     stock = float(initial_soc)
39     details, guards, guard_energy = [], 0, 0.
40     start = perf_counter()
41     for t in range(n):
42         ct, bt, item = feedback_action(day, t, stock, plan, archive,
43             load[:t], pv[:t], config, physical)
44         pc[t], pb[t] = ct, bt
45         ct, bt, guard = protect_intent(ct, bt, stock, plan.q[t], load[t], pv[t], physical)
46         item.update(guard)
47         item['physical_projection_count'] = int(guard['guard_applied'])

```

```

48     guards += int(guard['guard_applied'])
49     guard_energy += guard['guard_delta_bus_kwh']
50     one = settle_schedule([plan.q[t]], [ct], [bt], [load[t]], [pv[t]], stock, [price[t]],
51         eta_charge=physical.eta_c, eta_discharge=physical.eta_d,
52         soc_min=physical.s_min, soc_max=physical.s_max, bus_limit=physical.power_energy,
53         emergency_multiplier=physical.emergency_multiple, enforce_m0=False)
54     c[t], b[t], stock = ct, bt, float(one['soc_end'][0])
55     details.append(item)
56     ledger = settle_schedule(plan.q, c, b, load, pv, initial_soc, price,
57         eta_charge=physical.eta_c, eta_discharge=physical.eta_d,
58         soc_min=physical.s_min, soc_max=physical.s_max, bus_limit=physical.power_energy,
59         emergency_multiplier=physical.emergency_multiple, enforce_m0=False)
60     independent = independent_recalculator(plan.q, c, b, load, pv, initial_soc, price,
61         ↪ ledger,
62         eta_charge=physical.eta_c, eta_discharge=physical.eta_d,
63         soc_min=physical.s_min, soc_max=physical.s_max, bus_limit=physical.power_energy,
64         emergency_multiplier=physical.emergency_multiple, enforce_m0=False)
65     if not independent['passed']:
66         raise AssertionError(independent)
67     return ledger, dict(steps=details, proposed_c=pc, proposed_b=pb,
68         independent_recalculation=independent, execution_seconds=perf_counter()-start,
69         guard_count=guards, guard_energy_kwh=guard_energy,
70         feedback_projection_count=guards,
71         observation_contract='reference prefix [0,t]; measured protection uses current
72         ↪ aggregate')

```

model/amendments.py: 发布后合同调整

```

1  """One-signal planning and causal rolling final-contract execution for Q3."""
2  from dataclasses import asdict, dataclass
3  from time import perf_counter
4
5  import numpy as np
6
7  from model.history import ar1_coef
8  from model.planning import Block, Physical, solve
9  from model.control import protect_intent
10 from model.contracts import solve_shared, solve_signal_recourse
11 from model.contract_cash import settle, independent_recalculate
12
13 @dataclass(frozen=True)
14 class Config:
15     name: str
16     zero_model: str='Z1'
17     enabled_hours: tuple=(6,12,18)
18     amend: bool=True
19     fee: str='B'
20     controller: str='lagged'
21     terminal18: bool=False
22     scenario_count: int=30
23     control_scenario_count: int=10
24
25     def __post_init__(self):
26         if self.zero_model not in ('Z1','Z2') or self.fee not in ('A','B'):
27             raise ValueError('Invalid model or fee')
28         if self.controller not in ('lagged','fast') or
29             ↪ tuple(sorted(set(self.enabled_hours)))!=self.enabled_hours:
30             raise ValueError('Invalid controller or publication sequence')
31         if any(h not in (6,12,18) for h in self.enabled_hours):raise ValueError('Invalid
32             ↪ issue hour')
33         if self.terminal18 and 18 not in self.enabled_hours:raise ValueError('18 refresh
34             ↪ requires 18 publication')

```

```

33 @dataclass(frozen=True)
34 class PriceInputs:
35     """Day-indexed decision forecasts are distinct from realized settlement."""
36     decision: np.ndarray
37     settlement: np.ndarray
38     next_day_forecast: np.ndarray
39     provenance: str
40
41     def __post_init__(self):
42         arrays=[np.asarray(v,float) for v in
43                 ↪ (self.decision,self.settlement,self.next_day_forecast)]
44         if any(v.shape!=(365,144) or not np.isfinite(v).all() or v.min()<=0 for v in arrays):
45             raise ValueError('Each price input must be positive finite day x 144')
46         for key,value in zip(('decision','settlement','next_day_forecast'),arrays):
47             value=value.copy();value.setflags(write=False);object.__setattr__(self,key,value)
48
49     @classmethod
50     def q3(cls,data):
51         matrix=np.tile(data.price,(365,1))
52         return cls(matrix,matrix,matrix,'Observed attachment1 deterministic delivery-period
53                 ↪ tariff')
54
55 @dataclass
56 class DayPlan:
57     q: np.ndarray
58     c: np.ndarray
59     b: np.ndarray
60     soc: np.ndarray
61     lam: float
62     target: float
63     initial_soc: float
64     h2_load: np.ndarray
65     h2_pv: np.ndarray
66     h2_price: np.ndarray
67     seconds: float
68     metadata: dict
69
70     def effective_blocks(enabled_hours):
71         starts=[h*6 for h in enabled_hours]
72         return {a:b for a,b in zip(starts,starts[1:]+[144])}
73
74     def continuation_from_frozen(load,pv,price,s0,physical=Physical()):
75         p=solve([Block(load[0],pv[0],price[0]),Block(load[1],pv[1],price[1])],
76                s0,terminal='cyclic',no_battery_dump=True,physical=physical)
77         return p.lam,p.end_target,dict(source='two-day deterministic link dual',
78                lam=p.lam,cap=p.end_target,objective=p.objective,seconds=p.seconds)
79
80     def make_plan(day,stock,archive,prices,config,physical=Physical()):
81         start=perf_counter();lh,vh,point_meta=archive.point(day,0)
82         nextload,nextpv=archive.load_archive.point(day,day+1)
83         h2load=np.stack((lh,nextload));h2pv=np.stack((vh,nextpv))
84         h2price=np.stack((prices.decision[day],prices.next_day_forecast[day]))
85         lam,target,dual=continuation_from_frozen(h2load,h2pv,h2price,stock,physical)
86         loads,pvs,smeta=archive.scenarios(day,0,config.scenario_count)
87         if len(loads)==0:
88             loads,pvs=lh[None,:],vh[None,:]
89             smeta.update(cold_start_fallback='Assumed deterministic point; no empirical residual
90                 ↪ invented')
91         use_signal=config.zero_model=='Z2' and 6 in config.enabled_hours
92         if use_signal:
93             pairs=archive.z2_signal_pairs(day,config.scenario_count)
94             groups=pairs['groups'] if len(pairs['groups']) else np.zeros(1,int)
95             result=solve_signal_recourse(loads,pvs,prices.decision[day],stock,groups,
96                 allow_amend=config.amend,fee=config.fee,lam=lam,target=target,physical=physical)

```

```

94     c=result['group_weights']@result['c_by_group'];b=result['group_weights']@result['b_b']
95     ↪ y_group']
96     soc=result['group_weights']@result['soc_by_group']
97     extra=dict(signal=pairs['metadata'],signal_thresholds=pairs['thresholds'],
98               group_weights=result['group_weights'],pre_signal_state_error=result['pre_signal_']
99               ↪ state_error'],
100               hypothetical_group_contracts=result['m_by_group'],
101               hypothetical_group_soc=result['soc_by_group'])
102 else:
103     result=solve_shared(loads,pvs,prices.decision[day],stock,lam=lam,target=target,physi
104     ↪ cal=physical)
105     c,b,soc=result['c'],result['b'],result['soc'];extra={}
106     meta=dict(day=day,initial_soc=stock,config=asdict(config),point=point_meta,
107               scenarios=smeta,continuation=dual,objective=result['objective'],
108               expected_cash=result['expected_cost'],terminal_credit=result['terminal_credit'],
109               effective_zero_model='Z2' if use_signal else 'Z1',
110               q_latest_actual_slot=day*144-1,max_publication_hour_used=0,
111               day_ahead_actions_are_reference_only=True,
112               z2_contract_candidates_not_yet_committed=use_signal,**extra)
113     return DayPlan(result['q'].copy(),c,b,soc,lam,target,stock,h2load,h2pv,h2price,
114                   perf_counter()-start,meta)
115
116 def terminal_refresh18(day,plan,archive,physical=Physical(),replace_pv=True):
117     """Freeze the 00 H2 problem; substitute only tomorrow's published PV part."""
118     _,vp,meta=archive.point(day,18,as_of_abs_slot=day*144+108)
119     changed=plan.h2_pv.copy()
120     if replace_pv:changed[1,:108]=vp[36:144]
121     lam,target,dual=continuation_from_frozen(plan.h2_load,changed,plan.h2_price,
122     plan.initial_soc,physical)
123     dual.update(applied_from_slot=108,replace_tomorrow_pv=replace_pv,
124     frozen_day_zero_initial_soc=plan.initial_soc,
125     replacement_nextday_slots=[0,108],publication=meta['published_at'],
126     changed_pv_l1=float(np.abs(changed-plan.h2_pv).sum()),
127     interpretation='terminal approximation refresh, not realized-state continuation
128     ↪ value')
129     return lam,target,dual
130
131 class ConditionalArchive:
132     def __init__(self,archive):self.archive=archive;self._base={}
133
134     def base(self,day,hour,K):
135         key=(day,hour,K)
136         if key not in self._base:
137             lh,vh,_=self.archive.point(day,hour)
138             el,ev,indices,meta=self.archive.residuals(day,hour,30)
139             rl,rv=(ar1_coef(el),ar1_coef(ev)) if len(el) else (0.,0.)
140             order=np.arange(len(el));seed=day+365*hour
141             if len(order)>K:order=np.random.default_rng(seed).choice(len(order),K,replace=Fa
142             ↪ lse)
143             if len(order):el,ev,indices=el[order],ev[order],indices[order]
144             else:
145                 el,ev=np.zeros((1,144)),np.zeros((1,144))
146                 meta.update(cold_start_fallback='Assumed zero-error point path')
147                 self._base[key]=(lh,vh,el,ev,rl,rv,indices,meta,seed)
148         return self._base[key]
149
150     def remaining(self,day,t,hour,observed_load,observed_pv,config):
151         fast=config.controller=='fast';allowed=t+int(fast)
152         ol,ov=np.asarray(observed_load,float),np.asarray(observed_pv,float)
153         if ol.shape!=(allowed,) or ov.shape!=(allowed,) or not np.isfinite(ol).all() or not
154         ↪ np.isfinite(ov).all():
155             raise ValueError('Controller must receive exactly its allowed completed/current
156             ↪ prefix')
157         u=hour*6;i=t-u;n=144-t
158         if i<0:raise ValueError('Unpublished forecast requested')

```

```

152     lh,vh,el,ev,r1,rv,indices,meta,seed=self.base(day,hour,config.control_scenario_count)
153     r=t if fast else t-1;ri=r-u
154     if ri<0:
155         loads=lh[i:i+n][None,:]+el[:,i:i+n]
156         pvs=vh[i:i+n][None,:]+ev[:,i:i+n]
157     else:
158         steps=np.arange(t,144)-r;fl=r1**steps;fv=rv**steps
159         loads=lh[i:i+n][None,:]+el[:,i:i+n]+fl[None,:]*(ol[r]-lh[ri]-el[:,ri])
160         pvs=vh[i:i+n][None,:]+ev[:,i:i+n]+fv[None,:]*(ov[r]-vh[ri]-ev[:,ri])
161     loads,pvs=np.maximum.loads,0),np.maximum(pvs,0)
162     pvs[:,vh[i:i+n]<=1e-9]=0
163     if fast:loads[:,0]=ol[-1];pvs[:,0]=ov[-1]
164     return loads,pvs,dict(issue_hour=hour,observed_slots=allowed,
165         last_observed_slot=allowed-1,residual_anchor_slot=r if ri>=0 else None,
166         historical_days=indices.tolist(),rho_load=r1,rho_pv=rv,seed=seed,
167         historical_target_latest_abs_slot=meta['latest_target_actual_abs_slot'],
168         sample_count=len.loads),reference_uses_current=fast)
169
170 def amendment(day,t,stop,stock,q,archive,prices,config,lam,target,physical=Physical()):
171     """Return an effective current block and explicitly uncommitted candidates."""
172     hour=t//6;loads,pvs,meta=archive.scenarios(day,hour,config.scenario_count)
173     if len.loads)==0:
174         lh,vh,_=archive.point(day,hour);loads,pvs=lh[None:],vh[None:]
175         meta.update(cold_start_fallback='Assumed point path')
176     r=solve_shared.loads[:, :144-t],pvs[:, :144-t],prices.decision[day,t:],stock,
177         original_q=q[t:],fee=config.fee,lam=lam,target=target,physical=physical)
178     return r['m'][:stop-t].copy(),dict(issue_hour=hour,committed_start=t,
179         committed_stop=stop,candidate_start=stop,candidate_m=r['m'][:stop-t],
180         committed_m=r['m'][:stop-t],lp_objective=r['objective'],
181         expected_remaining_contract_cost=float(r['contract_cost'].sum()),
182         seconds=r['seconds'],forecast_metadata=meta,lam=lam,target=target,
183         original_plan_not_mutated=True)
184
185 def execute_day(day,stock,plan,archive,prices,config,physical=Physical(),conditional=None):
186     start=perf_counter();initial=float(stock)
187     conditional=conditional or ConditionalArchive(archive)
188     load,pv=archive.data.load[day],archive.data.pv[day]
189     q=plan.q;m=q.copy();c=np.empty(144);b=np.empty(144)
190     pc=np.empty(144);pb=np.empty(144);steps=[];amendments=[];refresh=None
191     blocks=effective_blocks(config.enabled_hours);hour=0;lam=plan.lam;target=plan.target
192     guards=0;guard_energy=0.
193     for t in range(144):
194         if t in blocks:
195             hour=t//6
196             if t==108 and config.terminal18:
197                 lam,target,refresh=terminal_refresh18(day,plan,archive,physical)
198             if config.amend:
199                 current,record=amendment(day,t,blocks[t],stock,q,archive,prices,config,lam,t)
200                 ↪ arget,physical)
201                 m[t:blocks[t]]=current;amendments.append(record)
202             prefix=t+int(config.controller=='fast')
203             ls,vs,info=conditional.remaining(day,t,hour,load[:prefix],pv[:prefix],config)
204             cap=max(0.,float(load[t]-pv[t]-m[t])) if config.controller=='fast' else None
205             result=solve_shared(ls,vs,prices.decision[day,t:],stock,fixed_m=m[t:],
206                 lam=lam,target=target,physical=physical,first_discharge_cap=cap)
207             pc[t],pb[t]=result['c'][0],result['b'][0]
208             ct,bt,guard=protect_intent(pc[t],pb[t],stock,m[t],load[t],pv[t],physical)
209             c[t],b[t]=ct,bt;stock+=physical.eta_c*ct-bt/physical.eta_d
210             guards+=int(guard['guard_applied']);guard_energy+=guard['guard_delta_bus_kwh']
211             steps.append(dict(t=t,lp_seconds=result['seconds'],lp_objective=result['objective'],
212                 lam=lam,target=target,reference=info,**guard))
212     ledger=settle(q,m,c,b,load,pv,initial,prices.settlement[day],config.fee,physical)
213     audit=independent_recalculate(q,m,c,b,load,pv,initial,prices.settlement[day],ledger,conf)
214     ↪ ig.fee,physical)
215     if not audit['passed']:raise AssertionError(audit)

```

```

215     if not config.amend and np.max(np.abs(m-q))>1e-9:raise AssertionError('Information-only
    ↪   changed contract')
216     return ledger,dict(steps=steps,amendments=amendments,terminal18=refresh,
217         proposed_c=pc,proposed_b=pb,guard_count=guards,guard_energy_kwh=guard_energy,
218         independent_recalculation=audit,execution_seconds=perf_counter()-start,
219         current_effective_contracts_only_in_control=True,
220         future_candidate_m_is_not_assumed_committed=True)
221

```

model/settlement.py: 物理结算及独立核算

```

1  """Physical settlement of already chosen Q2 actions, in kWh per interval."""
2  from __future__ import annotations
3
4  from typing import Any
5
6  import numpy as np
7
8  LEDGER_FIELDS = (
9      "q", "a", "u", "v", "w", "c", "b", "e", "soc_start", "soc_end",
10     "plan_cost", "emergency_cost", "cash_cost",
11 )
12
13 def _prepare_inputs(q, charge, discharge, load, pv, initial_soc, price,
14     eta_charge, eta_discharge, soc_min, soc_max,
15     bus_limit, emergency_multiplier, tolerance):
16     arrays = {}
17     for name, value in (("q", q), ("c", charge), ("b", discharge),
18         ("load", load), ("pv", pv), ("price", price)):
19         array = np.array(value, dtype=float, copy=True)
20         if array.ndim != 1 or array.size == 0:
21             raise ValueError(f"{name} must be a nonempty one-dimensional array")
22         if not np.isfinite(array).all():
23             raise ValueError(f"{name} contains nonfinite values")
24         arrays[name] = array
25     if len({len(array) for array in arrays.values()}) != 1:
26         raise ValueError("All input vectors must have the same length")
27     scalars = {
28         "initial_soc": initial_soc, "eta_charge": eta_charge,
29         "eta_discharge": eta_discharge, "soc_min": soc_min,
30         "soc_max": soc_max, "bus_limit": bus_limit,
31         "emergency_multiplier": emergency_multiplier, "tolerance": tolerance,
32     }
33     for name, value in scalars.items():
34         if not np.isscalar(value) or not np.isfinite(float(value)):
35             raise ValueError(f"{name} must be a finite scalar")
36     if not (0 < eta_charge <= 1 and 0 < eta_discharge <= 1):
37         raise ValueError("Charge and discharge efficiencies must lie in (0, 1]")
38     if not 0 <= soc_min < soc_max:
39         raise ValueError("Require 0 <= soc_min < soc_max")
40     if bus_limit <= 0 or emergency_multiplier <= 0 or tolerance <= 0:
41         raise ValueError("bus_limit, emergency_multiplier and tolerance must be positive")
42     return arrays
43
44 def _reject(mask, message, values=None):
45     positions = np.flatnonzero(mask)
46     if positions.size:
47         indices = positions[:8].tolist()
48         detail = "" if values is None else f";
    ↪   values={np.asarray(values)[positions[:8]].tolist()}"
49         raise ValueError(f"{message}; interval indices={indices}{detail}")
50
51 def settle_schedule(q, charge, discharge, load, pv, initial_soc, price, *,

```

```

52         eta_charge=0.9, eta_discharge=0.9, soc_min=1200.,
53         soc_max=10800., bus_limit=5000/6,
54         emergency_multiplier=5., enforce_m0=False, tolerance=1e-6):
55     """Settle a fixed schedule without modifying any submitted action."""
56     z = _prepare_inputs(q, charge, discharge, load, pv, initial_soc, price,
57         eta_charge, eta_discharge, soc_min, soc_max,
58         bus_limit, emergency_multiplier, tolerance)
59     for name, values in z.items():
60         _reject(values < -tolerance, f"Negative {name}", values)
61     q, c, b, load, pv, price = (z[k] for k in ("q", "c", "b", "load", "pv", "price"))
62     if not soc_min - tolerance <= initial_soc <= soc_max + tolerance:
63         raise ValueError(f"Initial SOC {initial_soc} outside [{soc_min}, {soc_max}]")
64     _reject(c > bus_limit + tolerance, "Charge bus power/energy limit exceeded", c)
65     _reject(b > bus_limit + tolerance, "Discharge bus power/energy limit exceeded", b)
66     _reject(np.minimum(c, b) > tolerance, "Simultaneous charging and discharging",
67         ↪ np.minimum(c, b))
68     soc_end = float(initial_soc) + np.cumsum(eta_charge*c - b/eta_discharge)
69     soc_start = np.r_[float(initial_soc), soc_end[:-1]]
70     _reject(soc_end < soc_min - tolerance, "SOC below lower bound", soc_end)
71     _reject(soc_end > soc_max + tolerance, "SOC above upper bound", soc_end)
72
73     requirement = load + c - b
74     _reject(requirement < -tolerance,
75         ↪ "A = load + charge - discharge is negative; battery export/dumping is forbidden",
76         requirement)
77     v = np.minimum(pv, requirement)
78     unserved_after_pv = requirement - v
79     a = np.minimum(q, unserved_after_pv)
80     e = unserved_after_pv - a
81     u = q - a
82     w = pv - v
83     m0_product = e*c
84     m0_mask = (e > tolerance) & (c > tolerance)
85     if enforce_m0:
86         _reject(m0_product > tolerance, "M0 forbids emergency energy while charging (e*c >
87             ↪ tolerance)", m0_product)
88     plan_cost = price*q
89     emergency_cost = emergency_multiplier*price*e
90     metadata = {
91         "energy_unit": "kWh_per_interval", "soc_unit": "internal_kWh", "cost_unit": "yuan",
92         "settlement_interpretation": "M0_enforced" if enforce_m0 else
93         ↪ "approved_v2_general_bus",
94         "m0_enforced": bool(enforce_m0),
95         "emergency_charge_overlap_periods": int(np.count_nonzero(m0_mask)),
96         "emergency_charge_overlap_fraction": float(np.mean(m0_mask)),
97         "overlap_definition": "e > tolerance and c > tolerance",
98         "max_charge_discharge_product_kwh2": float(np.max(c*b)),
99         "m0_max_e_times_c_kwh2": float(np.max(m0_product)),
100        "emergency_kwh_during_charging": float(np.sum(e[m0_mask])),
101        "charging_kwh_with_emergency": float(np.sum(c[m0_mask])),
102        "action_modification_count": 0,
103        "action_clipping_count": 0,
104        "signed_roundoff_values_retained": int(sum(np.count_nonzero(vv < 0) for vv in
105            ↪ z.values()))),
106        "tolerance": float(tolerance),
107        "n_periods": len(q),
108    }
109
110     return dict(q=q, a=a, u=u, v=v, w=w, c=c, b=b, e=e,
111         soc_start=soc_start, soc_end=soc_end,
112         plan_cost=plan_cost, emergency_cost=emergency_cost,
113         cash_cost=plan_cost+emergency_cost, metadata=metadata)
114
115 def independent_recalculator(q, charge, discharge, load, pv, initial_soc, price,
116     ledger, *, eta_charge=0.9, eta_discharge=0.9,
117     soc_min=1200., soc_max=10800., bus_limit=5000/6,

```

```

113             emergency_multiplier=5., enforce_m0=False,
114             tolerance=1e-6) -> dict[str, Any]:
115     """Independently replay a ledger using scalar allocation and SOC updates."""
116     inputs = _prepare_inputs(q, charge, discharge, load, pv, initial_soc, price,
117                             eta_charge, eta_discharge, soc_min, soc_max,
118                             bus_limit, emergency_multiplier, tolerance)
119     n = len(inputs["q"])
120     expected = {field: [] for field in LEDGER_FIELDS}
121     violations: dict[str, int] = {}
122
123     def mark(name, condition):
124         if condition:
125             violations[name] = violations.get(name, 0) + 1
126
127     state = float(initial_soc)
128     mark("initial_soc_out_of_bounds", state < soc_min-tolerance or state > soc_max+tolerance)
129     m0_count = 0
130     m0_max = 0.
131     for t in range(n):
132         qt, ct, bt, lt, vt, pt = (float(inputs[k][t]) for k in ("q", "c", "b", "load", "pv",
133             ↪ "price"))
134         for key, value in (("q", qt), ("c", ct), ("b", bt), ("load", lt), ("pv", vt),
135             ↪ ("price", pt)):
136             mark("negative_"+key, value < -tolerance)
137             mark("charge_limit", ct > bus_limit+tolerance)
138             mark("discharge_limit", bt > bus_limit+tolerance)
139             mark("charge_discharge_overlap", ct > tolerance and bt > tolerance)
140             before = state
141             state += float(eta_charge)*ct
142             state -= bt/float(eta_discharge)
143             mark("soc_bounds", state < soc_min-tolerance or state > soc_max+tolerance)
144             required = lt + ct - bt
145             mark("battery_export_or_dump", required < -tolerance)
146             # Source-by-source scalar allocation, independently from the vector
147             # min/residual expressions used by the production settlement.
148             if vt >= required:
149                 pv_used = required
150                 pv_spill = vt-required
151                 outstanding = 0.
152             else:
153                 pv_used = vt
154                 pv_spill = 0.
155                 outstanding = required-vt
156             if qt >= outstanding:
157                 called = outstanding
158                 uncalled = qt-outstanding
159                 emergency = 0.
160             else:
161                 called = qt
162                 uncalled = 0.
163                 emergency = outstanding-qt
164             emergency_charge_product = emergency*ct
165             m0_max = max(m0_max, emergency_charge_product)
166             if emergency_charge_product > tolerance:
167                 m0_count += 1
168                 mark("M0_emergency_charging", enforce_m0)
169             purchase_fee = qt*pt
170             emergency_fee = emergency*pt*float(emergency_multiplier)
171             values = (qt, called, uncalled, pv_used, pv_spill, ct, bt, emergency,
172                 ↪ before, state, purchase_fee, emergency_fee, purchase_fee+emergency_fee)
173             for field, value in zip(LEDGER_FIELDS, values):
174                 expected[field].append(value)
175
176     errors = {}
177     reported = {}

```

```

176     for field in LEDGER_FIELDS:
177         if field not in ledger:
178             mark("missing_ledger_field_"+field, True)
179             errors[field] = None
180             continue
181         value = np.asarray(ledger[field], dtype=float)
182         if value.shape != (n,) or not np.isfinite(value).all():
183             mark("invalid_ledger_field_"+field, True)
184             errors[field] = None
185             continue
186         reported[field] = value
187         error = float(np.max(np.abs(value-np.asarray(expected[field]))))
188         errors[field] = error
189         mark("mismatch_"+field, error > tolerance)
190
191     identities = {}
192     if len(reported) == len(LEDGER_FIELDS):
193         for name, residue in {
194             "bus_balance_kwh": reported["a"]+reported["e"]+reported["v"]+reported["b"]-input_
195                 ↪ s["load"]-reported["c"],
196             "plan_split_kwh": reported["q"]-reported["a"]-reported["u"],
197             "pv_split_kwh": inputs["pv"]-reported["v"]-reported["w"],
198             "soc_recursion_kwh": reported["soc_end"]-reported["soc_start"]-eta_charge*report_
199                 ↪ ed["c"]+reported["b"]/eta_discharge,
200             "cash_sum_yuan":
201                 ↪ reported["cash_cost"]-reported["plan_cost"]-reported["emergency_cost"],
202             "soc_continuity_kwh": np.r_[reported["soc_start"][0]-initial_soc,
203                 reported["soc_start"][1:]-reported["soc_end"][:-1]],
204         }.items():
205             identities[name] = float(np.max(np.abs(residue)))
206             mark("identity_"+name, identities[name] > tolerance)
207     return {
208         "passed": not violations, "n_periods": n, "tolerance": float(tolerance),
209         "violations": violations, "max_error_by_field": errors,
210         "max_identity_error": identities,
211         "max_absolute_error": max([0.]+[v for v in errors.values() if v is not
212             ↪ None]+list(identities.values()))),
213         "m0_enforced": bool(enforce_m0), "m0_violating_periods": m0_count,
214         "m0_violating_fraction": m0_count/n, "m0_max_e_times_c_kwh2": m0_max,
215         "totals": {name: float(sum(expected[name])) for name in
216             ↪ ("q", "a", "u", "v", "w", "c", "b", "e", "plan_cost", "emergency_cost",
217                 ↪ "cash_cost")},
218         "final_soc_kwh": state,
219     }
220 }

```

model/contract_cash.py: 最终合同结算

```

1  """Settle final effective contracts, keeping original q out of physical flow."""
2  import numpy as np
3
4  from model.planning import Physical
5  from model.settlement import settle_schedule
6  from model.contracts import contract_cost
7
8  LEDGER_FIELDS = ('q', 'm', 'a', 'u', 'v', 'w', 'c', 'b', 'e', 'soc_start', 'soc_end',
9                  'plan_cost', 'adjustment_cost', 'contract_cost', 'emergency_cost', 'cash_cost')
10
11 def settle(q, m, c, b, load, pv, s0, price, fee='B', physical=Physical()):
12     original=np.asarray(q, float)
13     z=settle_schedule(m, c, b, load, pv, s0, price, eta_charge=physical.eta_c,
14                     eta_discharge=physical.eta_d, soc_min=physical.s_min, soc_max=physical.s_max,
15                     bus_limit=physical.power_energy, emergency_multiplier=physical.emergency_multiple,
16                     enforce_m0=False)

```

```

17     if original.shape!=z['q'].shape or not np.isfinite(original).all() or original.min() <
    ↪ -1e-7:
18         raise ValueError('Invalid original contract')
19     z['m']=z.pop('q');z['q']=original.copy()
20     z['plan_cost']=np.asarray(price)*original
21     z['contract_cost']=contract_cost(original,z['m'],price,fee)
22     z['adjustment_cost']=z['contract_cost']-z['plan_cost']
23     z['cash_cost']=z['contract_cost']+z['emergency_cost']
24     z['metadata'].update(fee_interpretation=fee,physical_contract='final effective m',
25         original_q_charged_once=True,terminal_value_in_actual_cash=False,
26         unused_definition='u=m-a',candidate_contracts_charged=False)
27     return z
28
29 def independent_recalculate(q,m,c,b,load,pv,s0,price,ledger,fee='B',physical=Physical()):
30     """Scalar branch/flow reconstruction, without calling either fee/settler."""
31     arrays=[np.asarray(a,float) for a in (q,m,c,b,load,pv,price)]
32     if any(a.ndim!=1 or len(a)!=len(arrays[0]) or not np.isfinite(a).all() for a in arrays):
33         raise ValueError('Invalid audit arrays')
34     if fee not in ('A','B'):raise ValueError('Invalid fee')
35     rebuilt={k:[] for k in LEDGER_FIELDS};stock=float(s0);violations=[]
36     for t,(qt,mt,ct,bt,lt,vt,pt) in enumerate(zip(*arrays)):
37         need=lt+ct-bt;used_pv=min(vt,need);remaining=need-used_pv
38         called=min(mt,remaining);emergency=remaining-called
39         if mt>=qt:contract=pt*qt+1.5*pt*(mt-qt)
40         elif fee=='B':contract=pt*mt+.5*pt*(qt-mt)
41         else:contract=pt*qt+.5*pt*(qt-mt)
42         end=stock+physical.eta_c*ct-bt/physical.eta_d
43         vals=dict(q=qt,m=mt,a=called,u=mt-called,v=used_pv,w=vt-used_pv,
44             c=ct,b=bt,e=emergency,soc_start=stock,soc_end=end,
45             plan_cost=pt*qt,adjustment_cost=contract-pt*qt,contract_cost=contract,
46             emergency_cost=physical.emergency_multiple*pt*emergency,
47             cash_cost=contract+physical.emergency_multiple*pt*emergency)
48         for k,x in vals.items():rebuilt[k].append(x)
49         if (min(qt,mt,ct,bt,lt,vt,need,called,emergency)<-1e-6
50             or min(ct,bt)>1e-6 or max(ct,bt)>physical.power_energy+1e-6
51             or min(stock,end)<physical.s_min-1e-6 or max(stock,end)>physical.s_max+1e-6
52             or abs(used_pv+called+emergency+bt-lt-ct)>1e-6):
53             violations.append(t)
54         stock=end
55     errors={k:float(np.max(np.abs(np.asarray(v)-ledger[k]))) for k,v in rebuilt.items()}
56     return dict(passed=not violations and max(errors.values())<1e-6,
57         max_abs_error_by_field=errors,violating_slots=violations,
58         method='independent scalar piecewise fee and physical flow',periods=len(arrays[0]))

```

model/prices.py: 日际电价读取

```

1  """Immutable attachment-4 prices and origin-specific available price profiles."""
2  from dataclasses import dataclass
3  from datetime import datetime
4  from pathlib import Path
5
6  import numpy as np
7  from openpyxl import load_workbook
8
9  MODES = ('known_day_ahead', 'previous_day_forecast')
10
11 @dataclass(frozen=True)
12 class PriceDay:
13     day: int
14     mode: str
15     source_day: int
16     decision: np.ndarray
17

```

```

18     def __post_init__(self):
19         p = np.asarray(self.decision, float).copy()
20         if p.shape != (144,) or not np.isfinite(p).all() or p.min() <= 0:
21             raise ValueError('Decision prices must be positive finite 144-slot values')
22         if self.mode not in MODES or self.source_day != self.day - int(self.mode ==
23             ↪ 'previous_day_forecast'):
24             raise ValueError('Price origin violates the declared information mode')
25         p.setflags(write=False)
26         object.__setattr__(self, 'decision', p)
27
28     @property
29     def next_day_forecast(self):
30         # A declared persistence forecast, never the next observed price row.
31         return self.decision
32
33     def metadata(self):
34         return dict(mode=self.mode, origin_day=self.day,
35             ↪ decision_price_source_day=self.source_day,
36                 nextday_price_source_day=self.source_day,
37                 nextday_method='persistence of the current origin decision-price profile',
38                 current_full_price_profile_known_day_ahead_assumed=self.mode ==
39             ↪ 'known_day_ahead',
40                 price_profile_frozen_for_intraday_control=True,
41                 actual_nextday_price_not_read=True, price_unit='yuan/kWh')
42
43 @dataclass(frozen=True)
44 class PriceArchive:
45     actual: np.ndarray
46     dates: tuple
47     source: str
48
49     def __post_init__(self):
50         p = np.asarray(self.actual, float).copy()
51         if p.shape != (365, 144) or not np.isfinite(p).all() or p.min() <= 0 or
52             ↪ len(self.dates) != 365:
53             raise ValueError('Attachment-4 prices must be positive finite 365 x 144')
54         p.setflags(write=False)
55         object.__setattr__(self, 'actual', p)
56
57     def at_origin(self, day, mode):
58         if mode not in MODES or not 0 <= day < 365:
59             raise ValueError('Invalid origin/mode')
60         source_day = day - int(mode == 'previous_day_forecast')
61         if source_day < 0:
62             raise ValueError('Previous-day price is unavailable on source-year day zero')
63         return PriceDay(day, mode, source_day, self.actual[source_day])
64
65     def settle_price(self, day):
66         return self.actual[day]
67
68 def load_prices(root, expected_dates):
69     path = Path(root) / 'problem/attachments/C 题/附件 4.xlsx'
70     book = load_workbook(path, read_only=True, data_only=True)
71     try:
72         if book.sheetnames != ['Sheet1']:
73             raise ValueError('Unexpected attachment-4 sheets')
74         rows = list(book['Sheet1'].iter_rows(values_only=True))
75     finally:
76         book.close()
77     if len(rows) != 366 or any(len(row) != 145 for row in rows):
78         raise ValueError('Unexpected attachment-4 date/slot shape')
79     dates = tuple(row[0].date() if isinstance(row[0], datetime) else row[0] for row in
80         ↪ rows[1:])
81     if dates != tuple(expected_dates):
82         raise ValueError('Attachment-4 dates differ from attachment-2 actual rows')

```

```

78 # Check every interval-end header against attachment 2; never shift positions.
79 book = load_workbook(Path(root) / 'problem/attachments/C题/附件2.xlsx', read_only=True,
    ↪ data_only=True)
80 try:
81     actual_header = next(book.worksheets[0].iter_rows(min_row=1, max_row=1,
    ↪ values_only=True))
82 finally:
83     book.close()
84 if rows[0][1:] != actual_header[1:]:
85     raise ValueError('Attachment-4 time labels are not position-aligned with actuals')
86 return PriceArchive(np.asarray([row[1:] for row in rows[1:]], float), dates, str(path))

```

model/dynamic_prices.py: 4-2 变价执行

```

1  """Price-only adapter; every planning/control decision delegates to Q2 F-fast."""
2  from dataclasses import replace
3
4  import numpy as np
5
6  from model.planning import Physical
7  from model.control import PRIMARY_CONFIGS, make_plan as q2_make_plan, execute_day as
    ↪ q2_execute_day
8  from model.settlement import LEDGER_FIELDS, settle_schedule, independent_recalculator
9
10 CONFIG = PRIMARY_CONFIGS['F_fast']
11 PHYSICAL_FIELDS = tuple(key for key in LEDGER_FIELDS if not key.endswith('cost'))
12
13 class DecisionArchive:
14     """The Q2 data contract with only its deterministic price vector replaced."""
15     def __init__(self, archive, price_day):
16         self.data = replace(archive.data, price=price_day.decision)
17         self.archive = archive
18         self.price_day = price_day
19
20     def point(self, *args, **kwargs):
21         return self.archive.point(*args, **kwargs)
22
23     def scenarios(self, *args, **kwargs):
24         return self.archive.scenarios(*args, **kwargs)
25
26     def conditional(self, *args, **kwargs):
27         return self.archive.conditional(*args, **kwargs)
28
29 def make_plan(day, stock, archive, price_day, physical=Physical()):
30     if price_day.day != day:
31         raise ValueError('Price forecast is for another decision origin')
32     view = DecisionArchive(archive, price_day)
33     plan, meta = q2_make_plan(day, stock, view, CONFIG, physical)
34     meta.update(price_information=price_day.metadata(),
35               frozen_base_model='src.q2_v4.runtime.PRIMARY_CONFIGS[F_fast]',
36               q4_changes='price profile only; forecast/controller/physical/terminal rules
    ↪ unchanged')
37     meta['continuation'].update(price_day.metadata())
38     return plan, meta, view
39
40 def execute_actions(day, stock, plan, decision_archive, physical=Physical()):
41     """Execute Q2 F-fast under the decision price; no realized price is supplied."""
42     return q2_execute_day(day, stock, plan, decision_archive, CONFIG, physical)
43
44 def settle_actual(plan, decision_ledger, load, pv, initial_soc, actual_price,
    ↪ physical=Physical()):
45     """Independent actual-price cash, with exactly the already-fixed actions."""
46     args = dict(eta_charge=physical.eta_c, eta_discharge=physical.eta_d,

```

```

47     soc_min=physical.s_min, soc_max=physical.s_max, bus_limit=physical.power_energy,
48     emergency_multiplier=physical.emergency_multiple, enforce_m0=False)
49 ledger = settle_schedule(plan.q, decision_ledger['c'], decision_ledger['b'], load, pv,
50     initial_soc, actual_price, **args)
51 changes = {key: float(np.max(np.abs(ledger[key] - decision_ledger[key]))) for key in
    ↳ PHYSICAL_FIELDS}
52 if max(changes.values()) != 0.:
53     raise AssertionError('Settlement changed physical actions/allocations', changes))
54 audit = independent_recalculator(plan.q, ledger['c'], ledger['b'], load, pv,
55     initial_soc, actual_price, ledger, **args)
56 if not audit['passed']:
57     raise AssertionError(audit)
58 audit.update(physical_difference_vs_decision_price=changes,
59     realized_price_not_supplied_to_action_function=True)
60 return ledger, audit

```

model/amendment_prices.py: 4-3 变价接口

```

1 """Adapt the existing attachment-4 origin views to the unchanged Q3 engine."""
2 from __future__ import annotations
3
4 import numpy as np
5
6 from model.amendments import PriceInputs
7 from model.prices import MODES, PriceArchive, load_prices
8
9 def availability_rules(mode: str) -> dict:
10     if mode not in MODES:
11         raise ValueError('Unknown attachment-4 information mode')
12     return dict(
13         mode=mode,
14         decision_source='actual[d]' if mode == 'known_day_ahead' else 'actual[d-1]',
15         decision_classification=('Assumed available in full at day d 00:00'
16             if mode == 'known_day_ahead' else 'Derived previous-day persistence forecast'),
17         settlement_source='Observed attachment-4 actual[d]; used only for ex-post
    ↳ settlement',
18         nextday_source='the decision profile frozen at origin d; never actual[d+1]',
19         nextday_classification='Assumed persistence forecast',
20         price_profile_frozen_for_intraday_control=True,
21         current_full_price_profile_known_day_ahead_assumed=mode == 'known_day_ahead',
22         actual_nextday_price_not_read=True,
23         day_zero_previous_price='unavailable; attachment-1 tariff is an interface placeholder
    ↳ only',
24         day_zero_is_executable=False,
25         evaluation_day_indices=[31, 364],
26         initial_soc_source='unchanged common Q2 F-fast January warmup; not rerun with
    ↳ attachment 4',
27         price_unit='yuan/kWh',
28     )
29
30 def build_price_inputs(data, price_archive: PriceArchive, mode: str) -> PriceInputs:
31     """Create three immutable 365 x 144 matrices for the existing Q3 API."""
32     availability_rules(mode)
33     if tuple(data.dates) != tuple(price_archive.dates):
34         raise ValueError('Price dates do not align with load/PV observations')
35     decision = np.empty((365, 144), dtype=float)
36     for day in range(365):
37         if day == 0 and mode == 'previous_day_forecast':
38             # Required array-interface row only. There is no observed day -1,
39             # and neither production nor the simulation helper may execute it.
40             decision[day] = data.price
41         else:
42             decision[day] = price_archive.at_origin(day, mode).decision

```

```

43     provenance = (
44         'Q4-3; Observed attachment-4 actual settlement; '
45         + ('Assumed entire current-day price curve known at 00:00; '
46           if mode == 'known_day_ahead' else 'Derived previous-day decision-price
47           ↪ persistence; ')
48         + 'Assumed tomorrow-price persistence from this origin decision profile; '
49         + 'full daily profile frozen intraday; no actual tomorrow-price access; '
50         + 'day-zero previous-day row is an unexecuted attachment-1 interface placeholder'
51     )
52     return PriceInputs(decision, price_archive.actual, decision, provenance)

```

model/oracle.py: 完全预知下界

```

1  """Full-period perfect-information cash lower bound with common initial SOC."""
2  from __future__ import annotations
3
4  import argparse
5  from dataclasses import asdict
6  from datetime import datetime, timedelta, timezone
7  import json
8  from pathlib import Path
9  from time import perf_counter
10
11 import numpy as np
12 from scipy import sparse
13 from scipy.optimize import linprog
14
15 from model.history import START_DATE, T, load_data
16 from model.planning import Physical
17 from model.settlement import settle_schedule
18
19 ROOT = Path(__file__).resolve().parents[1]
20
21 def formulate_oracle(load, pv, price, initial_soc, physical=Physical()):
22     """Vectorized independent state-increment formulation; no Python row loop."""
23     load, pv, price = (np.asarray(value, float) for value in (load, pv, price))
24     if any(value.ndim != 1 or not len(value) or not np.isfinite(value).all()
25           for value in (load, pv, price)):
26         raise ValueError("Oracle requires finite nonempty one-dimensional inputs")
27     if load.shape != pv.shape or load.shape != price.shape:
28         raise ValueError("Oracle input shapes differ")
29     if np.min(load) < 0 or np.min(pv) < 0 or np.min(price) <= 0:
30         raise ValueError("Oracle requires nonnegative energies and positive prices")
31     if physical.emergency_multiple <= 1:
32         raise ValueError("Eliminating emergency purchases requires a multiplier above one")
33     if not physical.s_min <= initial_soc <= physical.s_max:
34         raise ValueError("Initial SOC outside physical limits")
35     n = len(load)
36     t = np.arange(n)
37     state = n+t
38     successor = state+1
39     rows = np.concatenate([t, t, t, n+t, n+t, n+t,
40                            2*n+t, 2*n+t, 3*n+t, 3*n+t])
41     cols = np.concatenate([t, successor, state, t, successor, state,
42                            successor, state, state, successor])
43     values = np.concatenate([-np.ones(n), np.full(n, 1/physical.eta_c), np.full(n,
44     ↪ -1/physical.eta_c),
45                             -np.ones(n), np.full(n, physical.eta_d), np.full(n,
46     ↪ -physical.eta_d),
47                             np.ones(n), -np.ones(n), np.ones(n), -np.ones(n)])
48     matrix = sparse.coo_matrix((values, (rows, cols)), shape=(4*n, 2*n+1)).tocsr()
49     rhs = np.concatenate([pv-load, pv-load, np.full(n, physical.eta_c*physical.power_energy),
50                           np.minimum(load, physical.power_energy)/physical.eta_d])

```

```

49     lower = np.r_[np.zeros(n), np.full(n+1, physical.s_min)]
50     upper = np.r_[np.full(n, np.inf), np.full(n+1, physical.s_max)]
51     lower[n] = upper[n] = initial_soc
52     return dict(matrix=matrix, rhs=rhs, lower=lower, upper=upper,
53                 objective=np.r_[price, np.zeros(n+1)], n=n)
54
55 def solve_oracle(load, pv, price, initial_soc, *, terminal_soc=None,
56                 physical=Physical(), problem=None, time_limit=None):
57     problem = formulate_oracle(load, pv, price, initial_soc, physical) if problem is None
58     ↪ else problem
59     n = problem["n"]
60     lower, upper = problem["lower"].copy(), problem["upper"].copy()
61     if terminal_soc is not None:
62         if not physical.s_min-1e-6 <= terminal_soc <= physical.s_max+1e-6:
63             raise ValueError("Matched terminal SOC outside physical tolerance")
64         lower[-1] = upper[-1] = float(terminal_soc)
65     options = {} if time_limit is None else {"time_limit": float(time_limit)}
66     start = perf_counter()
67     result = linprog(problem["objective"], A_ub=problem["matrix"], b_ub=problem["rhs"],
68                     bounds=np.c_[lower, upper], method="highs", options=options)
69     elapsed = perf_counter()-start
70     if not result.success:
71         raise RuntimeError(f"Full-period oracle failed: {result.status} {result.message}")
72     stock = result.x[n:]
73     increment = np.diff(stock)
74     charge = np.maximum(increment, 0)/physical.eta_c
75     discharge = np.maximum(-increment, 0)*physical.eta_d
76     q = result.x[:n]
77     ledger = settle_schedule(q, charge, discharge, load, pv, initial_soc, price,
78                             eta_charge=physical.eta_c, eta_discharge=physical.eta_d,
79                             soc_min=physical.s_min, soc_max=physical.s_max,
80                             bus_limit=physical.power_energy,
81                             emergency_multiplier=physical.emergency_multiple,
82                             ↪ enforce_m0=False)
83     cash = float(np.sum(ledger["cash_cost"]))
84     if abs(cash-result.fun) > 1e-5:
85         raise AssertionError("Oracle settlement cash differs from the solver objective")
86     if float(np.max(np.abs(ledger["soc_end"]-stock[1:]))) > 1e-6:
87         raise AssertionError("Oracle state reconstruction differs from solver states")
88     if terminal_soc is not None and abs(float(stock[-1])-terminal_soc) > 1e-6:
89         raise AssertionError("Oracle did not satisfy the matched terminal equality")
90     equality_target_error = None if terminal_soc is None else
91     ↪ abs(float(stock[-1])-terminal_soc)
92     # Full dual reconstruction supplies a separate optimality certificate.
93     dual = float(problem["rhs"] @ result.ineqlin.marginals + lower @ result.lower.marginals)
94     finite = np.isfinite(upper)
95     dual += float(upper[finite] @ result.upper.marginals[finite])
96     primal_violation = float(max(0., np.max(problem["matrix"] @ result.x-problem["rhs"]),
97                                         np.max(lower-result.x), np.max(result.x-upper)))
98     if primal_violation > 1e-6 or abs(float(result.fun)-dual) > max(1e-5,
99     ↪ abs(float(result.fun))*1e-9):
100         raise AssertionError("Oracle primal/dual certificate failed")
101     info = dict(status="OPTIMAL", solver_status=int(result.status),
102     ↪ solver_message=result.message,
103         objective_yuan=float(result.fun), cash_cost_yuan=cash,
104         dual_objective_yuan=dual, primal_dual_gap_yuan=float(result.fun)-dual,
105         max_primal_violation=primal_violation, seconds=elapsed,
106         iterations=int(result.nit), variables=2*n+1, inequalities=4*n,
107         sparse_nonzeros=int(problem["matrix"].nnz),
108         initial_soc_kwh=float(initial_soc), terminal_soc_kwh=float(stock[-1]),
109         requested_terminal_soc_kwh=terminal_soc,
110         ↪ terminal_equality_error_kwh=equality_target_error,
111         terminal_constraint="free within physical bounds" if terminal_soc is None
112         ↪ else "exact full-period equality",
113         emergency_kwh=float(ledger["e"].sum()),

```

```

107         daily_cyclic_constraints=0,
108         e_elimination_proof="With free normal q and positive tariff, replace (q,e) by
        ↪ (q+e,0); cash decreases by (m-1)*p*e. Therefore an optimum has e=0, and
        ↪ M0 holds.")
109     return ledger, info

```

model/explicit_check.py: 独立显式互斥模型

```

1  """Independent explicit charge/discharge MILP; constructed checks are Simulated."""
2
3  import numpy as np
4
5  from scipy import sparse
6
7  from scipy.optimize import Bounds, LinearConstraint, milp
8
9  from model.planning import Physical
10
11 class ExplicitBuilder:
12     def __init__(self):
13         self.cost, self.low, self.high, self.integer = [], [], [], []
14         self.rows, self.cols, self.values, self.rowlow, self.rowhigh = [], [], [], [], []
15
16     def variables(self, shape, *, low=0., high=np.inf, cost=0., integer=0):
17         count = int(np.prod(shape))
18         indices = np.arange(len(self.cost), len(self.cost)+count).reshape(shape)
19         for target, value in ((self.low, low), (self.high, high),
20                               (self.cost, cost), (self.integer, integer)):
21             target.extend(np.broadcast_to(value, shape).ravel().tolist())
22         return indices
23
24     def constraint(self, coefficients, low=-np.inf, high=np.inf):
25         r = len(self.rowlow)
26         for index, value in coefficients:
27             self.rows.append(r)
28             self.cols.append(int(index))
29             self.values.append(float(value))
30             self.rowlow.append(float(low))
31             self.rowhigh.append(float(high))
32
33     def equal(self, coefficients, value=0.):
34         self.constraint(coefficients, low=value, high=value)
35
36     def solve(self):
37         matrix = sparse.coo_matrix((self.values, (self.rows, self.cols)),
38                                     shape=(len(self.rowlow), len(self.cost))).tocsr()
39         answer = milp(np.asarray(self.cost), integrality=np.asarray(self.integer),
40                       bounds=Bounds(np.asarray(self.low), np.asarray(self.high)),
41                       constraints=LinearConstraint(matrix, self.rowlow, self.rowhigh),
42                       options={"mip_rel_gap": 1e-10, "time_limit": 30.})
43         if not answer.success:
44             raise AssertionError(f"Independent explicit MILP failed: {answer.status}:
45             ↪ {answer.message}")
46         constraint_values = matrix@answer.x
47         violation = max(0., float(np.max(np.asarray(self.rowlow)-constraint_values)),
48                        float(np.max(constraint_values-np.asarray(self.rowhigh))),
49                        float(np.max(np.asarray(self.low)-answer.x)),
50                        float(np.max(answer.x-np.asarray(self.high))))
51         return answer, violation
52
53 def explicit_milp(load, pv, price, s0, *, physical, kind="dayahead", original_q=None,
54                  fixed_m=None, fee="B", lam=0., target=None, groups=None, split=0,
55                  allow_amend=True, first_discharge_cap=None,

```

```

55         terminal_equal=False, terminal_floor=None, charge_limit=None,
        ↪ discharge_limit=None):
56     """Explicit physical and contract formulation used only by this validator."""
57     load, pv, price = np.asarray(load), np.asarray(pv), np.asarray(price)
58     k, n = load.shape
59     labels, group_index = np.unique(np.zeros(k, int) if groups is None else groups,
        ↪ return_inverse=True)
60     ng = len(labels)
61     weights = np.bincount(group_index, minlength=ng)/k
62     model = ExplicitBuilder()
63     charge_limit = physical.power_energy if charge_limit is None else charge_limit
64     discharge_limit = physical.power_energy if discharge_limit is None else discharge_limit
65     q = model.variables((n,), cost=price if kind != "fixed" else 0.)
66     m = model.variables((ng, n))
67     charge = model.variables((ng, n), high=charge_limit)
68     discharge_high = np.asarray([np.minimum(discharge_limit, load[group_index ==
        ↪ g].min(axis=0))
69                                     for g in range(ng)])
70     if first_discharge_cap is not None:
71         discharge_high[:, 0] = np.minimum(discharge_high[:, 0], max(0., first_discharge_cap))
72     discharge = model.variables((ng, n), high=discharge_high)
73     state = model.variables((ng, n+1), low=physical.s_min, high=physical.s_max)
74     mode = model.variables((ng, n), high=1., integer=1)
75     called = model.variables((k, n))
76     emergency = model.variables((k, n),
        ↪ cost=np.broadcast_to(physical.emergency_multiple*price/k, (k, n)))
77     curtailed = model.variables((k, n), high=pv)
78     up = down = None
79     if kind in ("amend", "signal"):
80         up = model.variables((ng, n), cost=weights[:, None]*1.5*price)
81         down = model.variables((ng, n), cost=weights[:, None]*(.5 if fee == "A" else
        ↪ -.5)*price)
82         # With up-down=m-q, increasing up and down together costs +2p under A
83         # or +p under B. A minimum therefore cannot include fictitious round trips.
84         for g in range(ng):
85             for t in range(n):
86                 model.equal([(m[g, t], 1.), (q[t], -1.), (up[g, t], -1.), (down[g, t], 1.)])
87     if kind == "amend":
88         for t in range(n):
89             model.equal([(q[t], 1.), original_q[t]])
90     elif kind == "fixed":
91         for t in range(n):
92             model.equal([(q[t], 1.), fixed_m[t]])
93         for g in range(ng):
94             model.equal([(m[g, t], 1.), fixed_m[t]])
95     elif kind == "dayahead":
96         for g in range(ng):
97             for t in range(n):
98                 model.equal([(m[g, t], 1.), (q[t], -1.)])
99     elif kind != "signal":
100         raise ValueError(kind)
101     if kind == "signal":
102         for g in range(ng):
103             for t in range(split if allow_amend else n):
104                 model.equal([(m[g, t], 1.), (q[t], -1.)])
105         if g:
106             # Independent expression of nonanticipativity: actual battery
107             # charge and discharge commands coincide before the signal.
108             for t in range(split):
109                 model.equal([(charge[g, t], 1.), (charge[0, t], -1.)])
110                 model.equal([(discharge[g, t], 1.), (discharge[0, t], -1.)])
111     y = model.variables((ng,), high=physical.s_max if target is None else target,
112                       cost=-lam*weights) if lam else None
113     for g in range(ng):
114         if s0 is not None:

```

```

115         model.equal([(state[g, 0], 1.)], s0)
116     if terminal_equal:
117         model.equal([(state[g, -1], 1.), (state[g, 0], -1.)])
118     if terminal_floor is not None:
119         model.constraint([(state[g, -1], 1.)], low=terminal_floor)
120     for t in range(n):
121         model.equal([(state[g, t+1], 1.), (state[g, t], -1.),
122                     (charge[g, t], -physical.eta_c), (discharge[g, t],
123                      ↪ 1/physical.eta_d)])
124         model.constraint([(charge[g, t], 1.), (mode[g, t], -charge_limit)], high=0.)
125         model.constraint([(discharge[g, t], 1.), (mode[g, t], discharge_limit)],
126                          high=discharge_limit)
127     if y is not None:
128         model.constraint([(y[g], 1.), (state[g, -1], -1.)], high=0.)
129     for s in range(k):
130         g = group_index[s]
131         for t in range(n):
132             model.constraint([(called[s, t], 1.), (m[g, t], -1.)], high=0.)
133             model.equal([(called[s, t], 1.), (emergency[s, t], 1.),
134                         (discharge[g, t], 1.), (charge[g, t], -1.), (curtailed[s, t], -1.)],
135                         load[s, t]-pv[s, t])
136     answer, violation = model.solve()
137     x = answer.x
138     return {
139         "q": x[q], "m_by_group": x[m], "c_by_group": x[charge], "b_by_group": x[discharge],
140         "soc_by_group": x[state], "e": x[emergency], "called": x[called], "curtailed":
141         ↪ x[curtailed],
142         "group_weights": weights, "scenario_groups": group_index,
143         "objective": float(answer.fun), "mip_gap": float(answer.mip_gap),
144         "max_constraint_violation": violation,
145         "up": None if up is None else x[up], "down": None if down is None else x[down],
146     }
147
148 def cases():
149     generated = []
150     for index, (n, k) in enumerate(((4, 2), (5, 3), (6, 4), (4, 4), (5, 2), (6, 3))):
151         rng = np.random.default_rng(81031+index)
152         load = np.round(rng.uniform(.3, 9., (k, n)), 3)
153         pv = np.round(rng.uniform(0., 7., (k, n)), 3)
154         if index in (1, 4):
155             load[0, 0] = 0.
156         price = np.round(rng.uniform(.2, 1.9, n), 3)
157         physical = Physical(eta_c=.84+.02*(index%3), eta_d=.88+.015*(index%2),
158                             s_min=1., s_max=12., power_energy=3.8, emergency_multiple=5.)
159         generated.append({
160             "name": f"constructed_n{n}_k{k}_{index}", "seed": 81031+index,
161             "load": load, "pv": pv, "price": price, "s0": 4.+index,
162             "physical": physical, "original_q": np.round(rng.uniform(0., 8., n), 3),
163             "fixed_m": np.round(rng.uniform(0., 8., n), 3),
164             "lam": (0., .65, 1.8)[index%3], "target": (None, 8.4, 7.5)[index%3],
165             "groups": np.asarray([10+10*(s%min(k, 3)) for s in range(k)]),
166             "split": n//2,
167         })
168     return generated

```

verify.py: 数学与复算核验

```

1 """Re-run mathematical checks and derived diagnostics from the supplied inputs."""
2 import argparse
3 from dataclasses import replace
4 import hashlib
5 import json
6 from pathlib import Path

```

```

7
8 import numpy as np
9 from scipy.optimize import linprog
10 from model import q1, planning, contracts
11 from model.explicit_check import explicit_milp, cases
12 from model.history import load_data, ForecastArchive
13 from model.fusion import FusedForecastArchive as FullWindow
14 from model.remaining import FusedForecastArchive as RemainingWindow
15 from model import amendments
16 from model.prices import load_prices
17 from model.amendment_prices import build_price_inputs
18 from model.oracle import solve_oracle
19 from run import ROOT, save, fingerprints, designs, one_day
20
21
22 def small_models():
23     results = {'classification': 'Simulated small cases; Q1 variants use Observed attachment
↪ 1'}
24     config = json.loads((ROOT/'configs/q1_v2.json').read_text('utf-8'))
25     source, _ = q1.read_inputs(config)
26     q1_results = []
27     names = q1.DEFAULT_CASES +
↪ ['upper_plus_10', 'lower_minus_10', 'charge_power_plus_100', 'discharge_power_plus_100']
28     for name in names:
29         cfg = q1.case_config(config, name)
30         values = q1.interpreted(source, cfg['time_interpretation'])
31         p, l, v = values[:, 0], values[:, 1]*cfg['step_hours'], values[:, 2]*cfg['step_hours']
32         actual = q1.solve(p, l, v, cfg)
33         physical =
↪ planning.Physical(cfg['eta_charge'], cfg['eta_discharge'], cfg['soc_min_kwh'],
34                     cfg['soc_max_kwh'], cfg['max_charge_power_kw']*cfg['step_hours'], 5.)
35         independent =
↪ explicit_milp(l[None, :], v[None, :], p, cfg['initial_soc_kwh'], physical=physical,
36               terminal_equal=True, charge_limit=cfg['max_charge_power_kw']*cfg['step_hours'],
37               discharge_limit=cfg['max_discharge_power_kw']*cfg['step_hours'])
38         gap = abs(actual['primary'].fun-independent['objective'])
39         assert gap < 1e-6, (name, gap)
40         q1_results.append(dict(case=name, lp_cost=float(actual['primary'].fun), milp_gap=gap))
41     results['q1'] = q1_results
42     rng = np.random.default_rng(20260910); constructed = []
43     for index in range(24):
44         n = (4, 8, 16, 24)[index%4]
45         p, l, v = rng.uniform(.1, 2, n), rng.uniform(0, 8, n), rng.uniform(0, 12, n)
46         l[rng.random(n)<.15] = 0.; v[rng.random(n)<.30] = 0.
47         cfg = {**config, 'step_hours': 1., 'soc_min_kwh': 0., 'soc_max_kwh': float(rng.uniform(3, 1
↪ 5)),
48               'max_charge_power_kw': float(rng.uniform(.2, 8)), 'max_discharge_power_kw': float(rn
↪ g.uniform(.2, 8)),
49               'eta_charge': float(rng.uniform(.65, 1)), 'eta_discharge': float(rng.uniform(.65, 1))}
50         cfg['initial_soc_kwh'] = float(rng.uniform(0, cfg['soc_max_kwh']))
51         actual = q1.solve(p, l, v, cfg)
52         physical =
↪ planning.Physical(cfg['eta_charge'], cfg['eta_discharge'], 0., cfg['soc_max_kwh'],
53                     cfg['max_charge_power_kw'], 5.)
54         independent =
↪ explicit_milp(l[None, :], v[None, :], p, cfg['initial_soc_kwh'], physical=physical,
55               terminal_equal=True, charge_limit=cfg['max_charge_power_kw'],
56               discharge_limit=cfg['max_discharge_power_kw'])
57         gap = abs(actual['primary'].fun-independent['objective'])
58         assert gap < 1e-6, (index, gap)
59         constructed.append(dict(case=index, periods=n, objective_gap=gap, classification='Simul
↪ ated'))
60     results['q1_constructed'] = constructed
61     rng = np.random.default_rng(20260911)
62     comparisons = []

```

```

63     for index in range(18):
64         l,v = rng.uniform(0,950,(3,6)),rng.uniform(0,1200,(3,6))
65         p,s0 = rng.uniform(.37,1.4,6),float(rng.uniform(1200,9500))
66         term = ('free','value','cyclic')[index%3]
67         q = rng.uniform(0,300,6) if index%2 else None
68         lam = .47 if term == 'value' else 0.
69         target = s0 if term == 'cyclic' else 8500.
70         actual = planning.solve_state_lp(l,v,p,s0,terminal=term,target=target,lam=lam,
71                                         no_battery_dump=True,fixed_q=q)
72         independent = explicit_milp(l,v,p,s0,physical=planning.Physical(),
73                                     kind='dayahead' if q is None else 'fixed',fixed_m=q,lam=lam,target=target,
74                                     terminal_floor=s0 if term == 'cyclic' else None)
75         gap = abs(actual['objective']-independent['objective']-(0. if q is None else p@q))
76         assert gap < 1e-6,(index,gap)
77         comparisons.append(dict(case=index,objective_gap=float(gap)))
78     results['q2'] = comparisons
79     comparisons = []
80     for case in cases():
81         l,v,p,s0 = [case[k] for k in ('load','pv','price','s0')]
82         common = {k:case[k] for k in ('physical','lam','target')}
83         for kind,fee in (('dayahead','B'),('amend','B'),('amend','A'),('fixed','B')):
84             kwargs = dict(common,fee=fee,first_discharge_cap=.2 if case['seed']%2 == 0 else
85                           ↪ None)
86             if kind == 'amend': kwargs['original_q'] = case['original_q']
87             if kind == 'fixed': kwargs['fixed_m'] = case['fixed_m']
88             actual = contracts.solve_shared(l,v,p,s0,**kwargs)
89             independent = explicit_milp(l,v,p,s0,kind=kind,**kwargs)
90             gap = abs(actual['objective']-independent['objective'])
91             assert gap < 2e-6,(case['name'],kind,gap)
92             comparisons.append(dict(case=case['name'],kind=kind,fee=fee,objective_gap=gap))
93         for fee,allowed in (('B',True),('A',True),('B',False),('A',False)):
94             kwargs = dict(common,groups=case['groups'],split=case['split'],allow_amend=allow
95                           ↪ ed,fee=fee)
96             actual = contracts.solve_signal_recourse(l,v,p,s0,**kwargs)
97             independent = explicit_milp(l,v,p,s0,kind='signal',**kwargs)
98             gap = abs(actual['objective']-independent['objective'])
99             assert gap < 2e-6,(case['name'],'signal',gap)
100             comparisons.append(dict(case=case['name'],kind='signal',fee=fee,amend=allowed,ob
101                                   ↪ jective_gap=gap))
102     results['q3'] = comparisons
103     return results
104
105 def information_boundary():
106     data = load_data(ROOT);results = []
107     for day in (31,100,264):
108         cut = 72
109         load,pv = data.load.copy(),data.pv.copy()
110         load.reshape(-1)[day*144+cut+1:] *= 1.2
111         pv.reshape(-1)[day*144+cut+1:] *= .7
112         altered = replace(data,load=load,pv=pv)
113         for case in ('q2/F_fast','q2/lagged_reference'):
114             kind,config,physical,_ = designs()[case]
115             ledgers = [one_day(kind,day,8657.424357634898,ForecastArchive(value,cold_start='
116                           ↪ scaled'),
117                           config,physical)[0] for value in (data,altered)]
118             errors =
119                 ↪ {key:float(np.max(np.abs(ledgers[0][key][:cut+1]-ledgers[1][key][:cut+1]))
120                       for key in ('q','c','b','e','soc_end'))}
121             assert max(errors.values()) < 1e-6,(day,case,errors)
122             assert np.array_equal(ledgers[0]['q'],ledgers[1]['q'])
123             results.append(dict(day=day,case=case,observed_through_slot=cut,max_errors=error
124                               ↪ s))
125     for hour in (0,6,12,18):
126         load,pv = data.load.copy(),data.pv.copy()

```

```

122         boundary = day*144+hour*6
123         load.reshape(-1)[boundary:] *= 1.2;pv.reshape(-1)[boundary:] *= .7
124         changed = replace(data,load=load,pv=pv)
125         predictions = [RemainingWindow(value,root=ROOT,
126             load_archive=ForecastArchive(value,cold_start='scaled')).point(day,hour)
127             for value in (data,changed)]
128         gap = max(float(np.max(np.abs(predictions[0][k]-predictions[1][k]))) for k in
129             ↪ (0,1))
130         assert gap < 1e-6,(day,hour,gap)
131         results.append(dict(day=day,issue_hour=hour,prediction_error=gap))
132     return dict(classification='Simulated future-data perturbation; fixed valid
133     ↪ state',cases=results)
134
135 def forecast_diagnostics():
136     data = load_data(ROOT)
137     old = FullWindow(data,root=ROOT,load_archive=ForecastArchive(data,cold_start='scaled'))
138     new = RemainingWindow(data,root=ROOT,load_archive=ForecastArchive(data,cold_start='scale
139     ↪ d'))
140     errors = {}
141     for start,stop,period in ((0,31,'January'),(31,365,'February_December')):
142         for day in range(start,stop):
143             for hour in (0,6,12,18):
144                 boundary = day*144+hour*6
145                 _,history,official,_ = new.components(day,hour)
146                 previous = old.point(day,hour)[1]
147                 fused = new.point(day,hour)[1]
148                 for window,length in (('full',144),('remaining',144-hour*6)):
149                     if boundary+length > stop*144: continue
150                     actual = data.pv.reshape(-1)[boundary:boundary+length]
151                     for name,prediction in (('history',history),('official',official),('prev
152                     ↪ ious',previous),('fused',fused)):
153                         error = prediction[:length]-actual
154                         errors.setdefault((period>window,name,'all'),[]).append(error)
155                         errors.setdefault((period>window,name,str(hour)),[]).append(error)
156
157     metrics = {}
158     for keys,arrays in errors.items():
159         values = np.concatenate(arrays)
160         metrics['/'.join(keys)] = dict(targets=len(values),mae=float(np.abs(values).mean()),
161             bias=float(values.mean()),p90=float(np.quantile(np.abs(values),.9)))
162     weights = {str(hour):new.weight_at(31,hour)['official_weight'] for hour in (0,6,12,18)}
163     return dict(weights=weights,metrics=metrics,classification='Derived; January numerical
164     ↪ fitting only')
165
166 def optimal_face_and_marginals(output):
167     config = json.loads((ROOT/'configs/q1_v2.json').read_text('utf-8'))
168     data,_ = q1.read_inputs(config)
169     p,l,v = data[:,0],data[:,1]/6,data[:,2]/6
170     result = q1.solve(p,l,v,config)
171     model,answer = result['model'],result['primary']
172     ranges = [];widest = (0.,None,None)
173     for index in list(range(len(answer.x))+[None]):
174         values = [];vectors = []
175         for sign in (1.,-1.):
176             objective = np.zeros(len(answer.x))
177             if index is None: objective[:len(p)] = sign
178             else: objective[index] = sign
179             test = linprog(objective,A_ub=model['aub'],b_ub=model['bub'],
180                 A_eq=np.vstack((model['aeq'].toarray(),model['c'])),
181                 b_eq=np.r_[model['beq'],answer.fun],bounds=np.c_[model['lower'],model['upper']
182                 ↪ '']],method='highs')
183             assert test.success
184             values.append(float(objective@test.x/sign));vectors.append(test.x)
185     width = values[1]-values[0]

```

```

181         if width > 1e-6: ranges.append(dict(index=index,minimum=values[0],maximum=values[1]))
182         if index is not None and index < len(p) and width > widest[0]:
183             widest = (width,index,vectors)
184     witnesses = {}
185     for label,vector in zip(('minimum','maximum'),widest[2]):
186         solution = q1.recover(vector,len(p),l,v,config)
187         assert abs(p@solution['g']-answer.fun) < 1e-6
188         witnesses.update({f'{label}_{k}':value for k,value in solution.items()})
189     folder = output/'checks';folder.mkdir(parents=True,exist_ok=True)
190     np.savez_compressed(folder/'q1_face_witnesses.npz',**witnesses)
191     n = len(p);step = config['step_hours']
192     yb,yl,yu = answer.ineqlin.marginals,answer.lower.marginals,answer.upper.marginals
193     limit = config['max_discharge_power_kw']*step
194     changed = np.minimum(limit+100*step,1)-np.minimum(limit,1)
195     predictions = [(('upper_plus_10',10*yu[n+1:-1].sum()),
196                     ('lower_minus_10',-10*yl[n+1:-1].sum()),
197                     ('charge_power_plus_100',100*step*config['eta_charge']*yb[2*n:3*n].sum()),
198                     ('discharge_power_plus_100',yb[3*n:4*n]@(changed/config['eta_discharge'])))
199     margins = []
200     for case,prediction in predictions:
201         altered = q1.solve(p,l,v,q1.case_config(config,case))['primary'].fun-answer.fun
202         assert abs(altered-prediction) < 1e-6,(case,altered,prediction)
203         margins.append(dict(case=case,predicted_change=float(prediction),actual_change=float(
204             ↪ (altered),
205             absolute_difference=float(abs(altered-prediction))))
206     return dict(variable_count=len(answer.x),nonunique_coordinates=ranges,
207                 coordinate_minimizations=2*len(answer.x),total_purchase_minimizations=2,
208                 witness_coordinate=widest[1],witness_width=widest[0],
209                 primary_cost=float(answer.fun),marginal_checks=margins)
210
211 def quantile_replay(output,name):
212     data = load_data(ROOT)
213     archive = RemainingWindow(data,root=ROOT,load_archive=ForecastArchive(data,cold_start='s'
214     ↪ caled'))
215     config = designs()['q3/Z2_111'][1]
216     prices = amendments.PriceInputs.q3(data)
217     if name.startswith('q4_3/'):
218         selected = json.loads((output/'q3/selection.json').read_text('utf-8'))['selected']
219         config = designs()[selected][1]
220         prices = build_price_inputs(data,load_prices(ROOT,data.dates),name.split('/')[1])
221     with np.load(output/name/'ledger.npz') as z: ledger = {k:z[k].copy() for k in z.files}
222     maximum_violation = 0.;max_contract_error = 0.;count = positions = committed = 0
223     physical = planning.Physical()
224     fixed_action_bound = fixed_action_change = 0.;max_stock_roundoff = 0.
225     for index,day in enumerate(ledger['d']):
226         plan =
227         ↪ amendments.make_plan(int(day),ledger['soc_start'][index,0],archive,prices,config)
228         assert np.max(np.abs(plan.q-ledger['q'][index])) < 1e-6
229         # Rebuild the controller's sequential arithmetic. The settled cumsum
230         # can differ by 1e-11 kWh and select another degenerate LP vertex.
231         runtime_stock = np.empty(144);stock = float(ledger['soc_start'][index,0])
232         for slot in range(144):
233             runtime_stock[slot] = stock
234             stock +=
235             ↪ physical.eta_c*ledger['c'][index,slot]-ledger['b'][index,slot]/physical.eta_d
236         max_stock_roundoff = max(max_stock_roundoff,float(np.max(np.abs(runtime_stock-ledger[
237             ↪ ['soc_start'][index]))))
238         for hour in config.enabled_hours:
239             t = hour*6
240             stop = amendments.effective_blocks(config.enabled_hours)[t]
241             l,v,_ = archive.scenarios(int(day),hour,config.scenario_count)
242             p = prices.decision[day,t:]
243             old_q = plan.q[t:]
244             result = contracts.solve_shared(l[:, :144-t],v[:, :144-t],p,runtime_stock[t],

```

```

241         original_q=old_q,fee='B',lam=plan.lam,target=plan.target)
242     m = result['m'];net = l[:, :144-t]-v[:, :144-t]+result['c']-result['b']
243     # Left and right subgradients bracket zero; m=0 has only a right condition.
244     eps = 1e-6
245     left_contract = np.where(m <= old_q+eps,.5,1.5)*p
246     right_contract = np.where(m >= old_q-eps,1.5,.5)*p
247     left = left_contract-5*p*(net >= m[None,:]-eps).mean(axis=0)
248     right = right_contract-5*p*(net > m[None,:]+eps).mean(axis=0)
249     violation = max(float(np.max(left[m>eps],initial=0.)),float(np.max(-right,initial=0.)),0.)
250     maximum_violation = max(maximum_violation,violation)
251     error = float(np.max(np.abs(m[:stop-t]-ledger['m'][index,t:stop])))
252     max_contract_error = max(max_contract_error,error)
253     assert violation < 1e-6 and error < 1e-6,(name,int(day),hour,violation,error)
254     count += 1;positions += len(m);committed += stop-t
255     forecast = archive.point(int(day),18)[1][:36]
256     actual = data.pv[day,108:]
257     need = data.load[day,108:]+ledger['c'][index,108:]-ledger['b'][index,108:]-ledger['m']
258     ↪ '[index,108:]
259     tariff = 5*prices.settlement[day,108:]
260     fixed_action_bound += float(tariff@np.abs(forecast-actual))
261     fixed_action_change +=
262     ↪ float(tariff@(np.maximum(need-forecast,0)-np.maximum(need-actual,0)))
263     for owner in (archive,archive.load_archive):
264         for key in ('_errors','_residuals','_conditional_bases'):
265             cache = getattr(owner,key,None)
266             if isinstance(cache,dict): cache.clear()
267     assert max_stock_roundoff < 1e-6
268     return dict(days=len(ledger['d']),amendment_lps=count>window_positions=positions,
269     committed_positions=committed,max_subgradient_violation=maximum_violation,
270     max_committed_contract_error=max_contract_error,fixed_action_bound=fixed_action_bound,
271     ↪ d,
272     fixed_action_cash_change=fixed_action_change,max_runtime_vs_settlement_stock_roundoff=
273     ↪ f=max_stock_roundoff,
274     state_reconstruction='Sequential arithmetic of execute_day',passed=True)
275
276 def oracle_and_shadow(output):
277     data = load_data(ROOT)
278     with np.load(output/'q2/F_fast/ledger.npz') as z: ledger = {k:z[k].copy() for k in
279     ↪ z.files}
280     price = np.tile(data.price,len(ledger['d']))
281     entry = float(ledger['soc_start'][0,0])
282     terminal = float(ledger['soc_end'][-1,-1])
283     result = {}
284     for label,target in (('free',None),('matched',terminal)):
285         _,info = solve_oracle(ledger['load'].ravel(),ledger['pv'].ravel(),price,entry,terminal,
286         ↪ al_soc=target)
287         result[label] = info
288     archive = ForecastArchive(data,cold_start='scaled')
289     rows = []
290     for day in (31,100,264):
291         initial = float(ledger['soc_start'][np.flatnonzero(ledger['d'] == day)[0],0])
292         blocks = [planning.Block(*archive.point(day,target),data.price) for target in
293         ↪ (day,day+1)]
294         baseline = planning.solve(blocks,initial,terminal='cyclic',no_battery_dump=True,retail=
295         ↪ in_problem=True)
296         f = baseline.formulation;values = []
297         for change in (.1,-.1):
298             rhs = f['rhs'].copy();rhs[f['link_rows'][1]] += change
299             fit = linprog(f['objective'],A_ub=f['ub'],b_ub=f['ub_rhs'],A_eq=f['eq'],b_eq=rhs,
300             bounds=np.c_[f['lower'],f['upper']],method='highs')
301             assert fit.success
302             values.append((baseline.objective-fit.fun)/change)

```

```

296         assert max(abs(x-baseline.lam) for x in values) < 1e-6
297         rows.append(dict(day=day,initial_soc=initial,shadow=baseline.lam,differences=values))
298     result['shadow_checks'] = rows
299     return result
300
301
302 def annual_accounts(output):
303     data = load_data(ROOT);dynamic = load_prices(ROOT,data.dates)
304     report = {};primary = {}
305     for name,(kind,config,physical,_) in designs().items():
306         with np.load(output/name+'/ledger.npz') as source:z = {k:source[k].copy() for k in
            ↪ source.files}
307         assert np.array_equal(z['d'],np.arange(31,365)),name
308         load,pv = data.load[z['d']],data.pv[z['d']]
309         assert np.array_equal(load,z['load']) and np.array_equal(pv,z['pv'])
310         p = (np.stack([dynamic.settle_price(int(day)) for day in z['d']])
            ↪ if kind in ('q42','q43') else np.tile(data.price,(334,1)))
311         m = z.get('m',z['q']);c,b,q = z['c'],z['b'],z['q']
312         need = load+c-b;used_pv = np.minimum(pv,need)
313         called = np.minimum(m,need-used_pv);emergency = need-used_pv-called
314         fee = p*q
315         if 'm' in z:
316             fee += p*(1.5*np.maximum(m-q,0)-.5*np.maximum(q-m,0))
317         cash = fee+physical.emergency_multiple*p*emergency
318         errors = {
319             'flow':float(np.max(np.abs(z['v']+z['a']+z['e']+b-load-c))),
320             'state_step':float(np.max(np.abs(z['soc_start']+physical.eta_c*c-b/physical.eta_
            ↪ d-z['soc_end']))),
321             'interday':float(np.max(np.abs(z['soc_start'][1:,0]-z['soc_end'][:-1,-1]))),
322             'cash':float(np.max(np.abs(cash-z['cash_cost']))),
323             'emergency':float(np.max(np.abs(emergency-z['e']))),
324             'simultaneous':float(np.max(np.minimum(c,b))),
325             'state_bounds':max(0.,float(physical.s_min-z['soc_end'].min()),float(z['soc_end']
            ↪ .max()-physical.s_max)),
326             'power_bounds':max(0.,float(c.max()-physical.power_energy),float(b.max()-physica
            ↪ l.power_energy)),
327         }
328     }
329     assert max(errors.values()) < 1e-6,(name,errors)
330     report[name] = dict(periods=q.size,max_errors=errors,cash_yuan=float(cash.sum()),
331                         terminal_soc=float(z['soc_end'][-1,-1]))
332     if name in ('q2/F_fast','q3/Z2_111','q4_2/known_day_ahead','q4_3/known_day_ahead'):
333         primary[name] = (z,p)
334     z,p = primary['q2/F_fast'];overlap = (z['e']>1e-6)&(z['c']>1e-6)
335     unchanged_need = np.maximum(z['load']-z['pv']-z['b']-z['q'],0)
336     overlap_report =
    ↪ dict(periods=int(overlap.sum()),emergency_kwh=float(z['e'][overlap].sum()),
337         reduced_gap_kwh=float((z['e']-unchanged_need)[overlap].sum()),
338         remaining_gap_kwh=float(unchanged_need[overlap].sum()))
339     repricing = {}
340     for first,last in
    ↪ (('q2/F_fast','q4_2/known_day_ahead'),('q3/Z2_111','q4_3/known_day_ahead')):
341         z,p0 = primary[first];future,p = primary[last]
342         weight = z['q']+5*z['e']
343         if 'm' in z:weight += 1.5*np.maximum(z['m']-z['q'],0)-.5*np.maximum(z['q']-z['m'],0)
344         original,repriced = float((p0*weight).sum()),float((p*weight).sum())
345         mean = float(334*((p.mean(axis=0)-p0[0])*weight.mean(axis=0)).sum())
346         covariance = float(((p-p.mean(axis=0))*(weight-weight.mean(axis=0))).sum())
347         assert abs(repriced-original-mean-covariance) < 1e-6
348         repricing[first] = dict(original=original,repriced=repriced,mean_component=mean,
349                                 covariance_component=covariance,reoptimized=float(future['cash_cost'].sum()))
350     return dict(trajectories=report,emergency_charging_overlap=overlap_report,repricing=repr
    ↪ icing)
351
352
353 def main():

```

```

354 parser = argparse.ArgumentParser(description=__doc__)
355 parser.add_argument('--scope',choices=('small','boundary','forecast','face','annual','all'],
    ↪ 1'),default='all')
356 parser.add_argument('--out',type=Path,default=ROOT/'output')
357 args = parser.parse_args()
358 selected = ('small','boundary','forecast','face','annual') if args.scope == 'all' else
    ↪ (args.scope,)
359 for scope in selected:
360     if scope == 'small': report = small_models()
361     elif scope == 'boundary': report = information_boundary()
362     elif scope == 'forecast': report = forecast_diagnostics()
363     elif scope == 'face': report = optimal_face_and_marginals(args.out)
364     else:
365         report = {'oracle_and_shadow':oracle_and_shadow(args.out),'annual_accounts':annu
    ↪ al_accounts(args.out)}
366         for name in ('q3/Z2_111','q4_3/known_day_ahead','q4_3/previous_day_forecast'):
367             report[name] = quantile_replay(args.out,name)
368         hashes = fingerprints()
369         hashes['verify.py'] = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
370         save(args.out/'checks'/f'{scope}.json',dict(passed=True,results=report,source_hashes
    ↪ =hashes))
371         print(json.dumps(dict(scope=scope,passed=True)),flush=True)
372
373
374 if __name__ == '__main__':
375     main()

```

export.py: 五份模板导出

```

1 """Export the five official templates from freshly computed numerical ledgers."""
2
3 import argparse
4 from copy import copy
5 from datetime import date, datetime, timedelta
6 import hashlib
7 import json
8 import math
9 from pathlib import Path
10 import re
11 import tempfile
12 from zipfile import ZipFile, ZIP_DEFLATED
13 import numpy as np
14 from openpyxl import load_workbook
15 from openpyxl.styles import Alignment, Border, Font, Side
16 from openpyxl.workbook.properties import CalcProperties
17 from run import ROOT, save
18 CELL_XML = re.compile(rb'(<c\b[^\>]*\br="([^\"]+)"[^\>]*>)(.*?)(</c>)', re.DOTALL)
19 VALUE_XML = re.compile(rb'<v(?:\s[^\>]*)?>.*?</v>|<v\s*/>', re.DOTALL)
20
21
22 def digest(path: Path) -> str:
23     return hashlib.sha256(path.read_bytes()).hexdigest()
24
25 def sequential_sum(values) -> float:
26     """Preserve the previous SUM cache's left-to-right binary64 addition."""
27     total = 0.0
28     for value in values:
29         total += float(value)
30     return total
31
32 def exact_numeric_xml(source: Path, output: Path, sheet_numbers: dict[str, dict[str, float]])
    ↪ -> None:
33     """Keep binary64 source numbers and supply known SUM caches in OOXML.

```

```

34
35 openpyxl normally serializes 16 significant digits and cannot calculate
36 formula caches. Only our explicitly written numeric cells and SUM totals
37 are patched. Formula expressions remain intact and Excel can recalculate
38 them after an edit. This is not a general-purpose formula evaluator.
39 """
40 with ZipFile(source) as original, ZipFile(output, "w") as target:
41     for entry in original.infolist():
42         data = original.read(entry.filename)
43         if entry.filename in sheet_numbers:
44             numbers = sheet_numbers[entry.filename]
45             seen = set()
46
47             def patch(match):
48                 address = match.group(2).decode("ascii")
49                 if address not in numbers:
50                     return match.group(0)
51                 if b't="inlineStr"' in match.group(1) or b't="s"' in match.group(1):
52                     raise ValueError(f"Expected numeric cell:
53                     ↪ {entry.filename}!{address}")
54                 seen.add(address)
55                 value = float(numbers[address])
56                 if not math.isfinite(value):
57                     raise ValueError(f"Non-finite output: {address}")
58                 text_value = str(int(value)) if value.is_integer() else repr(value)
59                 replacement = b"<v>" + text_value.encode("ascii") + b"</v>"
60                 body = match.group(3)
61                 body = VALUE_XML.sub(replacement, body) if VALUE_XML.search(body) else
62                 ↪ body + replacement
63                 return match.group(1) + body + match.group(4)
64
65             data = CELL_XML.sub(patch, data)
66             if seen != set(numbers):
67                 raise ValueError(f"Numeric cells missing in saved XML: {entry.filename}")
68             data = data.replace(b' t="n"', b'') # OOXML's default numeric type.
69             target.writestr(entry, data, compress_type=ZIP_DEFLATED, compresslevel=9)
70
71 def export_one(name: str, model: dict, template: Path, destination: Path) -> dict:
72     workbook = load_workbook(template)
73     numeric = {sheet.title: {} for sheet in workbook}
74
75     def put(sheet, row, column, value, number_format=None):
76         cell = sheet.cell(row, column, value)
77         if number_format is not None:
78             cell.number_format = number_format
79         if isinstance(value, (int, float)) and not isinstance(value, bool):
80             numeric[sheet.title][cell.coordinate] = float(value)
81         return cell
82
83     if model["kind"] == "q1":
84         if len(model["q"]) != 144 or len(model["storage"]) != 6:
85             raise ValueError("Q1 result dimensions changed")
86         plan, storage = workbook["计划购电量"], workbook["充放电量"]
87         # Preserve every official time label, including its shifted labels.
88         for i, value in enumerate(model["q"], 2):
89             put(plan, i, 2, value, "0.00000")
90         for i, row in enumerate(model["storage"], 2):
91             for j, value in enumerate(row, 2):
92                 put(storage, i, j, value, "0.00000")
93             put(storage, 2, 5, model["initial_soc"], "0.00000")
94             put(storage, 3, 5, model["terminal_soc"], "0.00000")
95             storage.column_dimensions["E"].width = 20
96     else:
97         dates = model["dates"]
98         if len(dates) != 334 or len(model["storage_rows"]) != 2004:

```

```

97         raise ValueError("A complete 334-day computed trajectory is required")
98 plan_specs = [(" 计划购电量", "q", "plan_fee")]
99 if model["kind"] == "amendment":
100     plan_specs.append((" 调整购电量", "m", "final_fee"))
101 for sheet_name, quantity, fee in plan_specs:
102     sheet = workbook[sheet_name]
103     if (sheet.max_row, sheet.max_column) != (335, 147):
104         raise ValueError("Official plan template dimensions changed")
105     for i, (day, values, cost) in enumerate(zip(dates, model[quantity], model[fee]),
106 ↪ 2):
107         if len(values) != 144 or sheet.cell(i, 1).value.date().isoformat() != day:
108             raise ValueError(f"Date/slot mismatch: {sheet_name} row {i}")
109         for j, value in enumerate(values, 2):
110             put(sheet, i, j, value, "0.00")
111         total = put(sheet, i, 146, f"=SUM(B{i}:EO{i})", "0.00")
112         numeric[sheet_name][total.coordinate] = sequential_sum(values)
113         put(sheet, i, 147, cost, "0.00")
114
115 storage = workbook[" 充放电量"]
116 block_styles = [[copy(storage.cell(r, c)._style) for c in range(1, 7)] for r in
117 ↪ range(2, 8)]
118 for i, row in enumerate(model["storage_rows"], 2):
119     for j, value in enumerate(row, 1):
120         cell = storage.cell(i, j)
121         if not cell.has_style and model["kind"] == "no_amendment":
122             cell._style = copy(block_styles[(i - 2) % 6][j - 1])
123         cell.value = None
124         if j == 1 and value is not None:
125             value = datetime.fromisoformat(value)
126         put(storage, i, j, value)
127         if j == 1:
128             cell.number_format = "yyyy-mm-dd"
129         elif j in (3, 4, 6):
130             cell.number_format = "0.00"
131         elif j == 5 and (i - 2) % 6 == 0:
132             cell.number_format = "h:mm"
133
134 emergency = workbook[" 紧急购电量"]
135 first_styles = [copy(emergency.cell(2, c)._style) for c in range(1, 4)]
136 for i, row in enumerate(model["emergency_rows"], 2):
137     for j, value in enumerate(row, 1):
138         cell = emergency.cell(i, j)
139         if not cell.has_style and model["kind"] == "no_amendment":
140             cell._style = copy(first_styles[j - 1])
141         cell.value = None
142         if j == 1 and value is not None:
143             value = datetime.fromisoformat(value)
144         put(emergency, i, j, value)
145         if j == 1:
146             cell.number_format = "yyyy-mm-dd"
147         elif j == 3:
148             cell.number_format = "0.00"
149
150 if model["kind"] == "no_amendment":
151     # These are the readability adjustments in the original Q2/4-2 builder.
152     for column, width in {"A": 14, "B": 19, "C": 16, "D": 16, "F": 18}.items():
153         storage.column_dimensions[column].width = width
154     for column, width in {"A": 14, "B": 21, "C": 17}.items():
155         emergency.column_dimensions[column].width = width
156     line = Side(style="thin", color="FFD9D9D9")
157     for sheet in (storage, emergency):
158         for row in sheet.iter_rows(min_row=2):
159             sheet.row_dimensions[row[0].row].height = 21
160             for cell in row:
161                 cell.font = Font(name="SimSun", size=11, color="FF000000")

```

```

160         cell.border = Border(left=line, right=line, top=line, bottom=line)
161         right = (sheet is storage and cell.column in (3, 4, 6)) or (sheet is
        ↪ emergency and cell.column == 3)
162         cell.alignment = Alignment(horizontal="right" if right else "center",
        ↪ vertical="center")
163
164     workbook.calculation = CalcProperties(calcMode="auto", fullCalcOnLoad=True,
        ↪ forceFullCalc=True)
165     archive_numbers = {f"xl/worksheets/sheet{i}.xml": numeric[s.title] for i, s in
        ↪ enumerate(workbook, 1)}
166     with tempfile.TemporaryDirectory(prefix="workbook_export_", dir=destination.parent) as
        ↪ temporary:
167         intermediate = Path(temporary) / "openpyxl.xlsx"
168         finished = Path(temporary) / name
169         workbook.save(intermediate)
170         workbook.close()
171         exact_numeric_xml(intermediate, finished, archive_numbers)
172         finished.replace(destination)
173     return {"name": name, "output_sha256": digest(destination), "bytes":
        ↪ destination.stat().st_size,
174           "explicit_numeric_cells_and_formula_caches": sum(map(len, numeric.values()))}
175
176
177 def interval(first,last):
178     return f'{first//6}:{10*(first%6):02d}-{last//6}:{10*(last%6):02d}'
179
180
181 def annual_input(path,amendment):
182     with np.load(path,allow_pickle=False) as source:
183         z = {key:source[key].copy() for key in source.files}
184         assert np.array_equal(z['d'],np.arange(31,365))
185         dates = [(date(2025,1,1)+timedelta(days=int(d))).isoformat() for d in z['d']]
186         storage,emergency = [],[]
187         for i,day in enumerate(dates):
188             for block in range(6):
189                 section = slice(block*24,(block+1)*24)
190                 storage.append([day if block == 0 else None,interval(block*24,(block+1)*24),
191                               float(z['c'][i,section].sum()),float(z['b'][i,section].sum()),
192                               0 if block == 0 else '24:00' if block == 1 else None,
193                               float(z['soc_start'][i,0]) if block == 0 else float(z['soc_end'][i,-1]) if
        ↪ block == 1 else None])
194             first = None; pieces = []
195             for t in range(145):
196                 active = t < 144 and z['e'][i,t] > 1e-6
197                 if active and first is None: first = t
198                 elif not active and first is not None:
199                     pieces.append([day if not pieces else
        ↪ None,interval(first,t),math.fsum(z['e'][i,first:t])])
200                 first = None
201             emergency.extend(pieces or [[day,'无',0.]])
202     plan_fee = z['plan_cost'] if amendment else z['cash_cost']
203     model = dict(kind='amendment' if amendment else
        ↪ 'no_amendment',dates=dates,q=z['q'].tolist(),
204               plan_fee=[math.fsum(row) for row in
        ↪ plan_fee],storage_rows=storage,emergency_rows=emergency)
205     if amendment: model.update(m=z['m'].tolist(),final_fee=[math.fsum(row) for row in
        ↪ z['cash_cost']])
206     return model
207
208
209 def main():
210     parser = argparse.ArgumentParser(description=__doc__)
211     parser.add_argument('--out',type=Path,default=ROOT/'output')
212     parser.add_argument('--destination',type=Path,default=ROOT/'result_tables')
213     args = parser.parse_args();args.destination.mkdir(parents=True,exist_ok=True)

```

```

214     with np.load(args.out/'q1/base/solution.npz') as z:
215         model = dict(kind='q1',q=z['g'].tolist(),initial_soc=float(z['soc'][0]),
216             terminal_soc=float(z['soc'][-1]),storage=[[float(z[k][b*24:(b+1)*24].sum())
217                 for k in ('c','d')] for b in range(6)])
218     models = {'result1.xlsx':model}
219     selected = json.loads((args.out/'q3/selection.json').read_text('utf-8'))['selected']
220     for name,case,amend in
221         ↪ [ ('result2.xlsx','q2/F_fast',False),('result3.xlsx',selected,True),
222             ('result4-2.xlsx','q4_2/known_day_ahead',False),('result4-3.xlsx','q4_3/known_day_ah
223             ↪ ead',True)]:
224         models[name] = annual_input(args.out/case/'ledger.npz',amend)
225     report = [export_one(name,model,ROOT/'templates'/name,args.destination/name) for
226         ↪ name,model in models.items()]
227     save(args.destination/'export_report.json',dict(source='Freshly computed output
228         ↪ ledgers',workbooks=report))
229     print(json.dumps(dict(exported=len(report),destination=str(args.destination)),ensure_asc
230         ↪ ii=False))
231
232 if __name__ == '__main__':
233     main()

```

configs/q1_v2.json

```

1 {
2   "task_id": "C-Q1-v2",
3   "source_workbook": "problem/attachments/C 题/附件 1.xlsx",
4   "source_sheet": "Sheet1",
5   "source_sha256": "66b87134f5ecccd68184d3539bb1293ef039f9e0fdd955a589b9bfa7f227c377",
6   "step_hours": 0.16666666666666666,
7   "capacity_kwh": 12000.0,
8   "soc_min_kwh": 1200.0,
9   "soc_max_kwh": 10800.0,
10  "initial_soc_kwh": 6000.0,
11  "max_charge_power_kw": 5000.0,
12  "max_discharge_power_kw": 5000.0,
13  "eta_charge": 0.9,
14  "eta_discharge": 0.9,
15  "time_interpretation": "right_endpoint_representative",
16  "power_measurement_side": "microgrid_bus",
17  "pv_curtailment_allowed": true,
18  "grid_export_allowed": false,
19  "battery_dump_allowed": false,
20  "terminal_condition": "equal_initial",
21  "model": "exact_state_increment_epigraph_lp",
22  "selection_rule": "minimum_cost_then_minimum_bus_throughput",
23  "solver": "scipy.optimize.linprog/HiGHS dual simplex",
24  "primal_feasibility_tolerance": 1e-9,
25  "dual_feasibility_tolerance": 1e-9,
26  "validation_tolerance_kwh": 1e-6,
27  "status": "conditional_research_case_not_team_acceptance"
28 }

```

requirements.txt

```

1 numpy==2.3.5
2 scipy==1.18.1
3 openpyxl==3.1.5

```